

Elements of Programming Interviews in Python

The Insiders' Guide

Adnan Aziz

Tsung-Hsien Lee

Amit Prakash

This document is a sampling of our book, Elements of Programming Interviews in Python (EPI). Its purpose is to provide examples of EPI's organization, content, style, topics, and quality. The sampler focuses solely on problems; in particular, it does not include three chapters on the nontechnical aspects of interviewing. We'd love to hear from you—we're especially interested in your suggestions as to where the exposition can be improved, as well as any insights into interviewing trends you may have.

You can buy EPI Python at [Amazon.com](https://www.amazon.com/dp/1491998923) (paperback).

<http://ElementsOfProgrammingInterviews.com>

Adnan Aziz is a Research Scientist at Facebook. Previously, he was a professor at the Department of Electrical and Computer Engineering at The University of Texas at Austin, where he conducted research and taught classes in applied algorithms. He received his Ph.D. from The University of California at Berkeley; his undergraduate degree is from Indian Institutes of Technology Kanpur. He has worked at Google, Qualcomm, IBM, and several software startups. When not designing algorithms, he plays with his children, Laila, Imran, and Omar.

Tsung-Hsien Lee is a Senior Software Engineer at Uber. Previously, he worked as a Software Engineer at Google and as Software Engineer Intern at Facebook. He received both his M.S. and undergraduate degrees from National Tsing Hua University. He has a passion for designing and implementing algorithms. He likes to apply algorithms to every aspect of his life. He takes special pride in helping to organize Google Code Jam 2014 and 2015.

Amit Prakash is a co-founder and CTO of ThoughtSpot, a Silicon Valley startup. Previously, he was a Member of the Technical Staff at Google, where he worked primarily on machine learning problems that arise in the context of online advertising. Before that he worked at Microsoft in the web search team. He received his Ph.D. from The University of Texas at Austin; his undergraduate degree is from Indian Institutes of Technology Kanpur. When he is not improving business intelligence, he indulges in his passion for puzzles, movies, travel, and adventures with Nidhi and Aanya.

Elements of Programming Interviews: The Insiders' Guide

by Adnan Aziz, Tsung-Hsien Lee, and Amit Prakash

Copyright © 2017 Adnan Aziz, Tsung-Hsien Lee, and Amit Prakash. All rights reserved.

No part of this publication may be reproduced, stored in a retrieval system, or transmitted, in any form, or by any means, electronic, mechanical, photocopying, recording, or otherwise, without the prior consent of the authors.

The views and opinions expressed in this work are those of the authors and do not necessarily reflect the official policy or position of their employers.

We typeset this book using $\text{\LaTeX}$ and the Memoir class. We used TikZ to draw figures. Allan Ytac created the cover, based on a design brief we provided.

The companion website for the book includes contact information and a list of known errors for each version of the book. If you come across an error or an improvement, please let us know.

Website: <http://elementsofprogramminginterviews.com>

Distributed under the Attribution-NonCommercial-NoDerivs 3.0 License

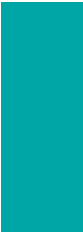


Table of Contents

I	Data Structures and Algorithms	1
1	Primitive Types	2
1.1	Computing the parity of a word	3
2	Arrays	7
2.1	The Dutch national flag problem	9
3	Strings	13
3.1	Interconvert strings and integers	14
3.2	Base conversion	15
4	Linked Lists	17
4.1	Test for cyclicity	19
5	Stacks and Queues	21
5.1	Implement a stack with max API	22
5.2	Compute binary tree nodes in order of increasing depth	26
6	Binary Trees	28
6.1	Test if a binary tree is height-balanced	30
7	Heaps	33
7.1	Merge sorted files	35
8	Searching	37
8.1	Search a sorted array for first occurrence of k	39
9	Hash Tables	42
9.1	Is an anonymous letter constructible?	46
10	Sorting	48
10.1	Compute the intersection of two sorted arrays	50
10.2	Render a calendar	51
11	Binary Search Trees	54

11.1	Test if a binary tree satisfies the BST property	56
12	Recursion	59
12.1	Generate the power set	60
13	Dynamic Programming	63
13.1	Count the number of ways to traverse a 2D array	65
14	Greedy Algorithms and Invariants	69
14.1	The 3-sum problem	71
15	Graphs	73
15.1	Paint a Boolean matrix	76
16	Parallel Computing	78
16.1	Implement a Timer class	80
II	Domain Specific Problems	81
17	Design Problems	82
17.1	Design a system for detecting copyright infringement	83
18	Language Questions	85
18.1	Closure	85
18.2	Shallow and deep copy	85
19	Object-Oriented Design	87
19.1	Creational Patterns	87
20	Common Tools	89
20.1	Merging in a version control system	89
III	The Honors Class	93
21	Honors Class	94
21.1	Compute the greatest common divisor 	95
21.2	Compute the maximum product of all entries but one 	96
IV	Notation and Index	99
	Notation	100
	Index of Terms	102

Part I

Data Structures and Algorithms

Primitive Types

Representation is the essence of programming.

— “The Mythical Man Month,”

F. P. BROOKS, 1975

A program updates variables in memory according to its instructions. Variables come in types—a type is a classification of data that spells out possible values for that type and the operations that can be performed on it. A type can be provided by the language or defined by the programmer. In Python everything is an object—this includes Booleans, integers, characters, etc.

Primitive types boot camp

Writing a program to count the number of bits that are set to 1 in an integer is a good way to get up to speed with primitive types. The following program tests bits one-at-a-time starting with the least-significant bit. It illustrates shifting and masking; it also shows how to avoid hard-coding the size of the integer word.

```
def count_bits(x):
    num_bits = 0
    while x:
        num_bits += x & 1
        x >>= 1
    return num_bits
```

Since we perform $O(1)$ computation per bit, the time complexity is $O(n)$, where n is the number of bits needed to represent the integer, e.g., 4 bits are needed to represent 12, which is $(1100)_2$ in binary. The techniques in [Solution 1.1 on the facing page](#), which is concerned with counting the number of bits modulo 2, i.e., the parity, can be used to improve the performance of the program given above.

Know your primitive types

Python has a number of build-in types: numerics (e.g., integer), sequences (e.g., list), mappings (e.g., dict), as well as classes, instances and exceptions. All instances of these types are objects.

Integers in Python3 are unbounded—the maximum integer representable is a function of the available memory. The constant `sys.maxsize` can be used to find the word-size; specifically, it's the maximum value integer that can be stored in the word, e.g., $2^{63} - 1$ on a 64-bit machine. Bounds on floats are specified in `sys.float_info`.

- Be very familiar with the bit-wise operators such as `6&4`, `1|2`, `8>>1`, `-16>>>2`, `1<<10`, `~0`, `15^x`. Negative numbers are treated as their 2's complement value. (There is no concept of an unsigned shift in Python, since integers have infinite precision.)

Be very comfortable with the **bitwise operators**, particularly XOR.

Understand how to use **masks** and create them in an **machine independent** way.

Know fast ways to **clear the lowermost set bit** (and by extension, set the lowermost 0, get its index, etc.)

Understand **signedness** and its implications to **shifting**.

Consider using a **cache** to accelerate operations by using it to brute-force small inputs.

Be aware that **commutativity** and **associativity** can be used to perform operations in **parallel** and **reorder** operations.

Table 1.1: Top Tips for Primitive Types

- The key methods for numeric types are `abs(-34.5)`, `math.ceil(2.17)`, `math.floor(3.14)`, `min(x, -4)`, `max(3.14, y)`, `pow(2.71, 3.14)` (alternately, `2.71 ** 3.14`), and `math.sqrt(225)`.
- Know how to interconvert integers and strings, e.g., `str(42)`, `int('42')`, floats and strings, e.g., `str(3.14)`, `float('3.14')`.
- Unlike integers, floats are not infinite precision, and it's convenient to refer to infinity as `float('inf')` and `float('-inf')`. These values are comparable to integers, and can be used to create pseudo max-int and pseudo min-int.
- When comparing floating point values consider using `math.isclose()`—it is robust, e.g., when comparing very large values, and can handle both absolute and relative differences.
- The key methods in `random` are `random.randrange(28)`, `random.randint(8, 16)`, `random.random()`, `random.shuffle(A)`, and `random.choice(A)`.

1.1 COMPUTING THE PARITY OF A WORD

The parity of a binary word is 1 if the number of 1s in the word is odd; otherwise, it is 0. For example, the parity of 1011 is 1, and the parity of 10001000 is 0. **Parity checks** are used to detect **single bit errors** in **data storage and communication**. It is fairly straightforward to write code that computes the parity of a single 64-bit word.

How would you compute the parity of a very large number of 64-bit words?

Hint: Use a lookup table, but don't use 2^{64} entries!

Solution: The brute-force algorithm iteratively tests the value of each bit while tracking the number of 1s seen so far. Since we only care if the number of 1s is even or odd, we can store the number modulo 2.

```
def parity(x):
    result = 0
    while x:
        result ^= x & 1
```

```
    x >>= 1
return result
```

The time complexity is $O(n)$, where n is the word size.

We will now describe several algorithms for parity computation that are superior to the brute-force algorithm.

The first improvement is based on erasing the lowest set bit in a word in a single operation, thereby improving performance in the best- and average-cases. Specifically, the expression $y = x \& \sim(x - 1)$, where $\&$ is the bitwise-AND operator and $\sim$ is the bitwise complement operator. The variable y is 1 at exactly the lowest bit of x that is 1; all other bits in y are 0. For example, if $x = (00101100)_2$, then $x - 1 = (00101011)_2$, $\sim(x - 1) = (11010100)_2$, and $y = (00101100)_2 \& (11010100)_2 = (00000100)_2$. This calculation is robust—it is correct for unsigned and two's-complement representations. Consequently, this bit may be removed from x by computing $x \oplus y$, where $\oplus$ is the bitwise-XOR function. The time complexity is $O(s)$, where s is the number of bits set to 1 in x .

```
def parity(x):
    result = 0
    while x:
        result ^= 1
        x &= x - 1 # Drops the lowest set bit of x.
    return result
```

Let k be the number of bits set to 1 in a particular word. (For example, for 10001010, $k = 3$.) Then time complexity of the algorithm above is $O(k)$.

The fact that $x \& \sim(x - 1)$ isolates the lowest bit that is 1 in x is important enough that you should memorize it. However, it is also fairly easy to derive. First, suppose x is not 0, i.e., it has a bit that is one. Subtracting one from x changes the rightmost bit to zero and sets all the lower bits to one (if you add one now, you get the original value back). The effect is to mask out the rightmost one. Therefore $x \& \sim(x - 1)$ has a single bit set to one, namely, the rightmost 1 in x . Now suppose x is 0. Subtracting one from x underflows, resulting in a word in which all bits are set to one. Again, $x \& \sim(x - 1)$ is 0.

A similar derivation shows that $x \& (x - 1)$ replaces the lowest bit that is 1 with 0. For example, if $x = (00101100)_2$, then $x - 1 = (00101011)_2$, so $x \& (x - 1) = (00101100)_2 \& (00101011)_2 = (00101000)_2$. This fact can also be very useful.

We now consider a qualitatively different approach. The problem statement refers to computing the parity for a very large number of words. When you have to perform a large number of parity computations, and, more generally, any kind of bit fiddling computations, two keys to performance are processing multiple bits at a time and caching results in an array-based lookup table.

First we demonstrate caching. Clearly, we cannot cache the parity of every 64-bit integer—we would need 2^{64} bits of storage, which is of the order of ten trillion exabytes. However, when computing the parity of a collection of bits, it does not matter how we group those bits, i.e., the computation is associative. Therefore, we can compute the parity of a 64-bit integer by grouping its bits into four nonoverlapping 16 bit subwords, computing the parity of each subword, and then computing the parity of these four subresults. We choose 16 since $2^{16} = 65536$ is relatively small,

which makes it feasible to cache the parity of all 16-bit words using an array. Furthermore, since 16 evenly divides 64, the code is simpler than if we were, for example, to use 10 bit subwords.

We illustrate the approach with a lookup table for 2-bit words. The cache is $\langle 0, 1, 1, 0 \rangle$ —these are the parities of $(00), (01), (10), (11)$, respectively. To compute the parity of (11001010) we would compute the parities of $(11), (00), (10), (10)$. By table lookup we see these are $0, 0, 1, 1$, respectively, so the final result is the parity of $0, 0, 1, 1$ which is 0.

To lookup the parity of the first two bits in (11101010) , we right shift by 6, to get (00000011) , and use this as an index into the cache. To lookup the parity of the next two bits, i.e., (10) , we right shift by 4, to get (10) in the two least-significant bit places. The right shift does not remove the leading (11) —it results in (00001110) . We cannot index the cache with this, it leads to an out-of-bounds access. To get the last two bits after the right shift by 4, we bitwise-AND (00001110) with (00000011) (this is the “mask” used to extract the last 2 bits). The result is (00000010) . Similar masking is needed for the two other 2-bit lookups.

```
def parity(x):
    MASK_SIZE = 16
    BIT_MASK = 0xFFFF
    return (PRECOMPUTED_PARITY[x >> (3 * MASK_SIZE)] ^
            PRECOMPUTED_PARITY[(x >> (2 * MASK_SIZE)) & BIT_MASK] ^
            PRECOMPUTED_PARITY[(x >> MASK_SIZE) & BIT_MASK] ^
            PRECOMPUTED_PARITY[x & BIT_MASK])
```

The time complexity is a function of the size of the keys used to index the lookup table. Let L be the width of the words for which we cache the results, and n the word size. Since there are n/L terms, the time complexity is $O(n/L)$, assuming word-level operations, such as shifting, take $O(1)$ time. (This does not include the time for initialization of the lookup table.)

We can improve on the $O(n)$ worst-case time complexity of the previous algorithms by exploiting some simple properties of parity. Specifically, the XOR of two bits is defined to be 0 if both bits are 0 or both bits are 1; otherwise it is 1. XOR has the property of being associative, i.e., it does not matter how we group bits, as well as commutative, i.e., the order in which we perform the XORs does not change the result. The XOR of a group of bits is its parity. We can exploit this fact to use the CPU’s word-level XOR instruction to process multiple bits at a time.

For example, the parity of $\langle b_{63}, b_{62}, \dots, b_3, b_2, b_1, b_0 \rangle$ equals the parity of the XOR of $\langle b_{63}, b_{62}, \dots, b_{32} \rangle$ and $\langle b_{31}, b_{30}, \dots, b_0 \rangle$. The XOR of these two 32-bit values can be computed with a single shift and a single 32-bit XOR instruction. We repeat the same operation on 32-, 16-, 8-, 4-, 2-, and 1-bit operands to get the final result. Note that the leading bits are not meaningful, and we have to explicitly extract the result from the least-significant bit.

We illustrate the approach with an 8-bit word. The parity of (11010111) is the same as the parity of (1101) XORed with (0111) , i.e., of (1010) . This in turn is the same as the parity of (10) XORed with (10) , i.e., of (00) . The final result is the XOR of (0) with (0) , i.e., 0. Note that the first XOR yields (11011010) , and only the last 4 bits are relevant going forward. The second XOR yields (11101100) , and only the last 2 bits are relevant. The third XOR yields (10011010) . The last bit is the result, and to extract it we have to bitwise-AND with (00000001) .

```
def parity(x):
    x ^= x >> 32
    x ^= x >> 16
```

```
x ^= x >> 8
x ^= x >> 4
x ^= x >> 2
x ^= x >> 1
return x & 0x1
```

The time complexity is $O(\log n)$, where n is the word size.

Note that we can combine caching with word-level operations, e.g., by doing a lookup in the XOR-based approach once we get to 16 bits.

The actual runtimes depend on the input data, e.g., the refinement of the brute-force algorithm is very fast on sparse inputs. However, for random inputs, the refinement of the brute-force is roughly 20% faster than the brute-force algorithm. The table-based approach is four times faster still, and using associativity reduces run time by another factor of two.

Variant: Write expressions that use bitwise operators, equality checks, and Boolean operators to do the following in $O(1)$ time.

- Right propagate the rightmost set bit in x , e.g., turns $(01010000)_2$ to $(01011111)_2$.
- Compute x modulo a power of two, e.g., returns 13 for $77 \bmod 64$.
- Test if x is a power of 2, i.e., evaluates to true for $x = 1, 2, 4, 8, \dots$, false for all other values.

Arrays

The machine can alter the scanned symbol and its behavior is in part determined by that symbol, but the symbols on the tape elsewhere do not affect the behavior of the machine.

— “Intelligent Machinery,”

A. M. TURING, 1948

The simplest data structure is the array, which is a contiguous block of memory. It is usually used to represent sequences. Given an array A , $A[i]$ denotes the $(i + 1)$ th object stored in the array. Retrieving and updating $A[i]$ takes $O(1)$ time. Insertion into a full array can be handled by resizing, i.e., allocating a new array with additional memory and copying over the entries from the original array. This increases the worst-case time of insertion, but if the new array has, for example, a constant factor larger than the original array, the average time for insertion is constant since resizing is infrequent. Deleting an element from an array entails moving all successive elements one over to the left to fill the vacated space. For example, if the array is $\langle 2, 3, 5, 7, 9, 11, 13, 17 \rangle$, then deleting the element at index 4 results in the array $\langle 2, 3, 5, 7, 11, 13, 17, 0 \rangle$. (We do not care about the last value.) The time complexity to delete the element at index i from an array of length n is $O(n - i)$. The same is true for inserting a new element (as opposed to updating an existing entry).

Array boot camp

The following problem gives good insight into working with arrays: Your input is an array of integers, and you have to reorder its entries so that the even entries appear first. This is easy if you use $O(n)$ space, where n is the length of the array. However, you are required to solve it without allocating additional storage.

When working with arrays you should take advantage of the fact that you can operate efficiently on both ends. For this problem, we can partition the array into three subarrays: Even, Unclassified, and Odd, appearing in that order. Initially Even and Odd are empty, and Unclassified is the entire array. We iterate through Unclassified, moving its elements to the boundaries of the Even and Odd subarrays via swaps, thereby expanding Even and Odd, and shrinking Unclassified.

```
def even_odd(A):
    next_even, next_odd = 0, len(A) - 1
    while next_even < next_odd:
        if A[next_even] % 2 == 0:
            next_even += 1
        else:
            A[next_even], A[next_odd] = A[next_odd], A[next_even]
            next_odd -= 1
```

The additional space complexity is clearly $O(1)$ —a couple of variables that hold indices, and a temporary variable for performing the swap. We do a constant amount of processing per entry, so the time complexity is $O(n)$.

Array problems often have simple brute-force solutions that use $O(n)$ space, but there are subtler solutions that use the array itself to reduce space complexity to $O(1)$. Filling an array from the front is slow, so see if it's possible to write values from the back. Instead of deleting an entry (which requires moving all entries to its right), consider overwriting it. When dealing with integers encoded by an array consider processing the digits from the back of the array. Alternately, reverse the array so the least-significant digit is the first entry. Be comfortable with writing code that operates on subarrays. It's incredibly easy to make off-by-1 errors when operating on arrays—reading past the last element of an array is a common error which has catastrophic consequences. Don't worry about preserving the integrity of the array (sortedness, keeping equal entries together, etc.) until it is time to return. An array can serve as a good data structure when you know the distribution of the elements in advance. For example, a Boolean array of length W is a good choice for representing a subset of $\{0, 1, \dots, W - 1\}$. (When using a Boolean array to represent a subset of $\{1, 2, 3, \dots, n\}$, allocate an array of size $n + 1$ to simplify indexing.) . When operating on 2D arrays, use parallel logic for rows and for columns. Sometimes it's easier to simulate the specification, than to analytically solve for the result. For example, rather than writing a formula for the i-th entry in the spiral order for an $n \times n$ matrix, just compute the output from the beginning.

Table 2.1: Top Tips for Arrays

Know your array libraries

Arrays in Python are provided by the list type. (The tuple type is very similar to the list type, with the constraint that it is immutable.) The key property of a list is that it is dynamically-resized, i.e., there's no bound as to how many values can be added to it. In the same way, values can be deleted and inserted at arbitrary locations.

- Know the syntax for instantiating a list, e.g., `[3, 5, 7, 11]`, `[1] + [0] * 10`, `list(range(100))`. (List comprehension, described later, is also a powerful tool for instantiating arrays.)
- The basic operations are `len(A)`, `A.append(42)`, `A.remove(2)`, and `A.insert(3, 28)`.
- Know how to instantiate a 2D array, e.g., `[[1, 2, 4], [3, 5, 7, 9], [13]]`.

- Checking if a value is present in an array is as simple as `a in A`. (This operation has $O(n)$ time complexity, where n is the length of the array.)
- Understand how copy works, i.e., the difference between `B = A` and `B = list(A)`. Understand what a deep copy is, and how it differs from a shallow copy, i.e., how `copy.copy(A)` differs from `copy.deepcopy(A)`.
- Key methods for list include `min(A)`, `max(A)`, binary search for sorted lists (`bisect.bisect(A, 6)`, `bisect.bisect_left(A, 6)`, and `bisect.bisect_right(A, 6)`), `A.reverse()` (in-place), `reversed(A)` (returns an iterator), `A.sort()` (in-place), `sorted(A)` (returns a copy), `del A[i]` (deletes the i -th element), and `del A[i:j]` (removes the slice).
- Slicing is a very succinct way of manipulating arrays. It can be viewed as a generalization of indexing: the most general form of slice is `A[i:j:k]`, with all of i , j , and k being optional. Let `A = [1, 6, 3, 4, 5, 2, 7]`. Here are some examples of slicing: `A[2:4]` is `[3, 4]`, `A[2:]` is `[3, 4, 5, 2, 7]`, `A[:4]` is `[1, 6, 3, 4]`, `A[:-1]` is `[1, 6, 3, 4, 5, 2]`, `A[-3:]` is `[5, 2, 7]`, `A[-3:-1]` is `[5, 2]`, `A[1:5:2]` is `[6, 4]`, `A[5:1:-2]` is `[2, 4]`, and `A[::-1]` is `[7, 2, 5, 4, 3, 6, 1]` (reverses list). Slicing can also be used to rotate a list: `A[k:] + A[:k]` rotates `A` by k to the left. It can also be used to create a copy: `B = A[:]` does a (shallow) copy of `A` into `B`.
- Python provides a feature called list comprehension that is a succinct way to create lists. A list comprehension consists of (1.) an input sequence, (2.) an iterator over the input sequence, (3.) a logical condition over the iterator (this is optional), and (4.) an expression that yields the elements of the derived list. For example, `[x**2 for a in range(6)]` yields `[1, 4, 9, 16, 25]`, and `[x**2 for a in range(6) if x % 2 == 0]` yields `[4, 16]`.

Although list comprehensions can always be rewritten using `map()`, `filter()`, and `lambdas`, they are clearer to read, in large part because they do not need lambdas.

List comprehension supports multiple levels of looping. This can be used to create the product of sets, e.g., if `A = [1, 3, 5]` and `B = ['a', 'b']`, then `[(x, y) for x in A for y in B]` creates `[(1, 'a'), (1, 'b'), (3, 'a'), (3, 'b'), (5, 'a'), (5, 'b')]`. It can also be used to convert a 2D list to a 1D list, e.g., if `M = [['a', 'b', 'c'], ['d', 'e', 'f']]`, `x for row in M for x in row` creates `['a', 'b', 'c', 'd', 'e', 'f']`. Two levels of looping also allow for iterating over each entry in a 2D list, e.g., if `A = [[1, 2, 3], [4, 5, 6]]` then `[x**2 for x in row] for row in M` yields `[[1, 4, 9], [16, 25, 36]]`.

As a general rule, it is best to avoid more than two nested comprehensions, and use conventional nested for loops—the indentation makes it easier to read the program.

Finally, sets and dictionaries also support list comprehensions, with the same benefits.

2.1 THE DUTCH NATIONAL FLAG PROBLEM

The quicksort algorithm for sorting arrays proceeds recursively—it selects an element (the “pivot”), reorders the array to make all the elements less than or equal to the pivot appear first, followed by all the elements greater than the pivot. The two subarrays are then sorted recursively.

Implemented naively, quicksort has large run times and deep function call stacks on arrays with many duplicates because the subarrays may differ greatly in size. One solution is to reorder the array so that all elements less than the pivot appear first, followed by elements equal to the pivot,

followed by elements greater than the pivot. This is known as Dutch national flag partitioning, because the Dutch national flag consists of three horizontal bands, each in a different color.

As an example, assuming that black precedes white and white precedes gray, Figure 2.1(b) is a valid partitioning for Figure 2.1(a). If gray precedes black and black precedes white, Figure 2.1(c) is a valid partitioning for Figure 2.1(a).

Generalizing, suppose $A = \langle 0, 1, 2, 0, 2, 1, 1 \rangle$, and the pivot index is 3. Then $A[3] = 0$, so $\langle 0, 0, 1, 2, 2, 1, 1 \rangle$ is a valid partitioning. For the same array, if the pivot index is 2, then $A[2] = 2$, so the arrays $\langle 0, 1, 0, 1, 1, 2, 2 \rangle$ as well as $\langle 0, 0, 1, 1, 1, 2, 2 \rangle$ are valid partitionings.

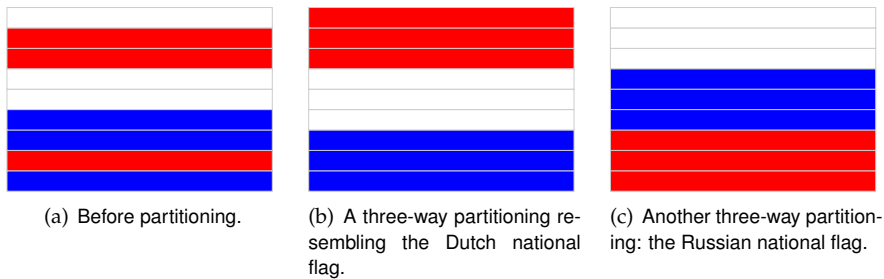


Figure 2.1: Illustrating the Dutch national flag problem.

Write a program that takes an array A and an index i into A , and rearranges the elements such that all elements less than $A[i]$ (the “pivot”) appear first, followed by elements equal to the pivot, followed by elements greater than the pivot.

Hint: Think about the partition step in quicksort.

Solution: The problem is trivial to solve with $O(n)$ additional space, where n is the length of A . We form three lists, namely, elements less than the pivot, elements equal to the pivot, and elements greater than the pivot. Consequently, we write these values into A . The time complexity is $O(n)$.

We can avoid using $O(n)$ additional space at the cost of increased time complexity as follows. In the first stage, we iterate through A starting from index 0, then index 1, etc. In each iteration, we seek an element smaller than the pivot—as soon as we find it, we move it to the subarray of smaller elements via an exchange. This moves all the elements less than the pivot to the start of the array. The second stage is similar to the first one, the difference being that we move elements greater than the pivot to the end of the array. Code illustrating this approach is shown below.

```
RED, WHITE, BLUE = range(3)

def dutch_flag_partition(pivot_index, A):
    pivot = A[pivot_index]
    # First pass: group elements smaller than pivot.
    for i in range(len(A)):
        # Look for a smaller element.
        for j in range(i + 1, len(A)):
            if A[j] < pivot:
                A[i], A[j] = A[j], A[i]
                break
    # Second pass: group elements larger than pivot.
```

```

for i in reversed(range(len(A))):
    if A[i] < pivot:
        break
    # Look for a larger element. Stop when we reach an element less than
    # pivot, since first pass has moved them to the start of A.
    for j in reversed(range(i)):
        if A[j] > pivot:
            A[i], A[j] = A[j], A[i]
            break

```

The additional space complexity is now $O(1)$, but the time complexity is $O(n^2)$, e.g., if $i = n/2$ and all elements before i are greater than $A[i]$, and all elements after i are less than $A[i]$. Intuitively, this approach has bad time complexity because in the first pass when searching for each additional element smaller than the pivot we start from the beginning. However, there is no reason to start from so far back—we can begin from the last location we advanced to. (Similar comments hold for the second pass.)

To improve time complexity, we make a single pass and move all the elements less than the pivot to the beginning. In the second pass we move the larger elements to the end. It is easy to perform each pass in a single iteration, moving out-of-place elements as soon as they are discovered.

```
RED, WHITE, BLUE = range(3)
```

```

def dutch_flag_partition(pivot_index, A):
    pivot = A[pivot_index]
    # First pass: group elements smaller than pivot.
    smaller = 0
    for i in range(len(A)):
        if A[i] < pivot:
            A[i], A[smaller] = A[smaller], A[i]
            smaller += 1
    # Second pass: group elements larger than pivot.
    larger = len(A) - 1
    for i in reversed(range(len(A))):
        if A[i] < pivot:
            break
        elif A[i] > pivot:
            A[i], A[larger] = A[larger], A[i]
            larger -= 1

```

The time complexity is $O(n)$ and the space complexity is $O(1)$.

The algorithm we now present is similar to the one sketched above. The main difference is that it performs classification into elements less than, equal to, and greater than the pivot in a single pass. This reduces runtime, at the cost of a trickier implementation. We do this by maintaining four subarrays: *bottom* (elements less than pivot), *middle* (elements equal to pivot), *unclassified*, and *top* (elements greater than pivot). Initially, all elements are in *unclassified*. We iterate through elements in *unclassified*, and move elements into one of *bottom*, *middle*, and *top* groups according to the relative order between the incoming unclassified element and the pivot.

As a concrete example, suppose the array is currently $A = \langle -3, 0, -1, 1, 1, ?, ?, ?, 4, 2 \rangle$, where the pivot is 1 and ? denotes unclassified elements. There are three possibilities for the first unclassified

element, $A[5]$.

- $A[5]$ is less than the pivot, e.g., $A[5] = -5$. We exchange it with the first 1, i.e., the new array is $\langle -3, 0, -1, -5, 1, 1, ?, ?, 4, 2 \rangle$.
- $A[5]$ is equal to the pivot, i.e., $A[5] = 1$. We do not need to move it, we just advance to the next unclassified element, i.e., the array is $\langle -3, 0, -1, 1, 1, 1, ?, ?, 4, 2 \rangle$.
- $A[5]$ is greater than the pivot, e.g., $A[5] = 3$. We exchange it with the last unclassified element, i.e., the new array is $\langle -3, 0, -1, 1, 1, ?, ?, 3, 4, 2 \rangle$.

Note how the number of unclassified elements reduces by one in each case.

```
RED, WHITE, BLUE = range(3)
```

```
def dutch_flag_partition(pivot_index, A):
    pivot = A[pivot_index]
    # Keep the following invariants during partitioning:
    # bottom group: A[:smaller].
    # middle group: A[smaller:equal].
    # unclassified group: A[equal:larger].
    # top group: A[larger:].
    smaller, equal, larger = 0, 0, len(A)
    # Keep iterating as long as there is an unclassified element.
    while equal < larger:
        # A[equal] is the incoming unclassified element.
        if A[equal] < pivot:
            A[smaller], A[equal] = A[equal], A[smaller]
            smaller, equal = smaller + 1, equal + 1
        elif A[equal] == pivot:
            equal += 1
        else: # A[equal] > pivot.
            larger -= 1
            A[equal], A[larger] = A[larger], A[equal]
```

Each iteration decreases the size of *unclassified* by 1, and the time spent within each iteration is $O(1)$, implying the time complexity is $O(n)$. The space complexity is clearly $O(1)$.

Variant: Assuming that keys take one of three values, reorder the array so that all objects with the same key appear together. The order of the subarrays is not important. For example, both Figures 2.1(b) and 2.1(c) on Page 10 are valid answers for Figure 2.1(a) on Page 10. Use $O(1)$ additional space and $O(n)$ time.

Variant: Given an array A of n objects with keys that takes one of four values, reorder the array so that all objects that have the same key appear together. Use $O(1)$ additional space and $O(n)$ time.

Variant: Given an array A of n objects with Boolean-valued keys, reorder the array so that objects that have the key `false` appear first. Use $O(1)$ additional space and $O(n)$ time.

Variant: Given an array A of n objects with Boolean-valued keys, reorder the array so that objects that have the key `false` appear first. The relative ordering of objects with key `true` should not change. Use $O(1)$ additional space and $O(n)$ time.

Strings

String pattern matching is an important problem that occurs in many areas of science and information processing. In computing, it occurs naturally as part of data processing, text editing, term rewriting, lexical analysis, and information retrieval.

—“Algorithms For Finding Patterns in Strings,”

A. V. AHO, 1990

Strings are ubiquitous in programming today—scripting, web development, and bioinformatics all make extensive use of strings.

A string can be viewed as a special kind of array, namely one made out of characters. We treat strings separately from arrays because certain operations which are commonly applied to strings—for example, comparison, joining, splitting, searching for substrings, replacing one string by another, parsing, etc.—do not make sense for general arrays.

You should know how strings are represented in memory, and understand basic operations on strings such as comparison, copying, joining, splitting, matching, etc. Advanced string processing algorithms often use hash tables (Chapter 9) and dynamic programming (Page 63). In this chapter we present problems on strings which can be solved using basic techniques.

Strings boot camp

A palindromic string is one which reads the same when it is reversed. The program below checks whether a string is palindromic. Rather than creating a new string for the reverse of the input string, it traverses the input string forwards and backwards, thereby saving space. Notice how it uniformly handles even and odd length strings.

```
def is_palindromic(s):
    # Note that s[~i] for i in [0, len(s) - 1] is s[-(i + 1)].
    return all(s[i] == s[~i] for i in range(len(s) // 2))
```

The time complexity is $O(n)$ and the space complexity is $O(1)$, where n is the length of the string.

Know your string libraries

The key operators and functions on strings are `s[3]`, `len(s)`, `s + t`, `s[2:4]` (and all the other variants of slicing for lists described on Page 9), `s in t`, `s.strip()`, `s.startswith(prefix)`, `s.endswith(suffix)`, `'Euclid,Axiom 5,Parallel Lines'.split(',')`, `3 * '01'`, `','.join(('Gauss', 'Prince of Mathematicians', '1777-1855'))`, `s.lower()`, and `'Name name, Rank rank'.format(name='Archimedes', rank=3)`.

It's important to remember that strings are immutable—operations like `s = s[1:]` or `s += '123'` imply creating a new array of characters that is then assigned back to `s`. This implies that concatenating a single character n times to a string in a for loop has $O(n^2)$ time complexity. (Some

Similar to arrays, string problems often have simple brute-force solutions that use $O(n)$ space solution, but subtler solutions that use the string itself to **reduce space complexity** to $O(1)$.

Understand the **implications** of a string type which is **immutable**, e.g., the need to allocate a new string when concatenating immutable strings. Know **alternatives** to immutable strings, e.g., a list in Python.

Updating a mutable string from the front is slow, so see if it's possible to **write values from the back**.

Table 3.1: Top Tips for Strings

implementations of Python use tricks under-the-hood to avoid having to do this allocation, reducing the complexity to $O(n)$.)

3.1 INTERCONVERT STRINGS AND INTEGERS

A string is a sequence of characters. A string may encode an integer, e.g., "123" encodes 123. In this problem, you are to implement methods that take a string representing an integer and return the corresponding integer, and vice versa. Your code should handle negative integers. You cannot use library functions like `stoi` in C++, `parseInt` in Java, and `int` in Python.

Implement string/integer inter-conversion functions.

Hint: Build the result one digit at a time.

Solution: Let's consider the integer to string problem first. If the number to convert is a single digit, i.e., it is between 0 and 9, the result is easy to compute: it is the string consisting of the single character encoding that digit.

If the number has more than one digit, it is natural to perform the conversion digit-by-digit. The key insight is that for any positive integer x , the least significant digit in the decimal representation of x is $x \bmod 10$, and the remaining digits are $x/10$. This approach computes the digits in reverse order, e.g., if we begin with 423, we get 3 and are left with 42 to convert. Then we get 2, and are left with 4 to convert. Finally, we get 4 and there are no digits to convert. The natural algorithm would be to prepend digits to the partial result. However, adding a digit to the beginning of a string is expensive, since all remaining digit have to be moved. A more time efficient approach is to add each computed digit to the end, and then reverse the computed sequence.

If x is negative, we record that, negate x , and then add a '-' before reversing. If x is 0, our code breaks out of the iteration without writing any digits, in which case we need to explicitly set a 0.

To convert from a string to an integer we recall the basic working of a positional number system. A base-10 number $d_2d_1d_0$ encodes the number $10^2 \times d_2 + 10^1 \times d_1 + d_0$. A brute-force algorithm then is to begin with the rightmost digit, and iteratively add $10^i \times d_i$ to a cumulative sum. The efficient way to compute 10^{i+1} is to use the existing value 10^i and multiply that by 10.

A more elegant solution is to begin from the leftmost digit and with each succeeding digit, multiply the partial result by 10 and add that digit. For example, to convert "314" to an integer, we initial the partial result r to 0. In the first iteration, $r = 3$, in the second iteration $r = 3 \times 10 + 1 = 31$, and in the third iteration $r = 31 \times 10 + 4 = 314$, which is the final result.

Negative numbers are handled by recording the sign and negating the result.

```

def int_to_string(x):
    is_negative = False
    if x < 0:
        x, is_negative = -x, True

    s = []
    while True:
        s.append(chr(ord('0') + x % 10))
        x //= 10
        if x == 0:
            break

    # Adds the negative sign back if is_negative
    return ('-' if is_negative else '') + ''.join(reversed(s))

def string_to_int(s):
    return functools.reduce(lambda running_sum,
                             c: running_sum * 10 + string.digits.index(c),
                             s[s[0] == '-':],
                             0) * (-1 if s[0] == '-' else 1)

```

3.2 BASE CONVERSION

In the decimal number system, the position of a digit is used to signify the power of 10 that digit is to be multiplied with. For example, “314” denotes the number $3 \times 100 + 1 \times 10 + 4 \times 1$. The base b number system generalizes the decimal number system: the string “ $a_{k-1}a_{k-2} \dots a_1a_0$ ”, where $0 \leq a_i < b$, denotes in base- b the integer $a_0 \times b^0 + a_1 \times b^1 + a_2 \times b^2 + \dots + a_{k-1} \times b^{k-1}$.

Write a program that performs base conversion. The input is a string, an integer b_1 , and another integer b_2 . The string represents an integer in base b_1 . The output should be the string representing the integer in base b_2 . Assume $2 \leq b_1, b_2 \leq 16$. Use “A” to represent 10, “B” for 11, ..., and “F” for 15. (For example, if the string is “615”, b_1 is 7 and b_2 is 13, then the result should be “1A7”, since $6 \times 7^2 + 1 \times 7 + 5 = 1 \times 13^2 + 10 \times 13 + 7$.)

Hint: What base can you easily convert to and from?

Solution: A brute-force approach might be to convert the input to a unary representation, and then group the 1s as multiples of b_2 , b_2^2 , b_2^3 , etc. For example, $(102)_3 = (111111111)_1$. To convert to base 4, there are two groups of 4 and with three 1s remaining, so the result is $(23)_4$. This approach is hard to implement, and has terrible time and space complexity.

The insight to a good algorithm is the fact that all languages have an integer type, which supports arithmetical operations like multiply, add, divide, modulus, etc. These operations make the conversion much easier. Specifically, we can convert a string in base b_1 to integer type using a sequence of multiply and adds. Then we convert that integer type to a string in base b_2 using a sequence of modulus and division operations. For example, for the string is “615”, $b_1 = 7$ and $b_2 = 13$, then the integer value, expressed in decimal, is 306. The least significant digit of the result is $306 \bmod 13 = 7$, and we continue with $306/13 = 23$. The next digit is $23 \bmod 13 = 10$, which we denote by ‘A’. We continue with $23/13 = 1$. Since $1 \bmod 13 = 1$ and $1/13 = 0$, the final digit is 1, and the overall result is “1A7”. The design of the algorithm nicely illustrates the use of reduction.

Since the conversion algorithm is naturally expressed in terms of smaller similar subproblems, it is natural to implement it using recursion.

```
def convert_base(num_as_string, b1, b2):
    def construct_from_base(num_as_int, base):
        return ('' if num_as_int == 0 else
                construct_from_base(num_as_int // base, base) +
                string.hexdigits[num_as_int % base].upper())

    is_negative = num_as_string[0] == '-'
    num_as_int = functools.reduce(
        lambda x, c: x * b1 + string.hexdigits.index(c.lower()),
        num_as_string[is_negative:], 0)
    return ('-' if is_negative else '') + ('0' if num_as_int == 0 else
                                           construct_from_base(num_as_int, b2))
```

The time complexity is $O(n(1 + \log_{b_2} b_1))$, where n is the length of s . The reasoning is as follows. First, we perform n multiply-and-adds to get x from s . Then we perform $\log_{b_2} x$ multiply and adds to get the result. The value x is upper-bounded by b_1^n , and $\log_{b_2}(b_1^n) = n \log_{b_2} b_1$.

Linked Lists

The S-expressions are formed according to the following recursive rules.

1. The atomic symbols p_1, p_2 , etc., are S-expressions.
2. A null expression $\wedge$ is also admitted.
3. If e is an S-expression so is (e) .
4. If e_1 and e_2 are S-expressions so is (e_1, e_2) .

— “Recursive Functions Of Symbolic Expressions,”

J. McCARTHY, 1959

A list implements an ordered collection of values, which may include repetitions. Sepcifically, a *singly linked list* is a data structure that contains a sequence of nodes such that each node contains an object and a reference to the next node in the list. The first node is referred to as the *head* and the last node is referred to as the *tail*; the tail’s next field is null. The structure of a singly linked list is given in Figure 4.1. There are many variants of linked lists, e.g., in a *doubly linked list*, each node has a link to its predecessor; similarly, a sentinel node or a self-loop can be used instead of null to mark the end of the list. The structure of a doubly linked list is given in Figure 4.2.

A list is similar to an array in that it contains objects in a linear order. The key differences are that inserting and deleting elements in a list has time complexity $O(1)$. On the other hand, obtaining the k th element in a list is expensive, having $O(n)$ time complexity. Lists are usually building blocks of more complex data structures. However, as we will see in this chapter, they can be the subject of tricky problems in their own right.

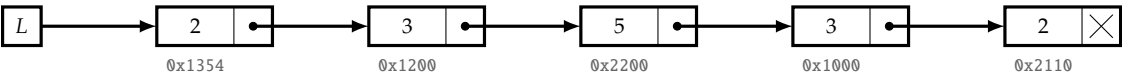


Figure 4.1: Example of a singly linked list. The number in hex below a node indicates the memory address of that node.

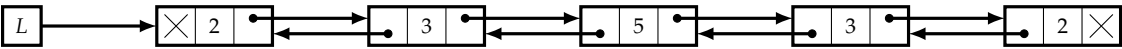


Figure 4.2: Example of a doubly linked list.

For all problems in this chapter, unless otherwise stated, each node has two entries—a data field, and a next field, which points to the next node in the list, with the next field of the last node being null. Its prototype is as follows:

```
class ListNode:

    def __init__(self, data=0, next_node=None):
        self.data = data
        self.next = next_node
```

Linked lists boot camp

There are two types of list-related problems—those where you have to implement your own list, and those where you have to exploit the standard list library. We will review both these aspects here, starting with implementation, then moving on to list libraries.

Implementing a basic list API—search, insert, delete—for singly linked lists is an excellent way to become comfortable with lists.

Search for a key:

```
def search_list(L, key):
    while L and L.data != key:
        L = L.next
    # If key was not present in the list, L will have become null.
    return L
```

Insert a new node after a specified node:

```
# Insert new_node after node.
def insert_after(node, new_node):
    new_node.next = node.next
    node.next = new_node
```

Delete a node:

```
# Delete the node past this one. Assume node is not a tail.
def delete_after(node):
    node.next = node.next.next
```

Insert and delete are local operations and have $O(1)$ time complexity. Search requires traversing the entire list, e.g., if the key is at the last node or is absent, so its time complexity is $O(n)$, where n is the number of nodes.

List problems often have a simple brute-force solution that uses $O(n)$ space, but a subtler solution that uses the existing list nodes to reduce space complexity to $O(1)$. Very often, a problem on lists is conceptually simple, and is more about cleanly coding what’s specified, rather than designing an algorithm. Consider using a dummy head (sometimes referred to as a sentinel) to avoid having to check for empty lists. This simplifies code, and makes bugs less likely. It’s easy to forget to update next (and previous for double linked list) for the head and tail. Algorithms operating on singly linked lists often benefit from using two iterators, one ahead of the other, or one advancing quicker than the other.
--

Table 4.1: Top Tips for Linked Lists

Know your linked list libraries

We now review the standard linked list library, with the reminder that many interview problems that are directly concerned with lists require you to write your own list class.

Under the hood, the Python list type is typically implemented as a dynamically resized array, and the key methods on it are described on Page 8. This chapter is concerned specifically with

linked lists, which are not a standard type in Python. We define our own singly and doubly linked list types. Some of the key methods on these lists include returning the head/tail, adding an element at the head/tail, returning the value stored at the head/tail, and deleting the head, tail, or arbitrary node in the list.

4.1 TEST FOR CYCLICITY

Although a linked list is supposed to be a sequence of nodes ending in null, it is possible to create a cycle in a linked list by making the next field of an element reference to one of the earlier nodes.

Write a program that takes the head of a singly linked list and returns null if there does not exist a cycle, and the node at the start of the cycle, if a cycle is present. (You do not know the length of the list in advance.)

Hint: Consider using two iterators, one fast and one slow.

Solution: This problem has several solutions. If space is not an issue, the simplest approach is to explore nodes via the next field starting from the head and storing visited nodes in a hash table—a cycle exists if and only if we visit a node already in the hash table. If no cycle exists, the search ends at the tail (often represented by having the next field set to null). This solution requires $O(n)$ space, where n is the number of nodes in the list.

A brute-force approach that does not use additional storage and does not modify the list is to traverse the list in two loops—the outer loop traverses the nodes one-by-one, and the inner loop starts from the head, and traverses as many nodes as the outer loop has gone through so far. If the node being visited by the outer loop is visited twice, a loop has been detected. (If the outer loop encounters the end of the list, no cycle exists.) This approach has $O(n^2)$ time complexity.

This idea can be made to work in linear time—use a slow iterator and a fast iterator to traverse the list. In each iteration, advance the slow iterator by one and the fast iterator by two. The list has a cycle if and only if the two iterators meet. The reasoning is as follows: if the fast iterator jumps over the slow iterator, the slow iterator will equal the fast iterator in the next step.

Now, assuming that we have detected a cycle using the above method, we can find the start of the cycle, by first calculating the cycle length C . Once we know there is a cycle, and we have a node on it, it is trivial to compute the cycle length. To find the first node on the cycle, we use two iterators, one of which is C ahead of the other. We advance them in tandem, and when they meet, that node must be the first node on the cycle.

The code to do this traversal is quite simple:

```
def has_cycle(head):
    def cycle_len(end):
        start, step = end, 0
        while True:
            step += 1
            start = start.next
            if start is end:
                return step

    fast = slow = head
    while fast and fast.next and fast.next.next:
        slow, fast = slow.next, fast.next.next
    if slow is fast:
        # Finds the start of the cycle.
```

```

cycle_len_advanced_iter = head
for _ in range(cycle_len(slow)):
    cycle_len_advanced_iter = cycle_len_advanced_iter.next

it = head
# Both iterators advance in tandem.
while it is not cycle_len_advanced_iter:
    it = it.next
    cycle_len_advanced_iter = cycle_len_advanced_iter.next
return it # iter is the start of cycle.
return None # No cycle.

```

Let F be the number of nodes to the start of the cycle, C the number of nodes on the cycle, and n the total number of nodes. Then the time complexity is $O(F) + O(C) = O(n) — O(F)$ for both pointers to reach the cycle, and $O(C)$ for them to overlap once the slower one enters the cycle.

Variant: The following program purports to compute the beginning of the cycle without determining the length of the cycle; it has the benefit of being more succinct than the code listed above. Is the program correct?

```

def has_cycle(head):
    fast = slow = head
    while fast and fast.next and fast.next.next:
        slow, fast = slow.next, fast.next.next
    if slow is fast: # There is a cycle.
        # Tries to find the start of the cycle.
        slow = head
        # Both pointers advance at the same time.
        while slow is not fast:
            slow, fast = slow.next, fast.next
        return slow # slow is the start of cycle.
    return None # No cycle.

```

Stacks and Queues

Linear lists in which insertions, deletions, and accesses to values occur almost always at the first or the last node are very frequently encountered, and we give them special names . . .

— “The Art of Computer Programming, Volume 1,”
D. E. KNUTH, 1997

Stacks support last-in, first-out semantics for inserts and deletes, whereas queues are first-in, first-out. Stacks and queues are usually building blocks in a solution to a complex problem. As we will soon see, they can also make for stand-alone problems.

Stacks

A *stack* supports two basic operations—push and pop. Elements are added (pushed) and removed (popped) in last-in, first-out order, as shown in Figure 5.1. If the stack is empty, pop typically returns null or throws an exception.

When the stack is implemented using a linked list these operations have $O(1)$ time complexity. If it is implemented using an array, there is maximum number of entries it can have—push and pop are still $O(1)$. If the array is dynamically resized, the amortized time for both push and pop is $O(1)$. A stack can support additional operations such as peek, which returns the top of the stack without popping it.

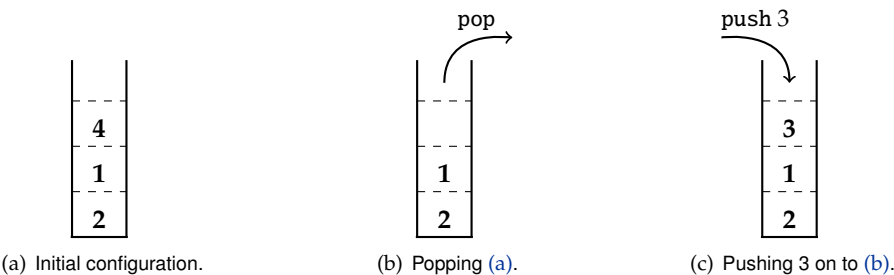


Figure 5.1: Operations on a stack.

Stacks boot camp

The last-in, first-out semantics of a stack make it very useful for creating reverse iterators for sequences which are stored in a way that would make it difficult or impossible to step back from a given element. This a program uses a stack to print the entries of a singly-linked list in reverse order.

```
def print_linked_list_in_reverse(head):
    nodes = []
    while head:
        nodes.append(head.data)
        head = head.next
    while nodes:
        print(nodes.pop())
```

The time and space complexity are $O(n)$, where n is the number of nodes in the list.

Learn to recognize when the stack **LIFO** property is **applicable**. For example, **parsing** typically benefits from a stack.

Consider **augmenting** the basic stack or queue data structure to support additional operations, such as finding the maximum element.

Table 5.1: Top Tips for Stacks

Know your stack libraries

Some of the problems require you to implement your own stack class; for others, use the built-in list-type.

- `s.append(e)` pushes an element onto the stack. Not much can go wrong with a call to push.
- `s[-1]` will retrieve, but does not remove, the element at the top of the stack.
- `s.pop()` will remove and return the element at the top of the stack.
- `len(s) == 0` tests if the stack is empty.

When called on an empty list `s`, both `s[-1]` and `s.pop()` raise an `IndexError` exception.

5.1 IMPLEMENT A STACK WITH MAX API

Design a stack that includes a max operation, in addition to push and pop. The max method should return the maximum value stored in the stack.

Hint: Use additional storage to track the maximum value.

Solution: The simplest way to implement a max operation is to consider each element in the stack, e.g., by iterating through the underlying array for an array-based stack. The time complexity is $O(n)$ and the space complexity is $O(1)$, where n is the number of elements currently in the stack.

The time complexity can be reduced to $O(\log n)$ using auxiliary data structures, specifically, a heap or a BST, and a hash table. The space complexity increases to $O(n)$ and the code is quite complex.

Suppose we use a single auxiliary variable, M , to record the element that is maximum in the stack. Updating M on pushes is easy: $M = \max(M, e)$, where e is the element being pushed. However, updating M on pop is very time consuming. If M is the element being popped, we have no way of knowing what the maximum remaining element is, and are forced to consider all the remaining elements.

We can dramatically improve on the time complexity of popping by caching, in essence, trading time for space. Specifically, for each entry in the stack, we cache the maximum stored at or below that entry. Now when we pop, we evict the corresponding cached value.

```

class Stack:
    ElementWithCachedMax = collections.namedtuple('ElementWithCachedMax',
                                                  ('element', 'max'))

    def __init__(self):
        self._element_with_cached_max = []

    def empty(self):
        return len(self._element_with_cached_max) == 0

    def max(self):
        if self.empty():
            raise IndexError('max(): empty stack')
        return self._element_with_cached_max[-1].max

    def pop(self):
        if self.empty():
            raise IndexError('pop(): empty stack')
        return self._element_with_cached_max.pop().element

    def push(self, x):
        self._element_with_cached_max.append(
            self.ElementWithCachedMax(x, x
                                       if self.empty() else max(x, self.max())))

```

Each of the specified methods has time complexity $O(1)$. The additional space complexity is $O(n)$, regardless of the stored keys.

We can improve on the best-case space needed by observing that if an element e being pushed is smaller than the maximum element already in the stack, then e can never be the maximum, so we do not need to record it. We cannot store the sequence of maximum values in a separate stack because of the possibility of duplicates. We resolve this by additionally recording the number of occurrences of each maximum value. See [Figure 5.2 on the following page](#) for an example.

```

class Stack:

    class MaxWithCount:

        def __init__(self, max, count):
            self.max, self.count = max, count

    def __init__(self):
        self._element = []
        self._cached_max_with_count = []

    def empty(self):
        return len(self._element) == 0

    def max(self):
        if self.empty():
            raise IndexError('max(): empty stack')
        return self._cached_max_with_count[-1].max

```

```

def pop(self):
    if self.empty():
        raise IndexError('pop(): empty stack')
    pop_element = self._element.pop()
    current_max = self._cached_max_with_count[-1].max
    if pop_element == current_max:
        self._cached_max_with_count[-1].count -= 1
        if self._cached_max_with_count[-1].count == 0:
            self._cached_max_with_count.pop()
    return pop_element

def push(self, x):
    self._element.append(x)
    if len(self._cached_max_with_count) == 0:
        self._cached_max_with_count.append(self.MaxWithCount(x, 1))
    else:
        current_max = self._cached_max_with_count[-1].max
        if x == current_max:
            self._cached_max_with_count[-1].count += 1
        elif x > current_max:
            self._cached_max_with_count.append(self.MaxWithCount(x, 1))

```

The worst-case additional space complexity is $O(n)$, which occurs when each key pushed is greater than all keys in the primary stack. However, when the number of distinct keys is small, or the maximum changes infrequently, the additional space complexity is less, $O(1)$ in the best-case. The time complexity for each specified method is still $O(1)$.

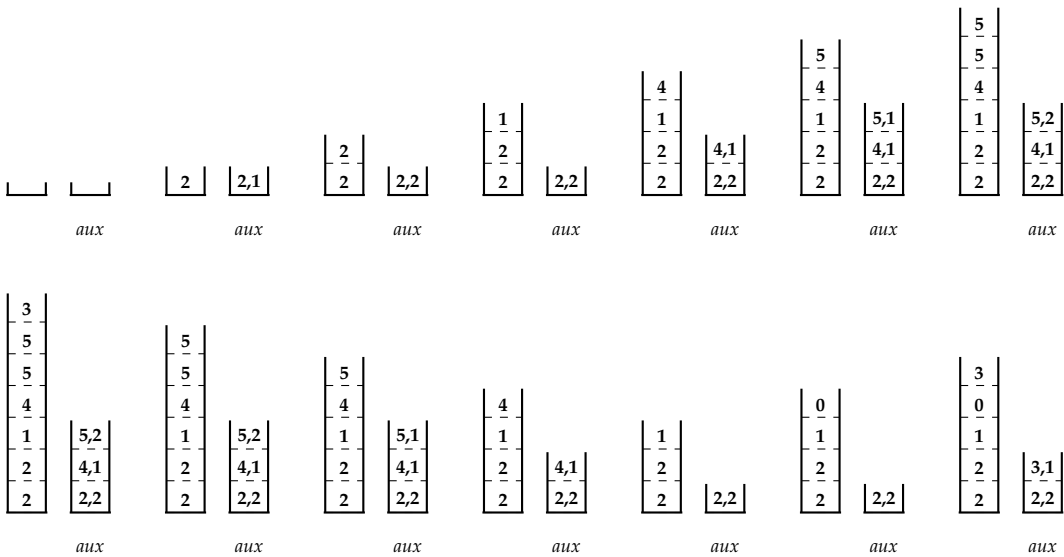


Figure 5.2: The primary and auxiliary stacks for the following operations: push 2, push 2, push 1, push 4, push 5, push 5, push 3, pop, pop, pop, pop, push 0, push 3. Both stacks are initially empty, and their progression is shown from left-to-right, then top-to-bottom. The top of the auxiliary stack holds the maximum element in the stack, and the number of times that element occurs in the stack. The auxiliary stack is denoted by *aux*.

Queues

A *queue* supports two basic operations—enqueue and dequeue. (If the queue is empty, dequeue typically returns null or throws an exception.) Elements are added (enqueued) and removed (dequeued) in first-in, first-out order. The most recently inserted element is referred to as the tail or back element, and the item that was inserted least recently is referred to as the head or front element.

A queue can be implemented using a linked list, in which case these operations have $O(1)$ time complexity. The queue API often includes other operations, e.g., a method that returns the item at the head of the queue without removing it, a method that returns the item at the tail of the queue without removing it, etc.

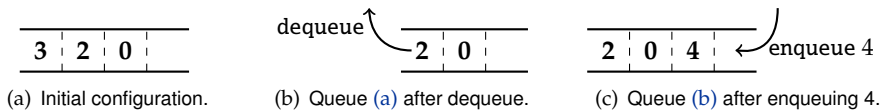


Figure 5.3: Examples of enqueueing and dequeuing.

A *deque*, also sometimes called a double-ended queue, is a doubly linked list in which all insertions and deletions are from one of the two ends of the list, i.e., at the head or the tail. An insertion to the front is commonly called a push, and an insertion to the back is commonly called an inject. A deletion from the front is commonly called a pop, and a deletion from the back is commonly called an eject. (Different languages and libraries may have different nomenclature.)

Queues boot camp

In the following program, we implement the basic queue API—enqueue and dequeue—as well as a max-method, which returns the maximum element stored in the queue. The basic idea is to use composition: add a private field that references a library queue object, and forward existing methods (enqueue and dequeue in this case) to that object.

```
class Queue:

    def __init__(self):
        self._data = []

    def enqueue(self, x):
        self._data.append(x)

    def dequeue(self):
        return self._data.pop(0)

    def max(self):
        return max(self._data)
```

The time complexity of enqueue and dequeue are the same as that of the library queue, namely, $O(1)$. The time complexity of finding the maximum is $O(n)$, where n is the number of entries.

Learn to recognize when the queue FIFO property is applicable. For example, queues are ideal when order needs to be preserved.

Table 5.2: Top Tips for Queues

Know your queue libraries

Some of the problems require you to implement your own queue class; for others, use the `collections.deque` class.

- `q.append(e)` pushes an element onto the queue. Not much can go wrong with a call to `push`.
- `q[0]` will retrieve, but not remove, the element at the front of the queue. Similarly, `q[-1]` will retrieve, but not remove, the element at the back of the queue.
- `q.popleft()` will remove and return the element at the front of the queue.

Dequeuing or accessing the head/tail of an empty collection results in an `IndexError` exception being raised.

5.2 COMPUTE BINARY TREE NODES IN ORDER OF INCREASING DEPTH

Binary trees are formally defined in Chapter 6. In particular, each node in a binary tree has a depth, which is its distance from the root.

Given a binary tree, return an array consisting of the keys at the same level. Keys should appear in the order of the corresponding nodes' depths, breaking ties from left to right. For example, you should return $\langle\langle 314\rangle, \langle 6, 6\rangle, \langle 271, 561, 2, 271\rangle, \langle 28, 0, 3, 1, 28\rangle, \langle 17, 401, 257\rangle, \langle 641\rangle\rangle$ for the binary tree in Figure 6.1 on Page 28.

Hint: First think about solving this problem with a pair of queues.

Solution: A brute force approach might be to write the nodes into an array while simultaneously computing their depth. We can use preorder traversal to compute this array—by traversing a node's left child first we can ensure that nodes at the same depth are sorted from left to right. Now we can sort this array using a stable sorting algorithm with node depth being the sort key. The time complexity is dominated by the time taken to sort, i.e., $O(n \log n)$, and the space complexity is $O(n)$, which is the space to store the node depths.

Intuitively, since nodes are already presented in a somewhat ordered fashion in the tree, it should be possible to avoid a full-blow sort, thereby reducing time complexity. Furthermore, by processing nodes in order of depth, we do not need to label every node with its depth.

In the following, we use a queue of nodes to store nodes at depth i and a queue of nodes at depth $i + 1$. After all nodes at depth i are processed, we are done with that queue, and can start processing the queue with nodes at depth $i + 1$, putting the depth $i + 2$ nodes in a new queue.

```
def binary_tree_depth_order(tree):
    result, curr_depth_nodes = [], collections.deque([tree])
    while curr_depth_nodes:
        next_depth_nodes, this_level = collections.deque([]), []
        while curr_depth_nodes:
            curr = curr_depth_nodes.popleft()
            if curr:
                this_level.append(curr.data)
```

```

        # Defer the null checks to the null test above.
        next_depth_nodes += [curr.left, curr.right]

    if this_level:
        result.append(this_level)
        curr_depth_nodes = next_depth_nodes
    return result

```

Since each node is enqueued and dequeued exactly once, the time complexity is $O(n)$. The space complexity is $O(m)$, where m is the maximum number of nodes at any single depth.

Variant: Write a program which takes as input a binary tree and returns the keys in top down, alternating left-to-right and right-to-left order, starting from left-to-right. For example, if the input is the tree in Figure 6.1 on the following page, your program should return $\langle\langle 314 \rangle, \langle 6, 6 \rangle, \langle 271, 561, 2, 271 \rangle, \langle 28, 1, 3, 0, 28 \rangle, \langle 17, 401, 257 \rangle, \langle 641 \rangle\rangle$.

Variant: Write a program which takes as input a binary tree and returns the keys in a bottom up, left-to-right order. For example, if the input is the tree in Figure 6.1 on the next page, your program should return $\langle\langle 641 \rangle, \langle 17, 401, 257 \rangle, \langle 28, 0, 3, 1, 28 \rangle, \langle 271, 561, 2, 271 \rangle, \langle 6, 6 \rangle, \langle 314 \rangle\rangle$.

Variant: Write a program which takes as input a binary tree with integer keys, and returns the average of the keys at each level. For example, if the input is the tree in Figure 6.1 on the following page, your program should return $\langle 314, 6, 276.25, 12, 225, 641 \rangle$.

Binary Trees

The method of solution involves the development of a theory of finite automata operating on infinite trees.

— “Decidability of Second Order Theories and Automata on Trees,”
M. O. RABIN, 1969

Formally, a binary tree is either empty, or a *root* node *r* together with a left binary tree and a right binary tree. The subtrees themselves are binary trees. The left binary tree is sometimes referred to as the *left subtree* of the root, and the right binary tree is referred to as the *right subtree* of the root.

Binary trees most commonly occur in the context of binary search trees, wherein keys are stored in a sorted fashion (Chapter 11 on Page 54). However, there are many other applications of binary trees: at a high level, *binary tree* are appropriate when dealing with hierarchies.

Figure 6.1 gives a graphical representation of a binary tree. Node *A* is the root. Nodes *B* and *I* are the left and right children of *A*.

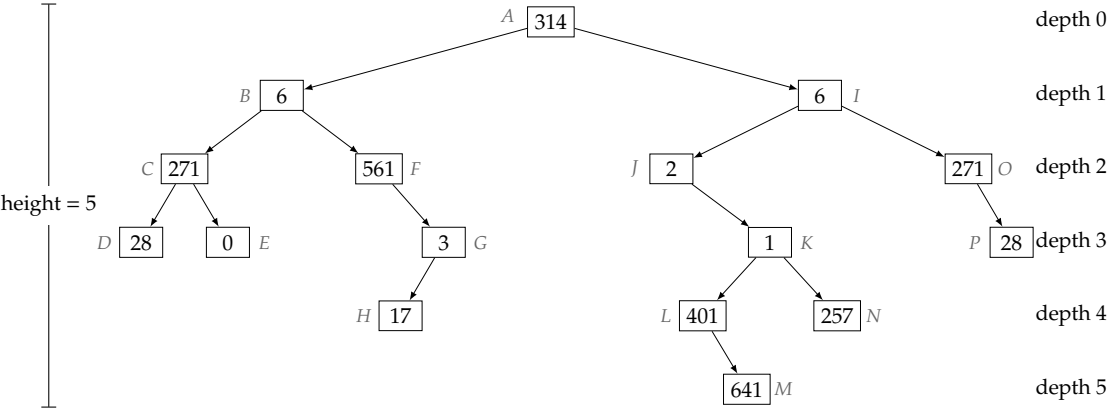


Figure 6.1: Example of a binary tree. The node depths range from 0 to 5. Node *M* has the highest depth (5) of any node in the tree, implying the height of the tree is 5.

Often the node stores additional data. Its prototype is listed as follows:

```

class BinaryTreeNode:

    def __init__(self, data=None, left=None, right=None):
        self.data = data
        self.left = left
        self.right = right

```

Each node, except the root, is itself the root of a left subtree or a right subtree. If l is the root of p 's left subtree, we will say l is the *left child* of p , and p is the *parent* of l ; the notion of *right child* is similar. If a node is a left or a right child of p , we say it is a *child* of p . Note that with the exception of the root, every node has a unique parent. Usually, but not universally, the node object definition includes a parent field (which is null for the root). Observe that for any node there exists a unique sequence of nodes from the root to that node with each node in the sequence being a child of the previous node. This sequence is sometimes referred to as the *search path* from the root to the node.

The parent-child relationship defines an ancestor-descendant relationship on nodes in a binary tree. Specifically, a node is an *ancestor* of d if it lies on the search path from the root to d . If a node is an ancestor of d , we say d is a *descendant* of that node. Our convention is that a node is an ancestor and descendant of itself. A node that has no descendants except for itself is called a *leaf*.

The *depth* of a node n is the number of nodes on the search path from the root to n , not including n itself. The *height* of a binary tree is the maximum depth of any node in that tree. A *level* of a tree is all nodes at the same depth. See Figure 6.1 on the preceding page for an example of the depth and height concepts.

As concrete examples of these concepts, consider the binary tree in Figure 6.1 on the facing page. Node I is the parent of J and O . Node G is a descendant of B . The search path to L is $\langle A, I, J, K, L \rangle$. The depth of N is 4. Node M is the node of maximum depth, and hence the height of the tree is 5. The height of the subtree rooted at B is 3. The height of the subtree rooted at H is 0. Nodes D, E, H, M, N , and P are the leaves of the tree.

A *full binary tree* is a binary tree in which every node other than the leaves has two children. A *perfect binary tree* is a full binary tree in which all leaves are at the same depth, and in which every parent has two children. A *complete binary tree* is a binary tree in which every level, except possibly the last, is completely filled, and all nodes are as far left as possible. (This terminology is not universal, e.g., some authors use complete binary tree where we write perfect binary tree.) It is straightforward to prove using induction that the number of nonleaf nodes in a full binary tree is one less than the number of leaves. A perfect binary tree of height h contains exactly $2^{h+1} - 1$ nodes, of which 2^h are leaves. A complete binary tree on n nodes has height $\lceil \log n \rceil$. A left-skewed tree is a tree in which no node has a right child; a right-skewed tree is a tree in which no node has a left child. In either case, we refer to the binary tree as being skewed.

A key computation on a binary tree is *traversing* all the nodes in the tree. (Traversing is also sometimes called *walking*.) Here are some ways in which this visit can be done.

- Traverse the left subtree, visit the root, then traverse the right subtree (an *inorder* traversal). An inorder traversal of the binary tree in Figure 6.1 on the preceding page visits the nodes in the following order: $\langle D, C, E, B, F, H, G, A, J, L, M, K, N, I, O, P \rangle$.
- Visit the root, traverse the left subtree, then traverse the right subtree (a *preorder* traversal). A preorder traversal of the binary tree in Figure 6.1 on the facing page visits the nodes in the following order: $\langle A, B, C, D, E, F, G, H, I, J, K, L, M, N, O, P \rangle$.
- Traverse the left subtree, traverse the right subtree, and then visit the root (a *postorder* traversal). A postorder traversal of the binary tree in Figure 6.1 on the preceding page visits the nodes in the following order: $\langle D, E, C, H, G, F, B, M, L, N, K, J, P, O, I, A \rangle$.

Let T be a binary tree of n nodes, with height h . Implemented recursively, these traversals have $O(n)$ time complexity and $O(h)$ additional space complexity. (The space complexity is dictated by the maximum depth of the function call stack.) If each node has a parent field, the traversals can be done with $O(1)$ additional space complexity.

The term tree is overloaded, which can lead to confusion; see Page 75 for an overview of the common variants.

Binary trees boot camp

A good way to get up to speed with binary trees is to implement the three basic traversals—inorder, preorder, and postorder.

```
def tree_traversal(root):
    if root:
        # Preorder: Processes the root before the traversals of left and right
        # children.
        print('Preorder: %d' % root.data)
        tree_traversal(root.left)
        # Inorder: Processes the root after the traversal of left child and
        # before the traversal of right child.
        print('Inorder: %d' % root.data)
        tree_traversal(root.right)
        # Postorder: Processes the root after the traversals of left and right
        # children.
        print('Postorder: %d' % root.data)
```

The time complexity of each approach is $O(n)$, where n is the number of nodes in the tree. Although no memory is explicitly allocated, the function call stack reaches a maximum depth of h , the height of the tree. Therefore, the space complexity is $O(h)$. The minimum value for h is $\log n$ (complete binary tree) and the maximum value for h is n (skewed tree).

Recursive algorithms are well-suited to problems on trees. Remember to include space implicitly allocated on the function call stack when doing space complexity analysis. Please read the introduction to Chapter 12 if you need a refresher on recursion. Some tree problems have simple brute-force solutions that use $O(n)$ space solution, but subtler solutions that uses the existing tree nodes to reduce space complexity to $O(1)$. Consider left- and right-skewed trees when doing complexity analysis. Note that $O(h)$ complexity, where h is the tree height, translates into $O(\log n)$ complexity for balanced trees, but $O(n)$ complexity for skewed trees. If each node has a parent field, use it to make your code simpler, and to reduce time and space complexity. It's easy to make the mistake of treating a node that has a single child as a leaf.
--

Table 6.1: Top Tips for Binary Trees

6.1 TEST IF A BINARY TREE IS HEIGHT-BALANCED

A binary tree is said to be height-balanced if for each node in the tree, the difference in the height of its left and right subtrees is at most one. A perfect binary tree is height-balanced, as is a complete binary tree. A height-balanced binary tree does not have to be perfect or complete—see Figure 6.2 on the next page for an example.

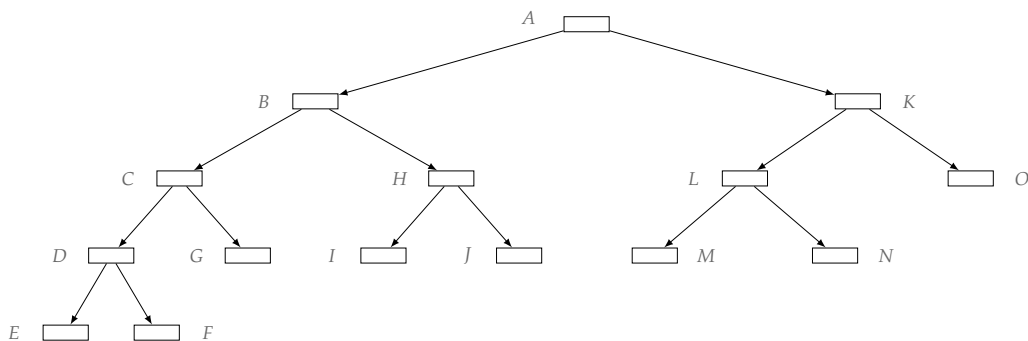


Figure 6.2: A height-balanced binary tree of height 4.

Write a program that takes as input the root of a binary tree and checks whether the tree is height-balanced.

Hint: Think of a classic binary tree algorithm.

Solution: Here is a brute-force algorithm. Compute the height for the tree rooted at each node x recursively. The basic computation is to compute the height for each node starting from the leaves, and proceeding upwards. For each node, we check if the difference in heights of the left and right children is greater than one. We can store the heights in a hash table, or in a new field in the nodes. This entails $O(n)$ storage and $O(n)$ time, where n is the number of nodes of the tree.

We can solve this problem using less storage by observing that we do not need to store the heights of all nodes at the same time. Once we are done with a subtree, all we need is whether it is height-balanced, and if so, what its height is—we do not need any information about descendants of the subtree's root.

```
def is_balanced_binary_tree(tree):
    BalancedStatusWithHeight = collections.namedtuple(
        'BalancedStatusWithHeight', ('balanced', 'height'))

    # First value of the return value indicates if tree is balanced, and if
    # balanced the second value of the return value is the height of tree.
    def check_balanced(tree):
        if not tree:
            return BalancedStatusWithHeight(True, -1) # Base case.

        left_result = check_balanced(tree.left)
        if not left_result.balanced:
            # Left subtree is not balanced.
            return BalancedStatusWithHeight(False, 0)

        right_result = check_balanced(tree.right)
        if not right_result.balanced:
            # Right subtree is not balanced.
            return BalancedStatusWithHeight(False, 0)

        is_balanced = abs(left_result.height - right_result.height) <= 1
        height = max(left_result.height, right_result.height) + 1
        return BalancedStatusWithHeight(is_balanced, height)
```

```
return check_balanced(tree).balanced
```

The program implements a postorder traversal with some calls possibly being eliminated because of early termination. Specifically, if any left subtree is not height-balanced we do not need to visit the corresponding right subtree. The function call stack corresponds to a sequence of calls from the root through the unique path to the current node, and the stack height is therefore bounded by the height of the tree, leading to an $O(h)$ space bound. The time complexity is the same as that for a postorder traversal, namely $O(n)$.

Variant: Write a program that returns the size of the largest subtree that is complete.

Variant: Define a node in a binary tree to be k -balanced if the difference in the number of nodes in its left and right subtrees is no more than k . Design an algorithm that takes as input a binary tree and positive integer k , and returns a node in the binary tree such that the node is not k -balanced, but all of its descendants are k -balanced. For example, when applied to the binary tree in [Figure 6.1 on Page 28](#), if $k = 3$, your algorithm should return Node J .

Heaps

Using F-heaps we are able to obtain improved running times for several network optimization algorithms.

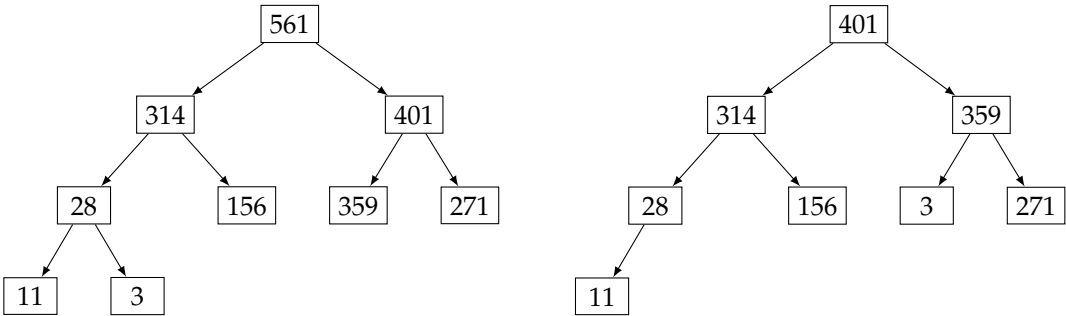
— “Fibonacci heaps and their uses,”
M. L. FREDMAN AND R. E. TARJAN, 1987

A *heap* is a specialized binary tree. Specifically, it is a complete binary tree as defined on Page 29. The keys must satisfy the *heap property*—the key at each node is at least as great as the keys stored at its children. See Figure 7.1(a) for an example of a max-heap. A max-heap can be implemented as an array; the children of the node at index i are at indices $2i + 1$ and $2i + 2$. The array representation for the max-heap in Figure 7.1(a) is $\langle 561, 314, 401, 28, 156, 359, 271, 11, 3 \rangle$.

A max-heap supports $O(\log n)$ insertions, $O(1)$ time lookup for the max element, and $O(\log n)$ deletion of the max element. The extract-max operation is defined to delete and return the maximum element. See Figure 7.1(b) for an example of deletion of the max element. Searching for arbitrary keys has $O(n)$ time complexity.

A heap is sometimes referred to as a priority queue because it behaves like a queue, with one difference: each element has a “priority” associated with it, and deletion removes the element with the highest priority.

The *min-heap* is a completely symmetric version of the data structure and supports $O(1)$ time lookups for the minimum element.



(a) A max-heap. Note that the root hold the maximum key, 561.

(b) After the deletion of max of the heap in (a). Deletion is performed by replacing the root's key with the key at the last leaf and then recovering the heap property by repeatedly exchanging keys with children.

Figure 7.1: A max-heap and deletion on that max-heap.

Heaps boot camp

Suppose you were asked to write a program which takes a sequence of strings presented in “streaming” fashion: you cannot back up to read an earlier value. Your program must compute the k longest strings in the sequence. All that is required is the k longest strings—it is not required to order these strings.

As we process the input, we want to track the k longest strings seen so far. Out of these k strings, the string to be evicted when a longer string is to be added is the shortest one. A min-heap (not a max-heap!) is the right data structure for this application, since it supports efficient find-min, remove-min, and insert. In the program below we use a heap with a custom compare function, wherein strings are ordered by length.

```
def top_k(k, stream):
    # Entries are compared by their lengths.
    min_heap = [(len(s), s) for s in itertools.islice(stream, k)]
    heapq.heapify(min_heap)
    for next_string in stream:
        # Push next_string and pop the shortest string in min_heap.
        heapq.heappushpop(min_heap, (len(next_string), next_string))
    return [p[1] for p in heapq.nsmallest(k, min_heap)]
```

Each string is processed in $O(\log k)$ time, which is the time to add and to remove the minimum element from the heap. Therefore, if there are n strings in the input, the time complexity to process all of them is $O(n \log k)$.

We could improve best-case time complexity by first comparing the new string’s length with the length of the string at the top of the heap (getting this string takes $O(1)$ time) and skipping the insert if the new string is too short to be in the set.

Use a heap when all you care about is the largest or smallest elements, and you do not need to support fast lookup, delete, or search operations for arbitrary elements.
--

A heap is a good choice when you need to compute the k largest or k smallest elements in a collection. For the former, use a min-heap, for the latter, use a max-heap.
--

Table 7.1: Top Tips for Heaps

Know your heap libraries

Heap functionality in Python is provided by the `heapq` module. The operations and functions we will use are

- `heapq.heapify(L)`, which transforms the elements in `L` into a heap in-place,
- `heapq.nlargest(k, L)` (`heapq.nsmallest(k, L)`) returns the k largest (smallest) elements in `L`,
- `heapq.heappush(h, e)`, which pushes a new element on the heap,
- `heapq.heappop(h)`, which pops the smallest element from the heap,
- `heapq.heappushpop(h, a)`, which pushes `a` on the heap and then pops and returns the smallest element, and
- `e = h[0]`, which returns the smallest element on the heap without popping it.

It’s very important to remember that `heapq` only provides min-heap functionality. If you need to build a max-heap on integers or floats, insert their negative to get the effect of a max-heap using `heapq`. For objects, implement `__lt()` appropriately.

7.1 MERGE SORTED FILES

This problem is motivated by the following scenario. You are given 500 files, each containing stock trade information for an S&P 500 company. Each trade is encoded by a line in the following format: 1232111,AAPL,30,456.12.

The first number is the time of the trade expressed as the number of milliseconds since the start of the day's trading. Lines within each file are sorted in increasing order of time. The remaining values are the stock symbol, number of shares, and price. You are to create a single file containing all the trades from the 500 files, sorted in order of increasing trade times. The individual files are of the order of 5–100 megabytes; the combined file will be of the order of five gigabytes. In the abstract, we are trying to solve the following problem.

Write a program that takes as input a set of sorted sequences and computes the union of these sequences as a sorted sequence. For example, if the input is $\langle 3, 5, 7 \rangle$, $\langle 0, 6 \rangle$, and $\langle 0, 6, 28 \rangle$, then the output is $\langle 0, 0, 3, 5, 6, 6, 7, 28 \rangle$.

Hint: Which part of each sequence is significant as the algorithm executes?

Solution: A brute-force approach is to concatenate these sequences into a single array and then sort it. The time complexity is $O(n \log n)$, assuming there are n elements in total.

The brute-force approach does not use the fact that the individual sequences are sorted. We can take advantage of this fact by restricting our attention to the first remaining element in each sequence. Specifically, we repeatedly pick the smallest element amongst the first element of each of the remaining part of each of the sequences.

A min-heap is ideal for maintaining a collection of elements when we need to add arbitrary values and extract the smallest element.

For ease of exposition, we show how to merge sorted arrays, rather than files. As a concrete example, suppose there are three sorted arrays to be merged: $\langle 3, 5, 7 \rangle$, $\langle 0, 6 \rangle$, and $\langle 0, 6, 28 \rangle$. For simplicity, we show the min-heap as containing entries from these three arrays. In practice, we need additional information for each entry, namely the array it is from, and its index in that array. (In the file case we do not need to explicitly maintain an index for next unprocessed element in each sequence—the file I/O library tracks the first unread entry in the file.)

The min-heap is initialized to the first entry of each array, i.e., it is $\{3, 0, 0\}$. We extract the smallest entry, 0, and add it to the output which is $\langle 0 \rangle$. Then we add 6 to the min-heap which is $\{3, 0, 6\}$ now. (We chose the 0 entry corresponding to the third array arbitrarily, it would be a perfectly acceptable to choose from the second array.) Next, extract 0, and add it to the output which is $\langle 0, 0 \rangle$; then add 6 to the min-heap which is $\{3, 6, 6\}$. Next, extract 3, and add it to the output which is $\langle 0, 0, 3 \rangle$; then add 5 to the min-heap which is $\{5, 6, 6\}$. Next, extract 5, and add it to the output which is $\langle 0, 0, 3, 5 \rangle$; then add 7 to the min-heap which is $\{7, 6, 6\}$. Next, extract 6, and add it to the output which is $\langle 0, 0, 3, 5, 6 \rangle$; assuming 6 is selected from the second array, which has no remaining elements, the min-heap is $\{7, 6\}$. Next, extract 6, and add it to the output which is $\langle 0, 0, 3, 5, 6, 6 \rangle$; then add 28 to the min-heap which is $\{7, 28\}$. Next, extract 7, and add it to the output which is $\langle 0, 0, 3, 5, 6, 6, 7 \rangle$; the min-heap is $\{28\}$. Next, extract 28, and add it to the output which is $\langle 0, 0, 3, 5, 6, 6, 7, 28 \rangle$; now, all elements are processed and the output stores the sorted elements.

```
def merge_sorted_arrays(sorted_arrays):
    min_heap = []
    # Builds a list of iterators for each array in sorted_arrays.
    sorted_arrays_iters = [iter(x) for x in sorted_arrays]
```

```

# Puts first element from each iterator in min_heap.
for i, it in enumerate(sorted_arrays_iters):
    first_element = next(it, None)
    if first_element is not None:
        heapq.heappush(min_heap, (first_element, i))

result = []
while min_heap:
    smallest_entry, smallest_array_i = heapq.heappop(min_heap)
    smallest_array_iter = sorted_arrays_iters[smallest_array_i]
    result.append(smallest_entry)
    next_element = next(smallest_array_iter, None)
    if next_element is not None:
        heapq.heappush(min_heap, (next_element, smallest_array_i))
return result

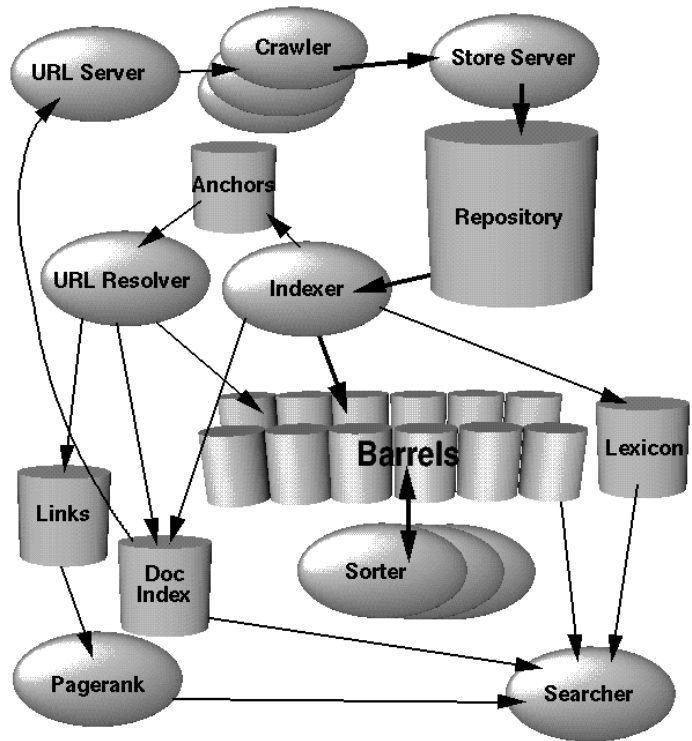
# Pythonic solution, uses the heapq.merge() method which takes multiple inputs.
def merge_sorted_arrays_pythonic(sorted_arrays):
    return list(heapq.merge(*sorted_arrays))

```

Let k be the number of input sequences. Then there are no more than k elements in the min-heap. Both extract-min and insert take $O(\log k)$ time. Hence, we can do the merge in $O(n \log k)$ time. The space complexity is $O(k)$ beyond the space needed to write the final result. In particular, if the data comes from files and is written to a file, instead of arrays, we would need only $O(k)$ additional storage.

Alternatively, we could recursively merge the k files, two at a time using the merge step from merge sort. We would go from k to $k/2$ then $k/4$, etc. files. There would be $\log k$ stages, and each has time complexity $O(n)$, so the time complexity is the same as that of the heap-based approach, i.e., $O(n \log k)$. The space complexity of any reasonable implementation of merge sort would end up being $O(n)$, which is considerably worse than the heap based approach when $k \ll n$.

Searching



— “The Anatomy of A Large-Scale Hypertextual Web Search Engine,”

S. M. BRIN AND L. PAGE, 1998

Search algorithms can be classified in a number of ways. Is the underlying collection static or dynamic, i.e., inserts and deletes are interleaved with searching? Is it worth spending the computational cost to preprocess the data so as to speed up subsequent queries? Are there statistical properties of the data that can be exploited? Should we operate directly on the data or transform it?

In this chapter, our focus is on static data stored in sorted order in an array. Data structures appropriate for dynamic updates are the subject of Chapters 7, 9, and 11.

The first collection of problems in this chapter are related to binary search. The second collection pertains to general search.

Binary search

Given an arbitrary collection of n keys, the only way to determine if a search key is present is by examining each element. This has $O(n)$ time complexity. Fundamentally, binary search is a natural elimination-based strategy for searching a sorted array. The idea is to eliminate half the keys from consideration by keeping the keys in sorted order. If the search key is not equal to the middle element of the array, one of the two sets of keys to the left and to the right of the middle element can be eliminated from further consideration.

Questions based on binary search are ideal from the interviewers perspective: it is a basic technique that every reasonable candidate is supposed to know and it can be implemented in a few lines of code. On the other hand, binary search is much trickier to implement correctly than it appears—you should implement it as well as write corner case tests to ensure you understand it properly.

Many published implementations are incorrect in subtle and not-so-subtle ways—a study reported that it is correctly implemented in only five out of twenty textbooks. Jon Bentley, in his book *“Programming Pearls”* reported that he assigned binary search in a course for professional programmers and found that 90% failed to code it correctly despite having ample time. (Bentley’s students would have been gratified to know that his own published implementation of binary search, in a column titled *“Writing Correct Programs”*, contained a bug that remained undetected for over twenty years.)

Binary search can be written in many ways—recursive, iterative, different idioms for conditionals, etc. Here is an iterative implementation adapted from Bentley’s book, which includes his bug.

```
def bsearch(t, A):
    L, U = 0, len(A) - 1
    while L <= U:
        M = (L + U) // 2
        if A[M] < t:
            L = M + 1
        elif A[M] == t:
            return M
        else:
            U = M - 1
    return -1
```

The error is in the assignment $M = (L + U) / 2$ in Line 4, which can potentially lead to overflow. This overflow can be avoided by using $M = L + (U - L) / 2$.

The time complexity of binary search is given by $T(n) = T(n/2) + c$, where c is a constant. This solves to $T(n) = O(\log n)$, which is far superior to the $O(n)$ approach needed when the keys are unsorted. A disadvantage of binary search is that it requires a sorted array and sorting an array takes $O(n \log n)$ time. However, if there are many searches to perform, the time taken to sort is not an issue.

Many variants of searching a sorted array require a little more thinking and create opportunities for missing corner cases.

Searching boot camp

When objects are comparable, they can be sorted and searched for using library search functions. Typically, the language knows how to compare built-in types, e.g., integers, strings, library classes

for date, URLs, SQL timestamps, etc. However, user-defined types used in sorted collections must explicitly implement comparison, and ensure this comparison has basic properties such as transitivity. (If the comparison is implemented incorrectly, you may find a lookup into a sorted collection fails, even when the item is present.)

Suppose we are given as input an array of students, sorted by descending GPA, with ties broken on name. In the program below, we show how to use the library binary search routine to perform fast searches in this array. In particular, we pass binary search a custom comparator which compares students on GPA (higher GPA comes first), with ties broken on name.

```
Student = collections.namedtuple('Student', ('name', 'grade_point_average'))

def comp_gpa(student):
    return (-student.grade_point_average, student.name)

def search_student(students, target, comp_gpa):
    i = bisect.bisect_left([comp_gpa(s) for s in students], comp_gpa(target))
    return 0 <= i < len(students) and students[i] == target
```

Assuming the i -th element in the sequence can be accessed in $O(1)$ time, the time complexity of the program is $O(\log n)$.

Binary search is an effective search tool. It is applicable to more than just searching in **sorted arrays**, e.g., it can be used to search an **interval of real numbers or integers**.

If your solution uses sorting, and the computation performed after sorting is faster than sorting, e.g., $O(n)$ or $O(\log n)$, **look for solutions that do not perform a complete sort**.

Consider **time/space tradeoffs**, such as making multiple passes through the data.

Table 8.1: Top Tips for Searching

Know your searching libraries

The bisect module provides binary search functions for sorted list. Specifically, assuming a is a sorted list.

- To find the first element that is not less than a targeted value, use `bisect.bisect_left(a, x)`. This call returns the index of the first entry that is greater than or equal to the targeted value. If all elements in the list are less than x , the returned value is `len(a)`.
- To find the first element that is greater than a targeted value, use `bisect.bisect_right(a, x)`. This call returns the index of the first entry that is greater than the targeted value. If all elements in the list are less than or equal to x , the returned value is `len(a)`.

In an interview, if it is allowed, use the above functions, instead of implementing your own binary search.

8.1 SEARCH A SORTED ARRAY FOR FIRST OCCURRENCE OF k

Binary search commonly asks for the index of *any* element of a sorted array that is equal to a specified element. The following problem has a slight twist on this.

-14	-10	2	108	108	243	285	285	285	401
A[0]	A[1]	A[2]	A[3]	A[4]	A[5]	A[6]	A[7]	A[8]	A[9]

Figure 8.1: A sorted array with repeated elements.

Write a method that takes a sorted array and a key and returns the index of the *first* occurrence of that key in the array. For example, when applied to the array in Figure 8.1 your algorithm should return 3 if the given key is 108; if it is 285, your algorithm should return 6.

Hint: What happens when every entry equals k ? Don't stop when you first see k .

Solution: A naive approach is to use binary search to find the index of any element equal to the key, k . (If k is not present, we simply return -1 .) After finding such an element, we traverse backwards from it to find the first occurrence of that element. The binary search takes time $O(\log n)$, where n is the number of entries in the array. Traversing backwards takes $O(n)$ time in the worst-case—consider the case where entries are equal to k .

The fundamental idea of binary search is to maintain a set of candidate solutions. For the current problem, if we see the element at index i equals k , although we do not know whether i is the first element equal to k , we do know that no subsequent elements can be the first one. Therefore we remove all elements with index $i + 1$ or more from the candidates.

Let's apply the above logic to the given example, with $k = 108$. We start with all indices as candidates, i.e., with $[0, 9]$. The midpoint index, 4 contains k . Therefore we can now update the candidate set to $[0, 3]$, and record 4 as an occurrence of k . The next midpoint is 1, and this index contains -10 . We update the candidate set to $[2, 3]$. The value at the midpoint 2 is 2, so we update the candidate set to $[3, 3]$. Since the value at this midpoint is 108, we update the first seen occurrence of k to 3. Now the interval is $[3, 2]$, which is empty, terminating the search—the result is 3.

```
def search_first_of_k(A, k):
    left, right, result = 0, len(A) - 1, -1
    # A[left:right + 1] is the candidate set.
    while left <= right:
        mid = (left + right) // 2
        if A[mid] > k:
            right = mid - 1
        elif A[mid] == k:
            result = mid
            right = mid - 1 # Nothing to the right of mid can be solution.
        else: # A[mid] < k.
            left = mid + 1
    return result
```

The complexity bound is still $O(\log n)$ —this is because each iteration reduces the size of the candidate set by half.

Variant: Design an efficient algorithm that takes a sorted array and a key, and finds the index of the *first* occurrence of an element greater than that key. For example, when applied to the array in Figure 8.1 your algorithm should return 9 if the key is 285; if it is -13 , your algorithm should return 1.

Variant: Let A be an unsorted array of n integers, with $A[0] \geq A[1]$ and $A[n-2] \leq A[n-1]$. Call an index i a *local minimum* if $A[i]$ is less than or equal to its neighbors. How would you efficiently find a local minimum, if one exists?

Variant: Write a program which takes a sorted array A of integers, and an integer k , and returns the interval enclosing k , i.e., the pair of integers L and U such that L is the first occurrence of k in A and U is the last occurrence of k in A . If k does not appear in A , return $[-1, -1]$. For example if $A = \langle 1, 2, 2, 4, 4, 4, 7, 11, 11, 13 \rangle$ and $k = 11$, you should return $[7, 8]$.

Variant: Write a program which tests if p is a prefix of a string in an array of sorted strings.

Hash Tables

The new methods are intended to reduce the amount of space required to contain the hash-coded information from that associated with conventional methods. The reduction in space is accomplished by exploiting the possibility that a small fraction of errors of commission may be tolerable in some applications.

— “Space/time trade-offs in hash coding with allowable errors,”

B. H. BLOOM, 1970

A hash table is a data structure used to store keys, optionally, with corresponding values. Inserts, deletes and lookups run in $O(1)$ time on average.

The underlying idea is to store keys in an array. A key is stored in the array locations (“slots”) based on its “hash code”. The hash code is an integer computed from the key by a hash function. If the hash function is chosen well, the objects are distributed uniformly across the array locations.

If two keys map to the same location, a “collision” is said to occur. The standard mechanism to deal with collisions is to maintain a linked list of objects at each array location. If the hash function does a good job of spreading objects across the underlying array and take $O(1)$ time to compute, on average, lookups, insertions, and deletions have $O(1 + n/m)$ time complexity, where n is the number of objects and m is the length of the array. If the “load” n/m grows large, rehashing can be applied to the hash table. A new array with a larger number of locations is allocated, and the objects are moved to the new array. Rehashing is expensive ($O(n + m)$ time) but if it is done infrequently (for example, whenever the number of entries doubles), its amortized cost is low.

A hash table is qualitatively different from a sorted array—keys do not have to appear in order, and randomization (specifically, the hash function) plays a central role. Compared to binary search trees (discussed in Chapter 11), inserting and deleting in a hash table is more efficient (assuming rehashing is infrequent). One disadvantage of hash tables is the need for a good hash function but this is rarely an issue in practice. Similarly, rehashing is not a problem outside of realtime systems and even for such systems, a separate thread can do the rehashing.

A hash function has one hard requirement—equal keys should have equal hash codes. This may seem obvious, but is easy to get wrong, e.g., by writing a hash function that is based on address rather than contents, or by including profiling data.

A softer requirement is that the hash function should “spread” keys, i.e., the hash codes for a subset of objects should be uniformly distributed across the underlying array. In addition, a hash function should be efficient to compute.

A common mistake with hash tables is that a key that’s present in a hash table will be updated. The consequence is that a lookup for that key will now fail, even though it’s still in the hash table. If you have to update a key, first remove it, then update it, and finally, add it back—this ensures it’s moved the correct array location. As a rule, you should avoid using mutable objects as keys.

Now we illustrate the steps for designing a hash function suitable for strings. First, the hash function should examine all the characters in the string. It should give a large range of values, and should not let one character dominate (e.g., if we simply cast characters to integers and multiplied them, a single 0 would result in a hash code of 0). We would also like a rolling hash function, one in which if a character is deleted from the front of the string, and another added to the end, the new hash code can be computed in $O(1)$ time. The following function has these properties:

```
def string_hash(s, modulus):  
    MULT = 997  
    return functools.reduce(lambda v, c: (v * MULT + ord(c)) % modulus, s, 0)
```

A hash table is a good data structure to represent a dictionary, i.e., a set of strings. In some applications, a trie, which is a tree data structure that is used to store a dynamic set of strings, has computational advantages. Unlike a BST, nodes in the tree do not store a key. Instead, the node's position in the tree defines the key which it is associated with.

Hash tables boot camp

We introduce hash tables with two examples—one is an application that benefits from the algorithmic advantages of hash tables, and the other illustrates the design of a class that can be used in a hash table.

An application of hash tables

Anagrams are popular word play puzzles, whereby rearranging letters of one set of words, you get another set of words. For example, “eleven plus two” is an anagram for “twelve plus one”. Crossword puzzle enthusiasts and Scrabble players benefit from the ability to view all possible anagrams of a given set of letters.

Suppose you were asked to write a program that takes as input a set of words and returns groups of anagrams for those words. Each group must contain at least two words.

For example, if the input is “debitcard”, “elvis”, “silent”, “badcredit”, “lives”, “freedom”, “listen”, “levis”, “money” then there are three groups of anagrams: (1.) “debitcard”, “badcredit”; (2.) “elvis”, “lives”, “levis”; (3.) “silent”, “listen”. (Note that “money” does not appear in any group, since it has no anagrams in the set.)

Let's begin by considering the problem of testing whether one word is an anagram of another. Since anagrams do not depend on the ordering of characters in the strings, we can perform the test by sorting the characters in the string. Two words are anagrams if and only if they result in equal strings after sorting. For example, sort(“logarithmic”) and sort(“algorithmic”) are both “acghiilmort”, so “logarithmic” and “algorithmic” are anagrams.

We can form the described grouping of strings by iterating through all strings, and comparing each string with all other remaining strings. If two strings are anagrams, we do not consider the second string again. This leads to an $O(n^2m \log m)$ algorithm, where n is the number of strings and m is the maximum string length.

Looking more carefully at the above computation, note that the key idea is to map strings to a representative. Given any string, its sorted version can be used as a unique identifier for the anagram group it belongs to. What we want is a map from a sorted string to the anagrams it corresponds to. Anytime you need to store a set of strings, a hash table is an excellent choice. Our

final algorithm proceeds by adding sort(s) for each string s in the dictionary to a hash table. The sorted strings are keys, and the values are arrays of the corresponding strings from the original input.

```
def find_anagrams(dictionary):
    sorted_string_to_anagrams = collections.defaultdict(list)
    for s in dictionary:
        # Sorts the string, uses it as a key, and then appends the original
        # string as another value into hash table.
        sorted_string_to_anagrams[''.join(sorted(s))].append(s)

    return [
        group for group in sorted_string_to_anagrams.values() if len(group) >= 2
    ]
```

The computation consists of n calls to sort and n insertions into the hash table. Sorting all the keys has time complexity $O(nm \log m)$. The insertions add a time complexity of $O(nm)$, yielding $O(nm \log m)$ time complexity in total.

Design of a hashable class

Consider a class that represents contacts. For simplicity, assume each contact is a string. Suppose it is a hard requirement that the individual contacts are to be stored in a list and it's possible that the list contains duplicates. Two contacts should be equal if they contain the same set of strings, regardless of the ordering of the strings within the underlying list. Multiplicity is not important, i.e., three repetitions of the same contact is the same as a single instance of that contact. In order to be able to store contacts in a hash table, we first need to explicitly define equality, which we can do by forming sets from the lists and comparing the sets.

In our context, this implies that the hash function should depend on the strings present, but not their ordering; it should also consider only one copy if a string appears in duplicate form. It should be pointed out that the hash function and equals methods below are very inefficient. In practice, it would be advisable to cache the underlying set and the hash code, remembering to void these values on updates.

```
class ContactList:

    def __init__(self, names):
        '''
        names is a list of strings.
        '''
        self.names = names

    def __hash__(self):
        # Conceptually we want to hash the set of names. Since the set type is
        # mutable, it cannot be hashed. Therefore we use frozenset.
        return hash(frozenset(self.names))

    def __eq__(self, other):
        return set(self.names) == set(other.names)

def merge_contact_lists(contacts):
```

```
'''
contacts is a list of ContactList.
'''
return list(set(contacts))
```

The time complexity of computing the hash is $O(n)$, where n is the number of strings in the contact list. Hash codes are often cached for performance, with the caveat that the cache must be cleared if object fields that are referenced by the hash function are updated.

Hash tables have the **best theoretical and real-world performance** for lookup, insert and delete. Each of these operations has $O(1)$ time complexity. The $O(1)$ time complexity for insertion is for the average case—a single insert can take $O(n)$ if the hash table has to be resized.

Consider using a hash code as a **signature** to enhance performance, e.g., to filter out candidates.

Consider using a precomputed lookup table instead of boilerplate if-then code for mappings, e.g., from character to value, or character to character.

When defining your own type that will be put in a hash table, be sure you understand the relationship between **logical equality** and the fields the hash function must inspect. Specifically, anytime equality is implemented, it is imperative that the correct hash function is also implemented, otherwise when objects are placed in hash tables, logically equivalent objects may appear in different buckets, leading to lookups returning false, even when the searched item is present.

Sometimes you'll need a **multimap**, i.e., a map that contains multiple values for a single key, or a bi-directional map. If the language's standard libraries do not provide the functionality you need, learn how to implement a multimap using lists as values, or find a **third party library** that has a multimap.

Table 9.1: Top Tips for Hash Tables

Know your hash table libraries

There are multiple hash table-based data structures commonly used in Python—`set`, `dict`, `collections.defaultdict`, and `collections.Counter`. The difference between `set` and the other three is that `set` simply stores keys, whereas the others store key-value pairs. All have the property that they do not allow for duplicate keys, unlike, for example, `list`.

In a `dict`, accessing value associated with a key that is not present leads to a `KeyError` exception. However, a `collections.defaultdict` returns the default value of the type that was specified when the collection was instantiated, e.g., if `d = collections.defaultdict(list)`, then if `k not in d` then `d[k]` is `[]`. A `collections.Counter` is used for counting the number of occurrences of keys, with a number of set-like operations, as illustrated below.

```
c = collections.Counter(a=3, b=1)
d = collections.Counter(a=1, b=2)
# add two counters together: c[x] + d[x], collections.Counter({'a': 4, 'b': 3})
c + d
# subtract (keeping only positive counts), collections.Counter({'a': 2})
c - d
# intersection: min(c[x], d[x]), collections.Counter({'a': 1, 'b': 1})
```

```
c & d
# union: max(c[x], d[x]), collections.Counter({'a': 3, 'b': 2})
c | d
```

The most important operations for set are `s.add(42)`, `s.remove(42)`, `s.discard(123)`, `x in s`, as well as `s <= t` (is `s` a subset of `t`), and `s - t` (elements in `s` that are not in `t`).

The basic operations on the three key-value collections are similar to those on set. One difference is with iterators—iteration over a key-value collection yields the keys. To iterate over the key-value pairs, iterate over `items()`; to iterate over values, use `values()`. (The `keys()` method returns an iterator to the keys.)

Not every type is “hashable”, i.e., can be added to a set or used as a key in a dict. In particular, mutable containers are not hashable—this is to prevent a client from modifying an object after adding it to the container, since the lookup will then fail to find it if the slot that the modified object hashes to is different.

Note that the built-in `hash()` function can greatly simplify the implementation of a hash function for a user-defined class, i.e., implementing `__hash(self)___`.

9.1 IS AN ANONYMOUS LETTER CONSTRUCTIBLE?

Write a program which takes text for an anonymous letter and text for a magazine and determines if it is possible to write the anonymous letter using the magazine. The anonymous letter can be written using the magazine if for each character in the anonymous letter, the number of times it appears in the anonymous letter is no more than the number of times it appears in the magazine.

Hint: Count the number of distinct characters appearing in the letter.

Solution: A brute force approach is to count for each character in the character set the number of times it appears in the letter and in the magazine. If any character occurs more often in the letter than the magazine we return false, otherwise we return true. This approach is potentially slow because it iterates over all characters, including those that do not occur in the letter or magazine. It also makes multiple passes over both the letter and the magazine—as many passes as there are characters in the character set.

A better approach is to make a single pass over the letter, storing the character counts for the letter in a single hash table—keys are characters, and values are the number of times that character appears. Next, we make a pass over the magazine. When processing a character `c`, if `c` appears in the hash table, we reduce its count by 1; we remove it from the hash when its count goes to zero. If the hash becomes empty, we return true. If we reach the end of the letter and the hash is nonempty, we return false—each of the characters remaining in the hash occurs more times in the letter than the magazine.

```
def is_letter_constructible_from_magazine(letter_text, magazine_text):
    # Compute the frequencies for all chars in letter_text.
    char_frequency_for_letter = collections.Counter(letter_text)

    # Checks if characters in magazine_text can cover characters in
    # char_frequency_for_letter.
    for c in magazine_text:
        if c in char_frequency_for_letter:
```

```

char_frequency_for_letter[c] -= 1
if char_frequency_for_letter[c] == 0:
    del char_frequency_for_letter[c]
    if not char_frequency_for_letter:
        # All characters for letter_text are matched.
        return True

# Empty char_frequency_for_letter means every char in letter_text can be
# covered by a character in magazine_text.
return not char_frequency_for_letter

# Pythonic solution that exploits collections.Counter. Note that the
# subtraction only keeps keys with positive counts.
def is_letter_constructible_from_magazine_pythonic(letter_text, magazine_text):
    return (not collections.Counter(letter_text) -
            collections.Counter(magazine_text))

```

In the worst-case, the letter is not constructible or the last character of the magazine is essentially required. Therefore, the time complexity is $O(m + n)$ where m and n are the number of characters in the letter and magazine, respectively. The space complexity is the size of the hash table constructed in the pass over the letter, i.e., $O(L)$, where L is the number of distinct characters appearing in the letter.

If the characters are coded in ASCII, we could do away with the hash table and use a 256 entry integer array A , with $A[i]$ being set to the number of times the character i appears in the letter.

Sorting

PROBLEM 14 (*Meshing*). Two monotone sequences S, T , of lengths n, m , respectively, are stored in two systems of $n(p+1), m(p+1)$ consecutive memory locations, respectively: $s, s+1, \dots, s+n(p+1)-1$ and $t, t+1, \dots, t+m(p+1)-1$. . . It is desired to find a monotone permutation R of the sum $[S, T]$, and place it at the locations $r, r+1, \dots, r+(n+m)(p+1)-1$.

— “Planning And Coding Of Problems For An Electronic Computing Instrument,”

H. H. GOLDSTINE AND J. VON NEUMANN, 1948

Sorting—rearranging a collection of items into increasing or decreasing order—is a common problem in computing. Sorting is used to preprocess the collection to make searching faster (as we saw with binary search through an array), as well as identify items that are similar (e.g., students are sorted on test scores).

Naive sorting algorithms run in $O(n^2)$ time. A number of sorting algorithms run in $O(n \log n)$ time—heapsort, merge sort, and quicksort are examples. Each has its advantages and disadvantages: for example, heapsort is in-place but not stable; merge sort is stable but not in-place; quicksort runs $O(n^2)$ time in worst-case. (An in-place sort is one which uses $O(1)$ space; a stable sort is one where entries which are equal appear in their original order.)

A well-implemented quicksort is usually the best choice for sorting. We briefly outline alternatives that are better in specific circumstances.

For short arrays, e.g., 10 or fewer elements, insertion sort is easier to code and faster than asymptotically superior sorting algorithms. If every element is known to be at most k places from its final location, a min-heap can be used to get an $O(n \log k)$ algorithm. If there are a small number of distinct keys, e.g., integers in the range $[0..255]$, counting sort, which records for each element, the number of elements less than it, works well. This count can be kept in an array (if the largest number is comparable in value to the size of the set being sorted) or a BST, where the keys are the numbers and the values are their frequencies. If there are many duplicate keys we can add the keys to a BST, with linked lists for elements which have the same key; the sorted result can be derived from an in-order traversal of the BST.

Most sorting algorithms are not stable. Merge sort, carefully implemented, can be made stable. Another solution is to add the index as an integer rank to the keys to break ties.

Most sorting routines are based on a compare function. However, it is also possible to use numerical attributes directly, e.g., in radix sort.

The heap data structure is discussed in detail in Chapter 7. Briefly, a max-heap (min-heap) stores keys drawn from an ordered set. It supports $O(\log n)$ inserts and $O(1)$ time lookup for the maximum (minimum) element; the maximum (minimum) key can be deleted in $O(\log n)$ time. Heaps can be helpful in sorting problems, as illustrated by Problem 7.1 on Page 35.

Sorting boot camp

It's important to know how to use effectively the sort functionality provided by your language's library. Let's say we are given a student class that implements a compare method that compares students by name. Then by default, the array sort library routine will sort by name. To sort an array of students by GPA, we have to explicitly specify the compare function to the sort routine.

```
class Student(object):
    def __init__(self, name, grade_point_average):
        self.name = name
        self.grade_point_average = grade_point_average

    def __lt__(self, other):
        return self.name < other.name

students = [
    Student('A', 4.0), Student('C', 3.0), Student('B', 2.0), Student('D', 3.2)
]

# Sort according to __lt__ defined in Student. students remained unchanged.
students_sort_by_name = sorted(students)

# Sort students in-place by grade_point_average.
students.sort(key=lambda student: student.grade_point_average)
```

The time complexity of any reasonable library sort is $O(n \log n)$ for an array with n entries. Most library sorting functions are based on quicksort, which has $O(1)$ space complexity.

Sorting problems come in two flavors: (1.) **use sorting to make subsequent steps in an algorithm simpler**, and (2.) design a **custom sorting routine**. For the former, it's fine to use a library sort function, possibly with a custom comparator. For the latter, use a data structure like a BST, heap, or array indexed by values.

Certain problems become easier to understand, as well as solve, when the input is sorted. The most natural reason to sort is if the inputs have a **natural ordering**, and sorting can be used as a preprocessing step to **speed up searching**.

For **specialized input**, e.g., a very small range of values, or a small number of values, it's possible to sort in $O(n)$ time rather than $O(n \log n)$ time.

It's often the case that sorting can be implemented in **less space** than required by a brute-force approach.

Sometimes it is not obvious what to sort on, e.g., should a collection of intervals be sorted on starting points or endpoints? (Problem [10.2 on Page 51](#))

Table 10.1: Top Tips for Sorting

Know your sorting libraries

To sort a list in-place, use the `sort()` method; to sort an iterable, use the function `sorted()`.

- The `sort()` method implements a stable in-place sort for list objects. It returns `None`—the calling list itself is updated. It takes two arguments, both optional: `key=None`, and

`reverse=False` If `key` is not `None`, it is assumed to be a function which takes list elements and maps them to objects which are comparable—this function defines the sort order. For example, if `a=[1, 2, 4, 3, 5, 0, 11, 21, 100]` then `a.sort(key=lambda x: str(x))` maps integers to strings, and `a` is updated to `[0, 1, 100, 11, 2, 21, 3, 4, 5]`, i.e., the entries are sorted according to the lexicographical ordering of their string representation. If `reverse` is set to `True`, the sort is in descending order; otherwise it is in ascending order.

- The `sorted` function takes an iterable and return a new list containing all items from the iterable in ascending order. The original list is unchanged. For example, `b = sorted(a, key=lambda x: str(x))` leaves `a` unchanged, and assigns `b` to `[0, 1, 100, 11, 2, 21, 3, 4, 5]`. The optional arguments `key` and `reverse` work identically to `sort`.

10.1 COMPUTE THE INTERSECTION OF TWO SORTED ARRAYS

A natural implementation for a search engine is to retrieve documents that match the set of words in a query by maintaining an inverted index. Each page is assigned an integer identifier, its *document-ID*. An inverted index is a mapping that takes a word w and returns a sorted array of page-ids which contain w —the sort order could be, for example, the page rank in descending order. When a query contains multiple words, the search engine finds the sorted array for each word and then computes the intersection of these arrays—these are the pages containing all the words in the query. The most computationally intensive step of doing this is finding the intersection of the sorted arrays.

Write a program which takes as input two sorted arrays, and returns a new array containing elements that are present in both of the input arrays. The input arrays may have duplicate entries, but the returned array should be free of duplicates. For example, the input is $\langle 2, 3, 3, 5, 5, 6, 7, 7, 8, 12 \rangle$ and $\langle 5, 5, 6, 8, 8, 9, 10, 10 \rangle$, your output should be $\langle 5, 6, 8 \rangle$.

Hint: Solve the problem if the input array lengths differ by orders of magnitude. What if they are approximately equal?

Solution: The brute-force algorithm is a “loop join”, i.e., traversing through all the elements of one array and comparing them to the elements of the other array. Let m and n be the lengths of the two input arrays.

```
def intersect_two_sorted_arrays(A, B):
    return [a for i, a in enumerate(A) if (i == 0 or a != A[i - 1]) and a in B]
```

The brute-force algorithm has $O(mn)$ time complexity.

Since both the arrays are sorted, we can make some optimizations. First, we can iterate through the first array and use binary search in array to test if the element is present in the second array.

```
def intersect_two_sorted_arrays(A, B):
    def is_present(k):
        i = bisect.bisect_left(B, k)
        return i < len(B) and B[i] == k

    return [
        a for i, a in enumerate(A)
        if (i == 0 or a != A[i - 1]) and is_present(a)
    ]
```

The time complexity is $O(m \log n)$, where m is the length of the array being iterated over. We can further improve our run time by choosing the shorter array for the outer loop since if n is much smaller than m , then $n \log(m)$ is much smaller than $m \log(n)$.

This is the best solution if one set is much smaller than the other. However, it is not the best when the array lengths are similar because we are not exploiting the fact that both arrays are sorted. We can achieve linear runtime by simultaneously advancing through the two input arrays in increasing order. At each iteration, if the array elements differ, the smaller one can be eliminated. If they are equal, we add that value to the intersection and advance both. (We handle duplicates by comparing the current element with the previous one.) For example, if the arrays are $A = \langle 2, 3, 3, 5, 7, 11 \rangle$ and $B = \langle 3, 3, 7, 15, 31 \rangle$, then we know by inspecting the first element of each that 2 cannot belong to the intersection, so we advance to the second element of A . Now we have a common element, 3, which we add to the result, and then we advance in both arrays. Now we are at 3 in both arrays, but we know 3 has already been added to the result since the previous element in A is also 3. We advance in both again without adding to the intersection. Comparing 5 to 7, we can eliminate 5 and advance to the fourth element in A , which is 7, and equal to the element that B 's iterator holds, so it is added to the result. We then eliminate 11, and since no elements remain in A , we return $\langle 3, 7 \rangle$.

```
def intersect_two_sorted_arrays(A, B):
    i, j, intersection_A_B = 0, 0, []
    while i < len(A) and j < len(B):
        if A[i] == B[j]:
            if i == 0 or A[i] != A[i - 1]:
                intersection_A_B.append(A[i])
            i, j = i + 1, j + 1
        elif A[i] < B[j]:
            i += 1
        else: # A[i] > B[j].
            j += 1
    return intersection_A_B
```

Since we spend $O(1)$ time per input array element, the time complexity for the entire algorithm is $O(m + n)$.

10.2 RENDER A CALENDAR

Consider the problem of designing an online calendaring application. One component of the design is to render the calendar, i.e., display it visually.

Suppose each day consists of a number of events, where an event is specified as a start time and a finish time. Individual events for a day are to be rendered as nonoverlapping rectangular regions whose sides are parallel to the X - and Y -axes. Let the X -axis correspond to time. If an event starts at time b and ends at time e , the upper and lower sides of its corresponding rectangle must be at b and e , respectively. Figure 10.1 on the next page represents a set of events.

Suppose the Y -coordinates for each day's events must lie between 0 and L (a pre-specified constant), and each event's rectangle must have the same "height" (distance between the sides parallel to the X -axis). Your task is to compute the maximum height an event rectangle can have. In essence, this is equivalent to the following problem.

Write a program that takes a set of events, and determines the maximum number of events that take place concurrently.

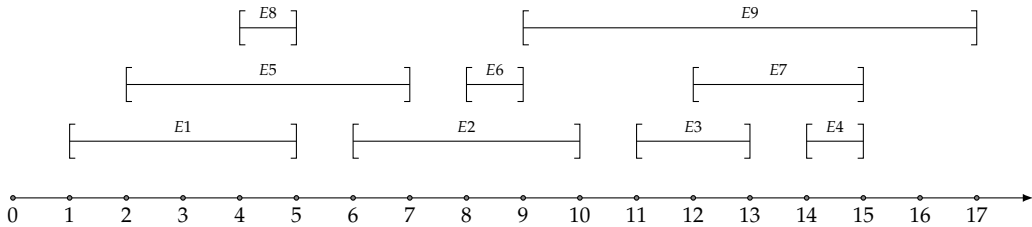


Figure 10.1: A set of nine events. The earliest starting event begins at time 1; the latest ending event ends at time 17. The maximum number of concurrent events is 3, e.g., $\{E1, E5, E8\}$ as well as others.

Hint: Focus on endpoints.

Solution: The number of events scheduled for a given time changes only at times that are start or end times of an event. This leads to the following brute-force algorithm. For each endpoint, compute the number of events that contain it. The maximum number of concurrent events is the maximum of this quantity over all endpoints. If there are n intervals, the total number of endpoints is $2n$. Computing the number of events containing an endpoint takes $O(n)$ time, since checking whether an interval contains a point takes $O(1)$ time. Therefore, the overall time complexity is $O(2n \times n) = O(n^2)$.

The inefficiency in the brute-force algorithm lies in the fact that it does not take advantage of locality, i.e., as we move from one endpoint to another. Intuitively, we can improve the run time by sorting the set of all the endpoints in ascending order. (If two endpoints have equal times, and one is a start time and the other is an end time, the one corresponding to a start time comes first. If both are start or finish times, we break ties arbitrarily.)

Now as we proceed through endpoints we can incrementally track the number of events taking place at that endpoint using a counter. For each endpoint that is the start of an interval, we increment the counter by 1, and for each endpoint that is the end of an interval, we decrement the counter by 1. The maximum value attained by the counter is maximum number of overlapping intervals.

For the example in Figure 10.1, the first seven endpoints are 1(start), 2(start), 4(start), 5(end), 5(end), 6(start), 7(end). The counter values are updated to 1, 2, 3, 2, 1, 2, 1.

```
# Event is a tuple (start_time, end_time)
Event = collections.namedtuple('Event', ('start', 'finish'))

# Endpoint is a tuple (start_time, 0) or (end_time, 1) so that if times
# are equal, start_time comes first
Endpoint = collections.namedtuple('Endpoint', ('time', 'is_start'))

def find_max_simultaneous_events(A):
    # Builds an array of all endpoints.
    E = ([Endpoint(event.start, True) for event in A] +
         [Endpoint(event.finish, False) for event in A])
    # Sorts the endpoint array according to the time, breaking ties by putting
    # start times before end times.
    E.sort(key=lambda e: (e.time, not e.is_start))

    # Track the number of simultaneous events, record the maximum number of
    # simultaneous events.
    max_num_simultaneous_events, num_simultaneous_events = 0, 0
    for e in E:
```

```
if e.is_start:
    num_simultaneous_events += 1
    max_num_simultaneous_events = max(num_simultaneous_events,
                                       max_num_simultaneous_events)
else:
    num_simultaneous_events -= 1
return max_num_simultaneous_events
```

Sorting the endpoint array takes $O(n \log n)$ time; iterating through the sorted array takes $O(n)$ time, yielding an $O(n \log n)$ time complexity. The space complexity is $O(n)$, which is the size of the endpoint array.

Variant: Users $1, 2, \dots, n$ share an Internet connection. User i uses b_i bandwidth from time s_i to f_i , inclusive. What is the peak bandwidth usage?

Binary Search Trees

The number of trees which can be formed with $n + 1$ given knots $\alpha, \beta, \gamma, \dots = (n + 1)^{n-1}$.

—“A Theorem on Trees,”
A. CAYLEY, 1889

BSTs are a workhorse of data structures and can be used to solve almost every data structures problem reasonably efficiently. They offer the ability to efficiently search for a key as well as find the *min* and *max* elements, look for the successor or predecessor of a search key (which itself need not be present in the BST), and enumerate the keys in a range in sorted order.

BSTs are similar to arrays in that the stored values (the “keys”) are stored in a sorted order. However, unlike with a sorted array, keys can be added to and deleted from a BST efficiently. Specifically, a BST is a binary tree as defined in Chapter 6 in which the nodes store keys that are comparable, e.g., integers or strings. The keys stored at nodes have to respect the BST property—the key stored at a node is greater than or equal to the keys stored at the nodes of its left subtree and less than or equal to the keys stored in the nodes of its right subtree. Figure 11.1 on the next page shows a BST whose keys are the first 16 prime numbers.

Key lookup, insertion, and deletion take time proportional to the height of the tree, which can in worst-case be $O(n)$, if insertions and deletions are naively implemented. However, there are implementations of insert and delete which guarantee that the tree has height $O(\log n)$. These require storing and updating additional data at the tree nodes. Red-black trees are an example of height-balanced BSTs and are widely used in data structure libraries.

A common mistake with BSTs is that an object that’s present in a BST is to be updated. The consequence is that a lookup for that object returns false, even though it’s still in the BST. As a rule, avoid putting mutable objects in a BST. Otherwise, when a mutable object that’s in a BST is to be updated, always first remove it from the tree, then update it, then add it back.

The BST prototype is as follows:

```
class BSTNode:

    def __init__(self, data=None, left=None, right=None):
        self.data, self.left, self.right = data, left, right
```

Binary search trees boot camp

Searching is the single most fundamental application of BSTs. Unlike a hash table, a BST offers the ability to find the min and max elements, and find the next largest/next smallest element. These operations, along with lookup, delete and find, take time $O(\log n)$ for library implementations of BSTs. Both BSTs and hash tables use $O(n)$ space—in practice, a BST uses slightly more space.

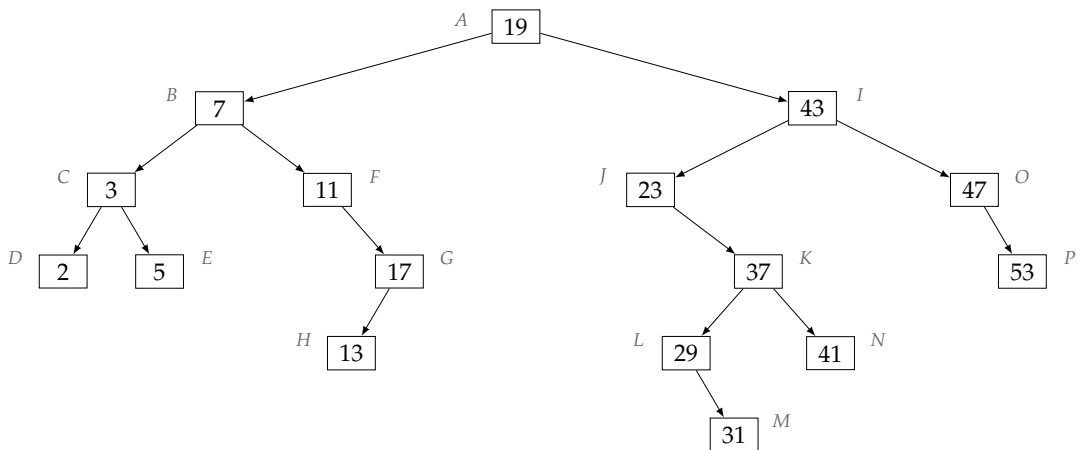


Figure 11.1: An example of a BST.

The following program demonstrates how to check if a given value is present in a BST. It is a nice illustration of the power of recursion when operating on BSTs.

```
def search_bst(tree, key):
    return (tree if not tree or tree.data == key else search_bst(tree.left, key)
            if key < tree.data else search_bst(tree.right, key))
```

Since the program descends tree with in each step, and spends $O(1)$ time per level, the time complexity is $O(h)$, where h is the height of the tree.

With a BST you can **iterate** through elements in **sorted order** in time $O(n)$ (regardless of whether it is balanced).

Some problems need a **combination of a BST and a hashtable**. For example, if you insert student objects into a BST and entries are ordered by GPA, and then a student's GPA needs to be updated and all we have is the student's name and new GPA, we cannot find the student by name without a full traversal. However, with an additional hash table, we can directly go to the corresponding entry in the tree.

The BST property is a **global property**—a binary tree may have the property that each node's key is greater than the key at its left child and smaller than the key at its right child, but it may not be a BST.

Table 11.1: Top Tips for Binary Search Trees

Know your binary search tree libraries

Some of the problems in this chapter entail writing a BST class; for others, you can use a BST library. Python does not come with a built-in BST library.

The `sortedcontainers` module the best-in-class module for sorted sets and sorted dictionaries—it is performant, has a clean API that is well documented, with a responsive community. The underlying data structure is a sorted list of sorted lists. Its asymptotic time complexity for inserts and deletes is $O(\sqrt{n})$ since these operations entail insertion into a list of length roughly $\sqrt{n}$, rather

than the $O(\log n)$ of balanced BSTs. In practice, this is not an issue, since CPUs are highly optimized for block data movements.

In the interests of pedagogy, we have elected to use the `bintrees` module which implements sorted sets and sorted dictionaries using balanced BSTs. However, any reasonable interviewer should accept `sortedcontainers` wherever we use `bintrees`.

Below, we describe the functionalities added by `bintrees`.

- `insert(e)` inserts new element e in the BST.
- `discard(e)` removes e in the BST if present.
- `min_item()/max_item()` yield the smallest and largest key-value pair in the BST.
- `min_key()/max_key()` yield the smallest and largest key in the BST.
- `pop_min()/pop_max()` remove the return the smallest and largest key-value pair in the BST.

It's particularly important to note that these operations take $O(\log n)$, since they are backed by the underlying tree.

The following program illustrates the use of `bintrees`.

```
t = bintrees.RBTree([(5, 'Alfa'), (2, 'Bravo'), (7, 'Charlie'), (3, 'Delta'),
                    (6, 'Echo')])

print(t[2]) # 'Bravo'

print(t.min_item(), t.max_item()) # (2, 'Bravo'), (7, 'Charlie')

# {2: 'Bravo', 3: 'Delta', 5: 'Alfa', 6: 'Echo', 7: 'Charlie', 9: 'Golf'}
t.insert(9, 'Golf')
print(t)

print(t.min_key(), t.max_key()) # 2, 9

t.discard(3)
print(t) # {2: 'Bravo', 5: 'Alfa', 6: 'Echo', 7: 'Charlie', 9: 'Golf'}

# a = (2: 'Bravo')
a = t.pop_min()
print(t) # {5: 'Alfa', 6: 'Echo', 7: 'Charlie', 9: 'Golf'}

# b = (9, 'Golf')
b = t.pop_max()
print(t) # {5: 'Alfa', 6: 'Echo', 7: 'Charlie'}
```

11.1 TEST IF A BINARY TREE SATISFIES THE BST PROPERTY

Write a program that takes as input a binary tree and checks if the tree satisfies the BST property.

Hint: Is it correct to check for each node that its key is greater than or equal to the key at its left child and less than or equal to the key at its right child?

Solution: A direct approach, based on the definition of a BST, is to begin with the root, and compute the maximum key stored in the root's left subtree, and the minimum key in the root's right subtree. We check that the key at the root is greater than or equal to the maximum from the left subtree and less than or equal to the minimum from the right subtree. If both these checks pass, we recursively check the root's left and right subtrees. If either check fails, we return false.

Computing the minimum key in a binary tree is straightforward: we take the minimum of the key stored at its root, the minimum key of the left subtree, and the minimum key of the right subtree. The maximum key is computed similarly. Note that the minimum can be in either subtree, since a general binary tree may not satisfy the BST property.

The problem with this approach is that it will repeatedly traverse subtrees. In the worst-case, when the tree is a BST and each node's left child is empty, the complexity is $O(n^2)$, where n is the number of nodes. The complexity can be improved to $O(n)$ by caching the largest and smallest keys at each node; this requires $O(n)$ additional storage for the cache.

We now present two approaches which have $O(n)$ time complexity.

The first approach is to check constraints on the values for each subtree. The initial constraint comes from the root. Every node in its left (right) subtree must have a key less than or equal (greater than or equal) to the key at the root. This idea generalizes: if all nodes in a tree must have keys in the range $[l, u]$, and the key at the root is w (which itself must be between $[l, u]$, otherwise the requirement is violated at the root itself), then all keys in the left subtree must be in the range $[l, w]$, and all keys stored in the right subtree must be in the range $[w, u]$.

As a concrete example, when applied to the BST in Figure 11.1 on Page 55, the initial range is $[-\infty, \infty]$. For the recursive call on the subtree rooted at B , the constraint is $[-\infty, 19]$; the 19 is the upper bound required by A on its left subtree. For the recursive call starting at the subtree rooted at F , the constraint is $[7, 19]$. For the recursive call starting at the subtree rooted at K , the constraint is $[23, 43]$. The binary tree in Figure 6.1 on Page 28 is identified as not being a BST when the recursive call reaches C —the constraint is $[-\infty, 6]$, but the key at F is 271, so the tree cannot satisfy the BST property.

```
def is_binary_tree_bst(tree, low_range=float('-inf'), high_range=float('inf')):
    if not tree:
        return True
    elif not low_range <= tree.data <= high_range:
        return False
    return (is_binary_tree_bst(tree.left, low_range, tree.data) and
            is_binary_tree_bst(tree.right, tree.data, high_range))
```

The time complexity is $O(n)$, and the additional space complexity is $O(h)$, where h is the height of the tree.

Alternatively, we can use the fact that an inorder traversal visits keys in sorted order. Furthermore, if an inorder traversal of a binary tree visits keys in sorted order, then that binary tree must be a BST. (This follows directly from the definition of a BST and the definition of an inorder walk.) Thus we can check the BST property by performing an inorder traversal, recording the key stored at the last visited node. Each time a new node is visited, its key is compared with the key of the previously visited node. If at any step in the walk, the key at the previously visited node is greater than the node currently being visited, we have a violation of the BST property.

All these approaches explore the left subtree first. Therefore, even if the BST property does not hold at a node which is close to the root (e.g., the key stored at the right child is less than the key stored at the root), their time complexity is still $O(n)$.

We can search for violations of the BST property in a BFS manner, thereby reducing the time complexity when the property is violated at a node whose depth is small.

Specifically, we use a queue, where each queue entry contains a node, as well as an upper and a lower bound on the keys stored at the subtree rooted at that node. The queue is initialized to the

root, with lower bound $-\infty$ and upper bound ∞ . We iteratively check the constraint on each node. If it violates the constraint we stop—the BST property has been violated. Otherwise, we add its children along with the corresponding constraint.

For the example in Figure 11.1 on Page 55, we initialize the queue with $(A, [-\infty, \infty])$. Each time we pop a node, we first check the constraint. We pop the first entry, $(A, [-\infty, \infty])$, and add its children, with the corresponding constraints, i.e., $(B, [-\infty, 19])$ and $(I, [19, \infty])$. Next we pop $(B, [-\infty, 19])$, and add its children, i.e., $(C, [-\infty, 7])$ and $(D, [7, 19])$. Continuing through the nodes, we check that all nodes satisfy their constraints, and thus verify the tree is a BST.

If the BST property is violated in a subtree consisting of nodes within a particular depth, the violation will be discovered without visiting any nodes at a greater depth. This is because each time we enqueue an entry, the lower and upper bounds on the node's key are the tightest possible.

```
def is_binary_tree_bst(tree):
    QueueEntry = collections.namedtuple('QueueEntry',
                                         ('node', 'lower', 'upper'))

    bfs_queue = collections.deque(
        [QueueEntry(tree, float('-inf'), float('inf'))])

    while bfs_queue:
        front = bfs_queue.popleft()
        if front.node:
            if not front.lower <= front.node.data <= front.upper:
                return False
            bfs_queue += [
                QueueEntry(front.node.left, front.lower, front.node.data),
                QueueEntry(front.node.right, front.node.data, front.upper)
            ]
    return True
```

The time complexity is $O(n)$, and the additional space complexity is $O(n)$.

Recursion

The power of recursion evidently lies in the possibility of defining an infinite set of objects by a finite statement. In the same manner, an infinite number of computations can be described by a finite recursive program, even if this program contains no explicit repetitions.

— “Algorithms + Data Structures = Programs,”
N. E. WIRTH, 1976

Recursion is an approach to problem solving where the solution depends partially on solutions to smaller instances of related problems. It is often appropriate when the input is expressed using recursive rules, such as a computer grammar. More generally, searching, enumeration, divide-and-conquer, and decomposing a complex problem into a set of similar smaller instances are all scenarios where recursion may be suitable.

A recursive function consists of base cases and calls to the same function with different arguments. Two key ingredients to a successful use of recursion are identifying the base cases, which are to be solved directly, and ensuring progress, that is the recursion converges to the solution.

Both backtracking and branch-and-bound are naturally formulated using recursion.

Recursion boot camp

The Euclidean algorithm for calculating the greatest common divisor (GCD) of two numbers is a classic example of recursion. The central idea is that if $y > x$, the GCD of x and y is the GCD of x and $y - x$. For example, $\text{GCD}(156, 36) = \text{GCD}((156 - 36), 36) = \text{GCD}(120, 36)$. By extension, this implies that the GCD of x and y is the GCD of x and $y \bmod x$, i.e., $\text{GCD}(156, 36) = \text{GCD}((156 \bmod 36), 36) = \text{GCD}(12, 36 \bmod 12 = 0) = 12$.

```
def gcd(x, y):
    return x if y == 0 else gcd(y, x % y)
```

Since with each recursive step one of the arguments is at least halved, it means that the time complexity is $O(\log \max(x, y))$. Put another way, the time complexity is $O(n)$, where n is the number of bits needed to represent the inputs. The space complexity is also $O(n)$, which is the maximum depth of the function call stack. (The program is easily converted to one which loops, thereby reducing the space complexity to $O(1)$.)

Now we describe a problem that can be elegantly solved using a recursive divide-and-conquer algorithm. A triomino is formed by joining three unit-sized squares in an L-shape. A mutilated chessboard (henceforth 8×8 Mboard) is made up of 64 unit-sized squares arranged in an 8×8 square, minus the top-left square, as depicted in Figure 12.1(a) on the next page. Suppose you are asked to design an algorithm that computes a placement of 21 triominoes that covers the 8×8

Mboard. Since the 8×8 Mboard contains 63 squares, and we have 21 triominoes, a valid placement cannot have overlapping triominoes or triominoes which extend out of the 8×8 Mboard.

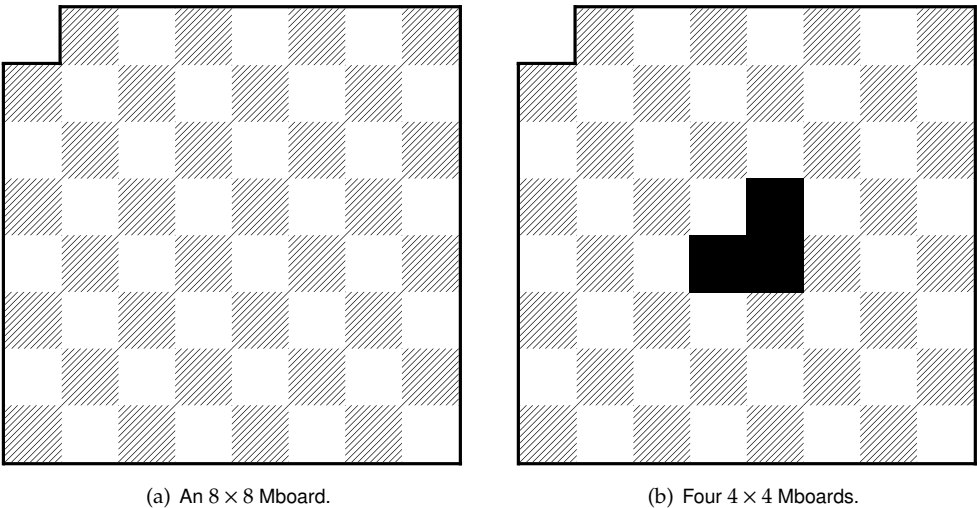


Figure 12.1: Mutilated chessboards.

Divide-and-conquer is a good strategy for this problem. Instead of the 8×8 Mboard, let’s consider an $n \times n$ Mboard. A 2×2 Mboard can be covered with one triomino since it is of the same exact shape. You may hypothesize that a triomino placement for an $n \times n$ Mboard with the top-left square missing can be used to compute a placement for an $(n + 1) \times (n + 1)$ Mboard. However, you will quickly see that this line of reasoning does not lead you anywhere.

Another hypothesis is that if a placement exists for an $n \times n$ Mboard, then one also exists for a $2n \times 2n$ Mboard. Now we can apply divide-and-conquer. Take four $n \times n$ Mboards and arrange them to form a $2n \times 2n$ square in such a way that three of the Mboards have their missing square set towards the center and one Mboard has its missing square outward to coincide with the missing corner of a $2n \times 2n$ Mboard, as shown in Figure 12.1(b). The gap in the center can be covered with a triomino and, by hypothesis, we can cover the four $n \times n$ Mboards with triominoes as well. Hence, a placement exists for any n that is a power of 2. In particular, a placement exists for the $2^3 \times 2^3$ Mboard; the recursion used in the proof directly yields the placement.

12.1 GENERATE THE POWER SET

The power set of a set S is the set of all subsets of S , including both the empty set $\emptyset$ and S itself. The power set of $\{0, 1, 2\}$ is graphically illustrated in Figure 12.2 on the next page.

Write a function that takes as input a set and returns its power set.

Hint: There are 2^n subsets for a given set S of size n . There are 2^k k -bit words.

Solution: A brute-force way is to compute all subsets U that do not include a particular element (which could be any single element). Then we compute all subsets V which do include that element. Each subset set must appear in U or in V , so the final result is just $U \cup V$. The construction is recursive, and the base case is when the input set is empty, in which case we return $\{\{\}\}$.

Recursion is especially suitable when the **input is expressed using recursive rules** such as a computer grammar.

Recursion is a good choice for **search**, **enumeration**, and **divide-and-conquer**.

Use recursion as **alternative to deeply nested iteration** loops. For example, recursion is much better when you have an undefined number of levels, such as the IP address problem generalized to k substrings.

If you are asked to **remove recursion** from a program, consider mimicking call stack with the **stack data structure**.

Recursion can be easily removed from a **tail-recursive** program by using a while-loop—no stack is needed. (Optimizing compilers do this.)

If a recursive function may end up being called with the **same arguments** more than once, **cache** the results—this is the idea behind Dynamic Programming (Chapter 13).

Table 12.1: Top Tips for Recursion

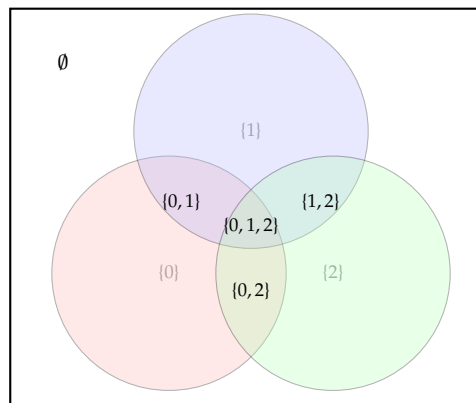


Figure 12.2: The power set of $\{0, 1, 2\}$ is $\{\emptyset, \{0\}, \{1\}, \{2\}, \{0, 1\}, \{1, 2\}, \{0, 2\}, \{0, 1, 2\}\}$.

As an example, let $S = \{0, 1, 2\}$. Pick any element, e.g., 0. First, we recursively compute all subsets of $\{1, 2\}$. To do this, we select 1. This leaves us with $\{2\}$. Now we pick 2, and get to a base case. So the set of subsets of $\{2\}$ is $\{\}$ union with $\{2\}$, i.e., $\{\{\}, \{2\}\}$. The set of subsets of $\{1, 2\}$ then is $\{\{\}, \{2\}\}$ union with $\{\{1\}, \{1, 2\}\}$, i.e., $\{\{\}, \{2\}, \{1\}, \{1, 2\}\}$. The set of subsets of $\{0, 1, 2\}$ then is $\{\{\}, \{2\}, \{1\}, \{1, 2\}\}$ union with $\{\{0\}, \{0, 2\}, \{0, 1\}, \{0, 1, 2\}\}$, i.e., $\{\{\}, \{2\}, \{1\}, \{1, 2\}, \{0\}, \{0, 2\}, \{0, 1\}, \{0, 1, 2\}\}$.

```
def generate_power_set(input_set):
    # Generate all subsets whose intersection with input_set[0], ...,
    # input_set[to_be_selected - 1] is exactly selected_so_far.
    def directed_power_set(to_be_selected, selected_so_far):
        if to_be_selected == len(input_set):
            power_set.append(list(selected_so_far))
            return

        directed_power_set(to_be_selected + 1, selected_so_far)
        # Generate all subsets that contain input_set[to_be_selected].
        directed_power_set(to_be_selected + 1,
```

```

        selected_so_far + [input_set[to_be_selected]])

power_set = []
directed_power_set(0, [])
return power_set

```

The number of recursive calls, $C(n)$ satisfies the recurrence $C(n) = 2C(n - 1)$, which solves to $C(n) = O(2^n)$. Since we spend $O(n)$ time within a call, the time complexity is $O(n2^n)$. The space complexity is $O(n2^n)$, since there are 2^n subsets, and the average subset size is $n/2$. If we just want to print the subsets, rather than returning all of them, we simply perform a print instead of adding the subset to the result, which reduces the space complexity to $O(n)$ —the time complexity remains the same.

For a given ordering of the elements of S , there exists a one-to-one correspondence between the 2^n bit arrays of length n and the set of all subsets of S —the 1s in the n -length bit array v indicate the elements of S in the subset corresponding to v . For example, if $S = \{a, b, c, d\}$, the bit array $\langle 1, 0, 1, 1 \rangle$ denotes the subset $\{a, c, d\}$. This observation can be used to derive a nonrecursive algorithm for enumerating subsets.

In particular, when n is less than or equal to the width of an integer on the architecture (or language) we are working on, we can enumerate bit arrays by enumerating integers in $[0, 2^n - 1]$ and examining the indices of bits set in these integers. These indices are determined by first isolating the lowest set bit by computing $y = x \& \sim(x - 1)$, and then getting the index by computing $\log y$.

```

def generate_power_set(S):
    power_set = []
    for int_for_subset in range(1 << len(S)):
        bit_array = int_for_subset
        subset = []
        while bit_array:
            subset.append(int(math.log2(bit_array & ~(bit_array - 1))))
            bit_array &= bit_array - 1
        power_set.append(subset)
    return power_set

```

Since each set takes $O(n)$ time to compute, the time complexity is $O(n2^n)$. In practice, this approach is very fast. Furthermore, its space complexity is $O(n)$ when we want to just enumerate subsets, e.g., to print them, rather than to return all the subsets.

Variant: Solve this problem when the input array may have duplicates, i.e., denotes a multiset. You should not repeat any multiset. For example, if $A = \langle 1, 2, 3, 2 \rangle$, then you should return $\langle \langle \rangle, \langle 1 \rangle, \langle 2 \rangle, \langle 3 \rangle, \langle 1, 2 \rangle, \langle 1, 3 \rangle, \langle 2, 2 \rangle, \langle 2, 3 \rangle, \langle 1, 2, 2 \rangle, \langle 1, 2, 3 \rangle, \langle 2, 2, 3 \rangle, \langle 1, 2, 2, 3 \rangle \rangle$.

Dynamic Programming

The important fact to observe is that we have attempted to solve a maximization problem involving a particular value of x and a particular value of N by first solving the general problem involving an arbitrary value of x and an arbitrary value of N .

—“Dynamic Programming,”
R. E. BELLMAN, 1957

DP is a general technique for solving optimization, search, and counting problems that can be decomposed into subproblems. You should consider using DP whenever you have to make choices to arrive at the solution, specifically, when the solution relates to subproblems.

Like divide-and-conquer, DP solves the problem by combining the solutions of multiple smaller problems, but what makes DP different is that the same subproblem may reoccur. Therefore, a key to making DP efficient is caching the results of intermediate computations. Problems whose solutions use DP are a popular choice for hard interview questions.

To illustrate the idea underlying DP, consider the problem of computing Fibonacci numbers. The first two Fibonacci numbers are 0 and 1. Successive numbers are the sums of the two previous numbers. The first few Fibonacci numbers are 0, 1, 1, 2, 3, 5, 8, 13, 21, The Fibonacci numbers arise in many diverse applications—biology, data structure analysis, and parallel computing are some examples.

Mathematically, the n th Fibonacci number $F(n)$ is given by the equation $F(n) = F(n - 1) + F(n - 2)$, with $F(0) = 0$ and $F(1) = 1$. A function to compute $F(n)$ that recursively invokes itself has a time complexity that is exponential in n . This is because the recursive function computes some $F(i)$ s repeatedly. Figure 13.1 on the following page graphically illustrates how the function is repeatedly called with the same arguments.

Caching intermediate results makes the time complexity for computing the n th Fibonacci number linear in n , albeit at the expense of $O(n)$ storage.

```
def fibonacci(n, cache={}):  
    if n <= 1:  
        return n  
    elif n not in cache:  
        cache[n] = fibonacci(n - 1) + fibonacci(n - 2)  
    return cache[n]
```

Minimizing cache space is a recurring theme in DP. Now we show a program that computes $F(n)$ in $O(n)$ time and $O(1)$ space. Conceptually, in contrast to the above program, this program iteratively fills in the cache in a bottom-up fashion, which allows it to reuse cache storage to reduce the space complexity of the cache.

```
def fibonacci(n):
```

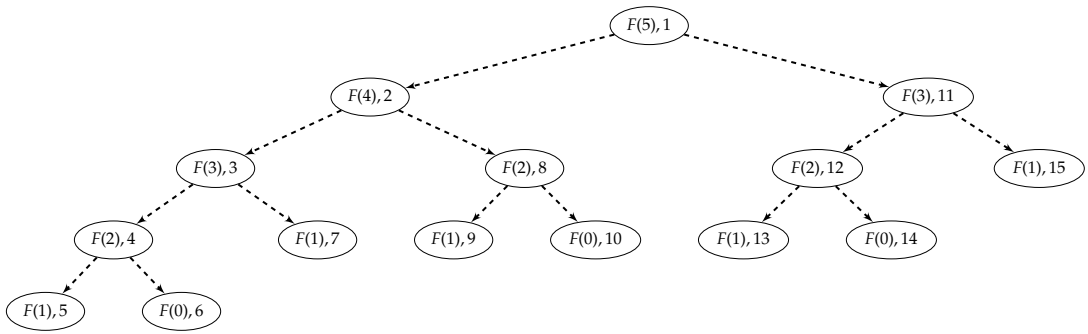


Figure 13.1: Tree of recursive calls when naively computing the 5th Fibonacci number, $F(5)$. Each node is a call: $F(x)$ indicates a call with argument x , and the italicized numbers on the right are the sequence in which calls take place. The children of a node are the subcalls made by that call. Note how there are 2 calls to $F(3)$, and 3 calls to each of $F(2)$, $F(1)$, and $F(0)$.

```

if n <= 1:
    return n

f_minus_2, f_minus_1 = 0, 1
for _ in range(1, n):
    f = f_minus_2 + f_minus_1
    f_minus_2, f_minus_1 = f_minus_1, f
return f_minus_1

```

The key to solving a DP problem efficiently is finding a way to break the problem into subproblems such that

- the original problem can be solved relatively easily once solutions to the subproblems are available, and
- these subproblem solutions are cached.

Usually, but not always, the subproblems are easy to identify.

Here is a more sophisticated application of DP. Consider the following problem: find the maximum sum over all subarrays of a given array of integer. As a concrete example, the maximum subarray for the array in Figure 13.2 starts at index 0 and ends at index 3.

904	40	523	12	-335	-385	-124	481	-31
$A[0]$	$A[1]$	$A[2]$	$A[3]$	$A[4]$	$A[5]$	$A[6]$	$A[7]$	$A[8]$

Figure 13.2: An array with a maximum subarray sum of 1479.

The brute-force algorithm, which computes each subarray sum, has $O(n^3)$ time complexity—there are $\frac{n(n+1)}{2}$ subarrays, and each subarray sum takes $O(n)$ time to compute. The brute-force algorithm can be improved to $O(n^2)$, at the cost of $O(n)$ additional storage, by first computing $S[k] = \sum A[0, k]$ for all k . The sum for $A[i, j]$ is then $S[j] - S[i - 1]$, where $S[-1]$ is taken to be 0.

Here is a natural divide-and-conquer algorithm. Take $m = \lfloor \frac{n}{2} \rfloor$ to be the middle index of A . Solve the problem for the subarrays $L = A[0, m]$ and $R = A[m + 1, n - 1]$. In addition to the answers for each, we also return the maximum subarray sum l for a subarray ending at the last entry in L , and the maximum subarray sum r for a subarray starting at the first entry of R . The maximum subarray

sum for A is the maximum of $l + r$, the answer for L , and the answer for R . The time complexity analysis is similar to that for quicksort, and the time complexity is $O(n \log n)$. Because of off-by-one errors, it takes some time to get the program just right.

Now we will solve this problem by using DP. A natural thought is to assume we have the solution for the subarray $A[0, n - 2]$. However, even if we knew the largest sum subarray for subarray $A[0, n - 2]$, it does not help us solve the problem for $A[0, n - 1]$. Instead we need to know the subarray amongst all subarrays $A[0, i]$, $i < n - 1$, with the smallest subarray sum. The desired value is $S[n - 1]$ minus this subarray's sum. We compute this value by iterating through the array. For each index j , the maximum subarray sum ending at j is equal to $S[j] - \min_{k \leq j} S[k]$. During the iteration, we track the minimum $S[k]$ we have seen so far and compute the maximum subarray sum for each index. The time spent per index is constant, leading to an $O(n)$ time and $O(1)$ space solution. It is legal for all array entries to be negative, or the array to be empty. The algorithm handles these input cases correctly.

```
def find_maximum_subarray(A):
    min_sum = max_sum = 0
    for running_sum in itertools.accumulate(A):
        min_sum = min(min_sum, running_sum)
        max_sum = max(max_sum, running_sum - min_sum)
    return max_sum
```

Dynamic programming boot camp

The programs presented in the introduction for computing the Fibonacci numbers and the maximum subarray sum are good illustrations of DP.

13.1 COUNT THE NUMBER OF WAYS TO TRAVERSE A 2D ARRAY

In this problem you are to count the number of ways of starting at the top-left corner of a 2D array and getting to the bottom-right corner. All moves must either go right or down. For example, we show three ways in a 5×5 2D array in Figure 13.3. (As we will see, there are a total of 70 possible ways for this example.)

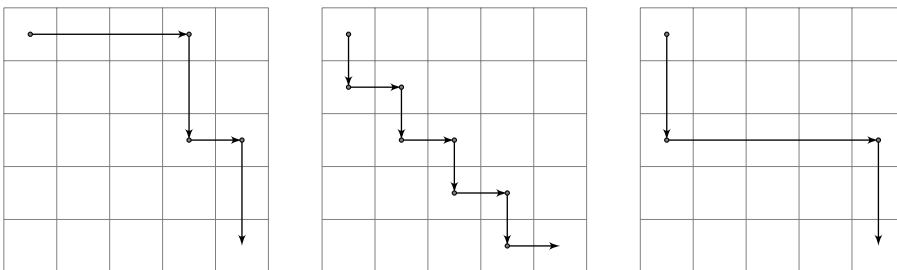


Figure 13.3: Paths through a 2D array.

Write a program that counts how many ways you can go from the top-left to the bottom-right in a 2D array.

Hint: If $i > 0$ and $j > 0$, you can get to (i, j) from $(i - 1, j)$ or $(j - 1, i)$.

Consider using DP whenever you have to **make choices** to arrive at the solution. Specifically, DP is applicable when you can construct a solution to the given instance from solutions to subinstances of smaller problems of the same kind.

In addition to optimization problems, DP is also **applicable to counting and decision problems**—any problem where you can express a solution recursively in terms of the same computation on smaller instances.

Although conceptually DP involves recursion, often for efficiency the cache is **built “bottom-up”**, i.e., iteratively.

When DP is implemented recursively the cache is typically a dynamic data structure such as a hash table or a BST; when it is implemented iteratively the cache is usually a one- or multi-dimensional array.

To save space, **cache space** may be **recycled** once it is known that a set of entries will not be looked up again.

A common **mistake** in solving DP problems is trying to think of the recursive case by splitting the problem into **two equal halves**, *a la* quicksort, i.e., solve the subproblems for subarrays $A[0, \lfloor \frac{n}{2} \rfloor]$ and $A[\lfloor \frac{n}{2} \rfloor + 1, n]$ and combine the results. However, in most cases, these two subproblems are not sufficient to solve the original problem.

DP is based on **combining solutions** to subproblems to **yield a solution** to the original problem. However, for some problems DP will not work. For example, if you need to compute the longest path from City 1 to City 2 without repeating an intermediate city, and this longest path passes through City 3, then the subpaths from City 1 to City 3 and City 3 to City 2 may not be individually longest paths without repeated cities.

Table 13.1: Top Tips for Dynamic Programming

Solution: A brute-force approach is to enumerate all possible paths. This can be done using recursion. However, there is a combinatorial explosion in the number of paths, which leads to huge time complexity.

The problem statement asks for the number of paths, so we focus our attention on that. A key observation is that because paths must advance down or right, the number of ways to get to the bottom-right entry is the number of ways to get to the entry immediately above it, plus the number of ways to get to the entry immediately to its left. Let's treat the origin (0,0) as the top-left entry. Generalizing, the number of ways to get to (i, j) is the number of ways to get to $(i - 1, j)$ plus the number of ways to get to $(i, j - 1)$. (If $i = 0$ or $j = 0$, there is only one way to get to (i, j) from the origin.) This is the basis for a recursive algorithm to count the number of paths. Implemented naively, the algorithm has exponential time complexity—it repeatedly recurses on the same locations. The solution is to cache results. For example, the number of ways to get to (i, j) for the configuration in Figure 13.3 on the previous page is cached in a matrix as shown in Figure 13.4 on the facing page.

```
def number_of_ways(n, m):
    def compute_number_of_ways_to_xy(x, y):
        if x == y == 0:
            return 1

        if number_of_ways[x][y] == 0:
```

```

ways_top = 0 if x == 0 else compute_number_of_ways_to_xy(x - 1, y)
ways_left = 0 if y == 0 else compute_number_of_ways_to_xy(x, y - 1)
number_of_ways[x][y] = ways_top + ways_left
return number_of_ways[x][y]

number_of_ways = [[0] * m for _ in range(n)]
return compute_number_of_ways_to_xy(n - 1, m - 1)

```

The time complexity is $O(nm)$, and the space complexity is $O(nm)$, where n is the number of rows and m is the number of columns.

1	1	1	1	1
1	2	3	4	5
1	3	6	10	15
1	4	10	20	35
1	5	15	35	70

Figure 13.4: The number of ways to get from $(0, 0)$ to (i, j) for $0 \leq i, j \leq 4$.

A more analytical way of solving this problem is to use the fact that each path from $(0, 0)$ to $(n - 1, m - 1)$ is a sequence of $m - 1$ horizontal steps and $n - 1$ vertical steps. There are $\binom{n+m-2}{n-1} = \frac{(n+m-2)!}{(n-1)!(m-1)!}$ such paths.

Variant: Solve the same problem using $O(\min(n, m))$ space.

Variant: Solve the same problem in the presence of obstacles, specified by a Boolean 2D array, where the presence of a true value represents an obstacle.

Variant: A fisherman is in a rectangular sea. The value of the fish at point (i, j) in the sea is specified by an $n \times m$ 2D array A . Write a program that computes the maximum value of fish a fisherman can catch on a path from the upper leftmost point to the lower rightmost point. The fisherman can only move down or right, as illustrated in Figure 13.5.

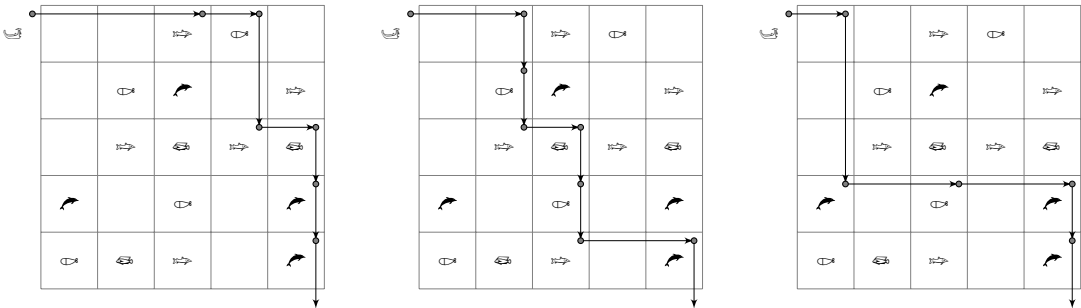


Figure 13.5: Alternate paths for a fisherman. Different types of fish have different values, which are known to the fisherman.

Variant: Solve the same problem when the fisherman can begin and end at any point. He must still move down or right. (Note that the value at (i, j) may be negative.)

Variant: A decimal number is a sequence of digits, i.e., a sequence over $\{0, 1, 2, \dots, 9\}$. The sequence has to be of length 1 or more, and the first element in the sequence cannot be 0. Call a decimal number D *monotone* if $D[i] \leq D[i + 1], 0 \leq i < |D|$. Write a program which takes as input a positive integer k and computes the number of decimal numbers of length k that are monotone.

Variant: Call a decimal number D , as defined above, *strictly monotone* if $D[i] < D[i + 1], 0 \leq i < |D|$. Write a program which takes as input a positive integer k and computes the number of decimal numbers of length k that are strictly monotone.

Greedy Algorithms and Invariants

The intended function of a program, or part of a program, can be specified by making general assertions about the values which the relevant variables will take after execution of the program.

— “An Axiomatic Basis for Computer Programming,”

C. A. R. HOARE, 1969

Sometimes, there are multiple greedy algorithms for a given problem, and only some of them are optimum. For example, consider $2n$ cities on a line, half of which are white, and the other half are black. Suppose we need to pair white with black cities in a one-to-one fashion so that the total length of the road sections required to connect paired cities is minimized. Multiple pairs of cities may share a single section of road, e.g., if we have the pairing $(0, 4)$ and $(1, 2)$ then the section of road between Cities 0 and 4 can be used by Cities 1 and 2.

The most straightforward greedy algorithm for this problem is to scan through the white cities, and, for each white city, pair it with the closest unpaired black city. This algorithm leads to suboptimum results. Consider the case where white cities are at 0 and at 3 and black cities are at 2 and at 5. If the white city at 3 is processed first, it will be paired with the black city at 2, forcing the cities at 0 and 5 to pair up, leading to a road length of 5. In contrast, the pairing of cities at 0 and 2, and 3 and 5 leads to a road length of 4.

A slightly more sophisticated greedy algorithm does lead to optimum results: iterate through all the cities in left-to-right order, pairing each city with the nearest unpaired city of opposite color. Note that the pairing for the first city must be optimum, since if it were to be paired with any other city, we could always change its pairing to be with the nearest black city without adding any road. This observation can be used in an inductive proof of overall optimality.

Greedy algorithms boot camp

For US currency, wherein coins take values 1, 5, 10, 25, 50, 100 cents, the greedy algorithm for making change results in the minimum number of coins. Here is an implementation of this algorithm. Note that once it selects the number of coins of a particular value, it never changes that selection; this is the hallmark of a greedy algorithm.

```
def change_making(cents):
    COINS = [100, 50, 25, 10, 5, 1]
    num_coins = 0
    for coin in COINS:
        num_coins += cents / coin
        cents %= coin
    return num_coins
```

We perform 6 iterations, and each iteration does a constant amount of computation, so the time complexity is $O(1)$.

A greedy algorithm is often the right choice for an **optimization problem** where there's a natural set of **choices to select from**.

It's often easier to conceptualize a greedy algorithm recursively, and then **implement** it using iteration for higher performance.

Even if the greedy approach does not yield an optimum solution, it can give insights into the optimum algorithm, or serve as a heuristic.

Sometimes the correct greedy algorithm is **not obvious**.

Table 14.1: Top Tips for Greedy Algorithms

Invariants

A common approach to designing an efficient algorithm is to use invariants. Briefly, an invariant is a condition that is true during execution of a program. This condition may be on the values of the variables of the program, or on the control logic. A well-chosen invariant can be used to rule out potential solutions that are suboptimal or dominated by other solutions.

For example, binary search, maintains the invariant that the space of candidate solutions contains all possible solutions as the algorithm executes.

Sorting algorithms nicely illustrate algorithm design using invariants. Intuitively, selection sort is based on finding the smallest element, the next smallest element, etc. and moving them to their right place. More precisely, we work with successively larger subarrays beginning at index 0, and preserve the invariant that these subarrays are sorted, their elements are less than or equal to the remaining elements, and the entire array remains a permutation of the original array.

Invariants boot camp

Suppose you were asked to write a program that takes as input a sorted array and a given target value and determines if there are two entries in the array that add up to that value. For example, if the array is $\langle -2, 1, 2, 4, 7, 11 \rangle$, then there are entries adding to 6 and to 10, but not to 0 and 13.

The brute-force algorithm for this problem consists of a pair of nested for loops. Its complexity is $O(n^2)$, where n is the length of the input array. A faster approach is to add each element of the array to a hash table, and test for each element e if $K - e$, where K is the target value, is present in the hash table. While reducing time complexity to $O(n)$, this approach requires $O(n)$ additional storage for the hash.

The most efficient approach uses invariants: maintain a subarray that is guaranteed to hold a solution, if it exists. This subarray is initialized to the entire array, and iteratively shrunk from one side or the other. The shrinking makes use of the sortedness of the array. Specifically, if the sum of the leftmost and the rightmost elements is less than the target, then the leftmost element can never be combined with some element to obtain the target. A similar observation holds for the rightmost element.

```
def has_two_sum(A, t):
    i, j = 0, len(A) - 1

    while i <= j:
        if A[i] + A[j] == t:
            return True
        elif A[i] + A[j] < t:
            i += 1
```

```

    else: #  $A[i] + A[j] > t$ .
        j -= 1
    return False

```

The time complexity is $O(n)$, where n is the length of the array. The space complexity is $O(1)$, since the subarray can be represented by two variables.

Identifying the right invariant is an art. The key strategy to determine whether to use an invariant when designing an algorithm is to work on **small examples** to hypothesize the invariant.

Often, the invariant is a subset of the set of input space, e.g., a subarray.

Table 14.2: Top Tips for Invariants

14.1 THE 3-SUM PROBLEM

Design an algorithm that takes as input an array and a number, and determines if there are three entries in the array (not necessarily distinct) which add up to the specified number. For example, if the array is $\langle 11, 2, 5, 7, 3 \rangle$ then there are three entries in the array which add up to 21 (3, 7, 11 and 5, 5, 11). (Note that we can use 5 twice, since the problem statement said we can use the same entry more than once.) However, no three entries add up to 22.

Hint: How would you check if a given array entry can be added to two more entries to get the specified number?

Solution: First, we consider the problem of computing a pair of entries which sum to K . Assume A is sorted. We start with the pair consisting of the first element and the last element: $(A[0], A[n-1])$. Let $s = A[0] + A[n-1]$. If $s = K$, we are done. If $s < K$, we increase the sum by moving to pair $(A[1], A[n-1])$. We need never consider $A[0]$; since the array is sorted, for all i , $A[0] + A[i] \leq A[0] + A[n-1] = K < s$. If $s > K$, we can decrease the sum by considering the pair $(A[0], A[n-2])$; by analogous reasoning, we need never consider $A[n-1]$ again. We iteratively continue this process till we have found a pair that sums up to K or the indices meet, in which case the search ends. This solution works in $O(n)$ time and $O(1)$ space in addition to the space needed to store A .

Now we describe a solution to the problem of finding three entries which sum to t . We sort A and for each $A[i]$, search for indices j and k such that $A[j] + A[k] = t - A[i]$. The additional space needed is $O(1)$, and the time complexity is the sum of the time taken to sort, $O(n \log n)$, and then to run the $O(n)$ algorithm described in the previous paragraph n times (one for each entry), which is $O(n^2)$ overall. The code for this approach is shown below.

```

def has_three_sum(A, t):
    A.sort()
    # Finds if the sum of two numbers in A equals to t - a.
    return any(has_two_sum(A, t - a) for a in A)

```

The additional space needed is $O(1)$, and the time complexity is the sum of the time taken to sort, $O(n \log n)$, and then to run the $O(n)$ algorithm to find a pair in a sorted array that sums to a specified value, which is $O(n^2)$ overall.

Variant: Solve the same problem when the three elements must be distinct. For example, if $A = \langle 5, 2, 3, 4, 3 \rangle$ and $t = 9$, then $A[2] + A[2] + A[2]$ is not acceptable, $A[2] + A[2] + A[4]$ is not acceptable, but $A[1] + A[2] + A[3]$ and $A[1] + A[3] + A[4]$ are acceptable.

Variant: Solve the same problem when k , the number of elements to sum, is an additional input.

Variant: Write a program that takes as input an array of integers A and an integer T , and returns a 3-tuple $(A[p], A[q], A[r])$ where p, q, r are all distinct, minimizing $|T - (A[p] + A[q] + A[r])|$, and $A[p] \leq A[r] \leq A[s]$.

Variant: Write a program that takes as input an array of integers A and an integer T , and returns the number of 3-tuples (p, q, r) such that $A[p] + A[q] + A[r] \leq T$ and $A[p] \leq A[q] \leq A[r]$.

Graphs

Concerning these bridges, it was asked whether anyone could arrange a route in such a way that he would cross each bridge once and only once.

— “The solution of a problem relating to the geometry of position,”

L. EULER, 1741

Informally, a graph is a set of vertices and connected by edges. Formally, a directed graph is a set V of *vertices* and a set $E \subset V \times V$ of edges. Given an edge $e = (u, v)$, the vertex u is its *source*, and v is its *sink*. Graphs are often decorated, e.g., by adding lengths to edges, weights to vertices, a start vertex, etc. A directed graph can be depicted pictorially as in Figure 15.1.

A *path* in a directed graph from u to vertex v is a sequence of vertices $\langle v_0, v_1, \dots, v_{n-1} \rangle$ where $v_0 = u$, $v_{n-1} = v$, and each (v_i, v_{i+1}) is an edge. The sequence may consist of a single vertex. The *length* of the path $\langle v_0, v_1, \dots, v_{n-1} \rangle$ is $n - 1$. Intuitively, the length of a path is the number of edges it traverses. If there exists a path from u to v , v is said to be *reachable* from u . For example, the sequence $\langle a, c, e, d, h \rangle$ is a path in the graph represented in Figure 15.1.

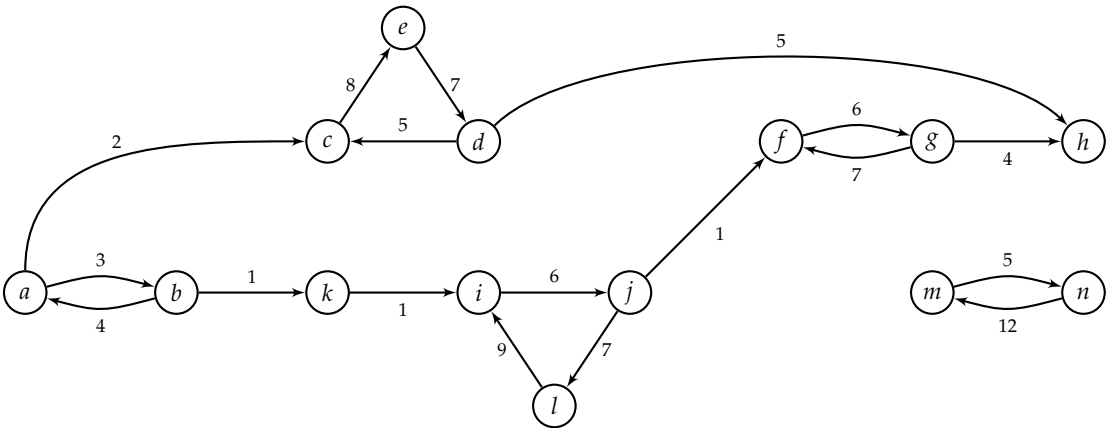


Figure 15.1: A directed graph with weights on edges.

A *directed acyclic graph* (DAG) is a directed graph in which there are no *cycles*, i.e., paths which contain one or more edges and which begin and end at the same vertex. See Figure 15.2 on the following page for an example of a directed acyclic graph. Vertices in a DAG which have no incoming edges are referred to as *sources*; vertices which have no outgoing edges are referred to as *sinks*. A *topological ordering* of the vertices in a DAG is an ordering of the vertices in which each edge is from a vertex earlier in the ordering to a vertex later in the ordering.

An undirected graph is also a tuple (V, E) ; however, E is a set of unordered pairs of vertices. Graphically, this is captured by drawing arrowless connections between vertices, as in Figure 15.3.

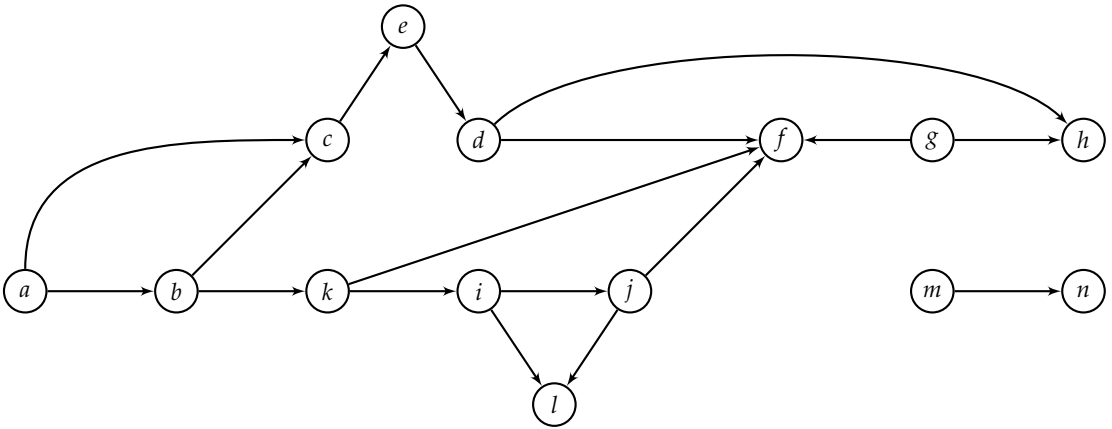


Figure 15.2: A directed acyclic graph. Vertices a, g, m are sources and vertices l, f, h, n are sinks. The ordering $\langle a, b, c, e, d, g, h, k, i, j, f, l, m, n \rangle$ is a topological ordering of the vertices.

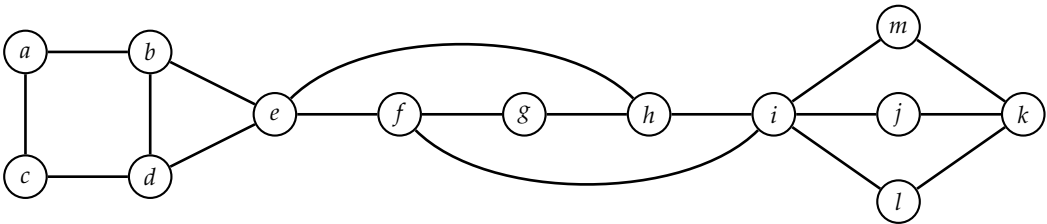


Figure 15.3: An undirected graph.

If G is an undirected graph, vertices u and v are said to be *connected* if G contains a path from u to v ; otherwise, u and v are said to be *disconnected*. A graph is said to be *connected* if every pair of vertices in the graph is connected. A *connected component* is a maximal set of vertices C such that each pair of vertices in C is connected in G . Every vertex belongs to exactly one connected component.

For example, the graph in Figure 15.3 is connected, and it has a single connected component. If edge (h, i) is removed, it remains connected. If additionally (f, i) is removed, it becomes disconnected and there are two connected components.

A directed graph is called *weakly connected* if replacing all of its directed edges with undirected edges produces an undirected graph that is connected. It is *connected* if it contains a directed path from u to v or a directed path from v to u for every pair of vertices u and v . It is *strongly connected* if it contains a directed path from u to v and a directed path from v to u for every pair of vertices u and v .

Graphs naturally arise when modeling geometric problems, such as determining connected cities. However, they are more general, and can be used to model many kinds of relationships.

A graph can be implemented in two ways—using *adjacency lists* or an *adjacency matrix*. In the adjacency list representation, each vertex v , has a list of vertices to which it has an edge. The adjacency matrix representation uses a $|V| \times |V|$ Boolean-valued matrix indexed by vertices, with a 1 indicating the presence of an edge. The time and space complexities of a graph algorithm are usually expressed as a function of the number of vertices and edges.

A *tree* (sometimes called a *free tree*) is a special sort of graph—it is an undirected graph that is

connected but has no cycles. (Many equivalent definitions exist, e.g., a graph is a free tree if and only if there exists a unique path between every pair of vertices.) There are a number of variants on the basic idea of a tree. A rooted tree is one where a designated vertex is called the root, which leads to a parent-child relationship on the nodes. An ordered tree is a rooted tree in which each vertex has an ordering on its children. Binary trees, which are the subject of Chapter 6, differ from ordered trees since a node may have only one child in a binary tree, but that node may be a left or a right child, whereas in an ordered tree no analogous notion exists for a node with a single child. Specifically, in a binary tree, there is position as well as order associated with the children of nodes.

As an example, the graph in Figure 15.4 is a tree. Note that its edge set is a subset of the edge set of the undirected graph in Figure 15.3 on the facing page. Given a graph $G = (V, E)$, if the graph $G' = (V, E')$ where $E' \subset E$, is a tree, then G' is referred to as a spanning tree of G .

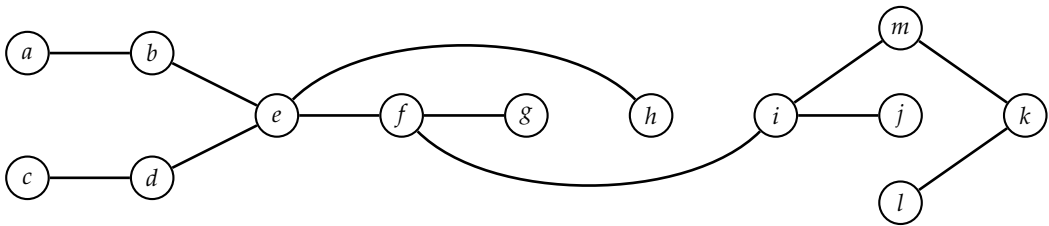


Figure 15.4: A tree.

Graphs boot camp

Graphs are ideal for modeling and analyzing relationships between pairs of objects. For example, suppose you were given a list of the outcomes of matches between pairs of teams, with each outcome being a win or loss. A natural question is as follows: given teams A and B , is there a sequence of teams starting with A and ending with B such that each team in the sequence has beaten the next team in the sequence?

A slick way of solving this problem is to model the problem using a graph. Teams are vertices, and an edge from one team to another indicates that the team corresponding to the source vertex has beaten the team corresponding to the destination vertex. Now we can apply graph reachability to perform the check. Both DFS and BFS are reasonable approaches—the program below uses DFS.

```

MatchResult = collections.namedtuple('MatchResult',
                                      ('winning_team', 'losing_team'))

def can_team_a_beat_team_b(matches, team_a, team_b):
    def build_graph():
        graph = collections.defaultdict(set)
        for match in matches:
            graph[match.winning_team].add(match.losing_team)
        return graph

    def is_reachable_dfs(graph, curr, dest, visited=set()):
        if curr == dest:
            return True
        elif curr in visited or curr not in graph:
            return False
        visited.add(curr)

```

```

    return any(is_reachable_dfs(graph, team, dest) for team in graph[curr])

return is_reachable_dfs(build_graph(), team_a, team_b)

```

The time complexity and space complexity are both $O(E)$, where E is the number of outcomes.

It's natural to use a graph when the problem involves **spatially connected** objects, e.g., road segments between cities.

More generally, consider using a graph when you need to analyze **any binary relationship**, between objects, such as interlinked webpages, followers in a social graph, etc. In such cases, quite often the problem can be reduced to a well-known graph problem.

Some graph problems entail **analyzing structure**, e.g., looking for cycles or connected components. **DFS** works particularly well for these applications.

Some graph problems are related to **optimization**, e.g., find the shortest path from one vertex to another. **BFS, Dijkstra's shortest path algorithm, and minimum spanning tree** are examples of graph algorithms appropriate for optimization problems.

Table 15.1: Top Tips for Graphs

Graph search

Computing vertices which are reachable from other vertices is a fundamental operation which can be performed in one of two idiomatic ways, namely depth-first search (DFS) and breadth-first search (BFS). Both have linear time complexity— $O(|V| + |E|)$ to be precise. In the worst-case there is a path from the initial vertex covering all vertices without any repeats, and the DFS edges selected correspond to this path, so the space complexity of DFS is $O(|V|)$ (this space is implicitly allocated on the function call stack). The space complexity of BFS is also $O(|V|)$, since in a worst-case there is an edge from the initial vertex to all remaining vertices, implying that they will all be in the BFS queue simultaneously at some point.

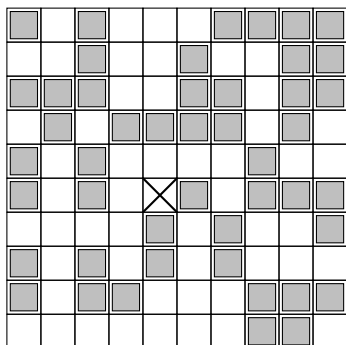
DFS and BFS differ from each other in terms of the additional information they provide, e.g., BFS can be used to compute distances from the start vertex and DFS can be used to check for the presence of cycles. Key notions in DFS include the concept of *discovery time* and *finishing time* for vertices.

15.1 PAINT A BOOLEAN MATRIX

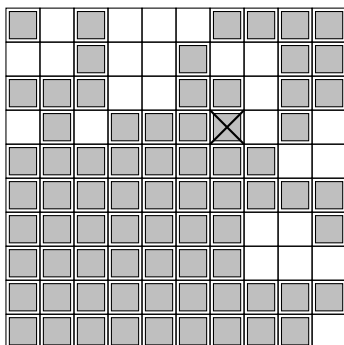
Let A be a Boolean 2D array encoding a black-and-white image. The entry $A(a, b)$ can be viewed as encoding the color at entry (a, b) . Call two entries adjacent if one is to the left, right, above or below the other. Note that the definition implies that an entry can be adjacent to at most four other entries, and that adjacency is symmetric, i.e., if $e0$ is adjacent to entry $e1$, then $e1$ is adjacent to $e0$.

Define a path from entry $e0$ to entry $e1$ to be a sequence of adjacent entries, starting at $e0$, ending at $e1$, with successive entries being adjacent. Define the region associated with a point (i, j) to be all points (i', j') such that there exists a path from (i, j) to (i', j') in which all entries are the same color. In particular this implies (i, j) and (i', j') must be the same color.

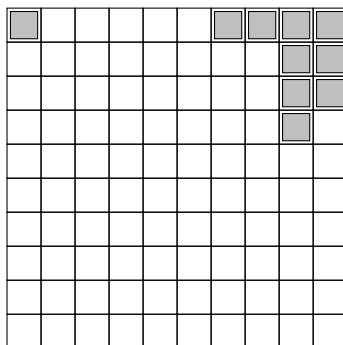
Implement a routine that takes an $n \times m$ Boolean array A together with an entry (x, y) and flips the color of the region associated with (x, y) . See Figure 15.5 for an example of flipping.



(a)



(b)



(c)

Figure 15.5: The color of all squares associated with the first square marked with a $\times$ in (a) have been recolored to yield the coloring in (b). The same process yields the coloring in (c).

Hint: Solve this conceptually, then think about implementation optimizations.

Solution:

Parallel Computing

The activity of a computer must include the proper reacting to a possibly great variety of messages that can be sent to it at unpredictable moments, a situation which occurs in all information systems in which a number of computers are coupled to each other.

—“Cooperating sequential processes,”
E. W. DIJKSTRA, 1965

Parallel computation has become increasingly common. For example, laptops and desktops come with multiple processors which communicate through shared memory. High-end computation is often done using clusters consisting of individual computers communicating through a network.

Parallelism provides a number of benefits:

- High performance—more processors working on a task (usually) means it is completed faster.
- Better use of resources—a program can execute while another waits on the disk or network.
- Fairness—letting different users or programs share a machine rather than have one program run at a time to completion.
- Convenience—it is often conceptually more straightforward to do a task using a set of concurrent programs for the subtasks rather than have a single program manage all the subtasks.
- Fault tolerance—if a machine fails in a cluster that is serving web pages, the others can take over.

Concrete applications of parallel computing include graphical user interfaces (GUI) (a dedicated thread handles UI actions while other threads are, for example, busy doing network communication and passing results to the UI thread, resulting in increased responsiveness), Java virtual machines (a separate thread handles garbage collection which would otherwise lead to blocking, while another thread is busy running the user code), web servers (a single logical thread handles a single client request), scientific computing (a large matrix multiplication can be split across a cluster), and web search (multiple machines crawl, index, and retrieve web pages).

The two primary models for parallel computation are the shared memory model, in which each processor can access any location in memory, and the distributed memory model, in which a processor must explicitly send a message to another processor to access its memory. The former is more appropriate in the multicore setting and the latter is more accurate for a cluster. The questions in this chapter are focused on the shared memory model.

Writing correct parallel programs is challenging because of the subtle interactions between parallel components. One of the key challenges is races—two concurrent instruction sequences access the same address in memory and at least one of them writes to that address. Other challenges to correctness are

- starvation (a processor needs a resource but never gets it),
- deadlock (Thread *A* acquires Lock *L1* and Thread *B* acquires Lock *L2*, following which *A* tries to acquire *L2* and *B* tries to acquire *L1*), and

- livelock (a processor keeps retrying an operation that always fails).

Bugs caused by these issues are difficult to find using testing. Debugging them is also difficult because they may not be reproducible since they are usually load dependent. It is also often true that it is not possible to realize the performance implied by parallelism—sometimes a critical task cannot be parallelized, making it impossible to improve performance, regardless of the number of processors added. Similarly, the overhead of communicating intermediate results between processors can exceed the performance benefits.

Parallel computing boot camp

A semaphore is a very powerful synchronization construct. Conceptually, a semaphore maintains a set of permits. A thread calling *acquire()* on a semaphore waits, if necessary, until a permit is available, and then takes it. A thread calling *release()* on a semaphore adds a permit and notifies threads waiting on that semaphore, potentially releasing a blocking acquirer.

```
import threading

class Semaphore():

    def __init__(self, max_available):
        self.cv = threading.Condition()
        self.MAX_AVAILABLE = max_available
        self.taken = 0

    def acquire(self):
        self.cv.acquire()
        while (self.taken == self.MAX_AVAILABLE):
            self.cv.wait()
        self.taken += 1
        self.cv.release()

    def release(self):
        self.cv.acquire()
        self.taken -= 1
        self.cv.notify()
        self.cv.release()
```

Start with an algorithm that **locks aggressively** and is easily seen to be correct. Then **add back concurrency**, while ensuring the **critical** parts are locked.

When analyzing parallel code, assume a worst-case thread scheduler. In particular, it may choose to schedule the same thread repeatedly, it may alternate between two threads, it may starve a thread, etc.

Try to work at a **higher level of abstraction**. In particular, know the **concurrency libraries**—don't implement your own **semaphores**, **thread pools**, **deferred execution**, etc. (You should know how these features are implemented, and implement them if asked to.)

Table 16.1: Top Tips for Concurrency

16.1 IMPLEMENT A TIMER CLASS

Consider a web-based calendar in which the server hosting the calendar has to perform a task when the next calendar event takes place. (The task could be sending an email or a Short Message Service (SMS).) Your job is to design a facility that manages the execution of such tasks.

Develop a timer class that manages the execution of deferred tasks. The timer constructor takes as its argument an object which includes a run method and a string-valued name field. The class must support—(1.) starting a thread, identified by name, at a given time in the future; and (2.) canceling a thread, identified by name (the cancel request is to be ignored if the thread has already started).

Hint: There are two aspects—data structure design and concurrency.

Solution: The two aspects to the design are the data structures and the locking mechanism.

We use two data structures. The first is a min-heap in which we insert key-value pairs: the keys are run times and the values are the thread to run at that time. A dispatch thread runs these threads; it sleeps from call to call and may be woken up if a thread is added to or deleted from the pool. If woken up, it advances or retards its remaining sleep time based on the top of the min-heap. On waking up, it looks for the thread at the top of the min-heap—if its launch time is the current time, the dispatch thread deletes it from the min-heap and executes it. It then sleeps till the launch time for the next thread in the min-heap. (Because of deletions, it may happen that the dispatch thread wakes up and finds nothing to do.)

The second data structure is a hash table with thread ids as keys and entries in the min-heap as values. If we need to cancel a thread, we go to the min-heap and delete it. Each time a thread is added, we add it to the min-heap; if the insertion is to the top of the min-heap, we interrupt the dispatch thread so that it can adjust its wake up time.

Since the min-heap is shared by the update methods and the dispatch thread, we need to lock it. The simplest solution is to have a single lock that is used for all read and writes into the min-heap and the hash table.

Part II

Domain Specific Problems

Design Problems

Don't be fooled by the many books on complexity or by the many complex and arcane algorithms you find in this book or elsewhere. Although there are no textbooks on simplicity, simple systems work and complex don't.

— “Transaction Processing: Concepts and Techniques,”
J. GRAY, 1992

You may be asked in an interview how to go about creating a set of services or a larger system, possibly on top of an algorithm that you have designed. These problems are fairly open-ended, and many can be the starting point for a large software project.

In an interview setting when someone asks such a question, you should have a conversation in which you demonstrate an ability to think creatively, understand design trade-offs, and attack unfamiliar problems. You should sketch key data structures and algorithms, as well as the technology stack (programming language, libraries, OS, hardware, and services) that you would use to solve the problem.

The answers in this chapter are presented in this context—they are meant to be examples of good responses in an interview and are not comprehensive state-of-the-art solutions.

We review patterns that are useful for designing systems in Table 17.1. Some other things to keep in mind when designing a system are implementation time, extensibility, scalability, testability, security, internationalization, and IP issues.

Table 17.1: System design patterns.

Design principle	Key points
Algorithms and Data Structures	Identify the basic algorithms and data structures
Decomposition	Split the functionality, architecture, and code into manageable, reusable components.
Scalability	Break the problem into subproblems that can be solved relatively independently on different machines. Shard data across machines to make it fit. Decouple the servers that handle writes from those that handle reads. Use replication across the read servers to gain more performance. Consider caching computation and later look it up to save work.

Decomposition

Good decompositions are critical to successfully solving system-level design problems. Functionality, architecture, and code all benefit from decomposition.

For example, in our solution to designing a system for online advertising, we decompose the goals into categories based on the stake holders. We decompose the architecture itself into a front-end and a back-end. The front-end is divided into user management, web page design, reporting functionality, etc. The back-end is made up of middleware, storage, database, cron services, and algorithms for ranking ads.

Decomposing code is a hallmark of object-oriented programming. The subject of design patterns is concerned with finding good ways to achieve code-reuse. Broadly speaking, design patterns are grouped into creational, structural, and behavioral patterns. Many specific patterns are very natural—strategy objects, adapters, builders, etc., appear in a number of places in our codebase. Freeman *et al.*'s "*Head First Design Patterns*" is, in our opinion, the right place to study design patterns.

Scalability

In the context of interview questions parallelism is useful when dealing with scale, i.e., when the problem is too large to fit on a single machine or would take an unacceptably long time on a single machine. The key insight you need to display is that you know how to decompose the problem so that

- each subproblem can be solved relatively independently, and
- the solution to the original problem can be efficiently constructed from solutions to the sub-problems.

Efficiency is typically measured in terms of central processing unit (CPU) time, random access memory (RAM), network bandwidth, number of memory and database accesses, etc.

Consider the problem of sorting a petascale integer array. If we know the distribution of the numbers, the best approach would be to define equal-sized ranges of integers and send one range to one machine for sorting. The sorted numbers would just need to be concatenated in the correct order. If the distribution is not known then we can send equal-sized arbitrary subsets to each machine and then merge the sorted results, e.g., using a min-heap.

Caching is a great tool whenever computations are repeated. For example, the central idea behind dynamic programming is caching results from intermediate computations. Caching is also extremely useful when implementing a service that is expected to respond to many requests over time, and many requests are repeated. Workloads on web services exhibit this property.

17.1 DESIGN A SYSTEM FOR DETECTING COPYRIGHT INFRINGEMENT

YouTV.com is a successful online video sharing site. Hollywood studios complain that much of the material uploaded to the site violates copyright.

Design a feature that allows a studio to enter a set V of videos that belong to it, and to determine which videos in the YouTV.com database match videos in V .

Hint: Normalize the video format and create signatures.

Solution: Videos differ from documents in that the same content may be encoded in many different formats, with different resolutions, and levels of compression.

One way to reduce the duplicate video problem to the duplicate document problem is to re-encode all videos to a common format, resolution, and compression level. This in itself does not

mean that two videos of the same content get reduced to identical files—the initial settings affect the resulting videos. However, we can now “signature” the normalized video.

A trivial signature would be to assign a 0 or a 1 to each frame based on whether it has more or less brightness than average. A more sophisticated signature would be a 3 bit measure of the red, green, and blue intensities for each frame. Even more sophisticated signatures can be developed, e.g., by taking into account the regions on individual frames. The motivation for better signatures is to reduce the number of false matches returned by the system, and thereby reduce the amount of time needed to review the matches.

The solution proposed above is algorithmic. However, there are alternative approaches that could be effective: letting users flag videos that infringe copyright (and possibly rewarding them for their effort), checking for videos that are identical to videos that have previously been identified as infringing, looking at meta-information in the video header, etc.

Variant: Design an online music identification service.

Language Questions

The limits of my language means the limits of my world.

— L. WITTGENSTEIN

18.1 CLOSURE

What does the following program print, and why?

```
increment_by_i = [lambda x: x + i for i in range(10)]  
  
print(increment_by_i[3](4))
```

Solution: The program prints 13 ($=9 + 4$) rather than the 7 ($=3 + 4$) than might be expected. This is because the the functions created in the loop have the same scope. They use the same variable name, and consequently, all refer to the same variable, i , which is 10 at the end of the loop, hence the 13 ($=9 + 4$).

There are many ways to get the desired behavior. A reasonable approach is to return the lambda from a function, thereby avoiding the naming conflict.

```
def create_increment_function(x):  
    return lambda y: y + x  
  
increment_by_i = [create_increment_function(i) for i in range(10)]  
  
print(increment_by_i[3](4))
```

18.2 SHALLOW AND DEEP COPY

Describe the differences between a shallow copy and a deep copy. When is each appropriate, and when is neither appropriate? How is deep copy implemented?

Solution: First, we note that assignment in Python does not copy—it simply a variable to a target. For objects that are mutable, or contain mutable items, a copy is needed if we want to change the copy without changing the original. For example, if $A = [1, 2, 4, 8]$ and we assign A to B , i.e., $B = A$, then if we change B , e.g., $B.append(16)$, then A will also change.

There are two ways to make a copy: shallow and deep. These are implemented in the copy module, specifically as `copy.copy(x)` and `copy.deepcopy(x)`. A shallow copy constructs a new compound object and then (to the extent possible) inserts references into it to the objects found in

the original. A deep copy constructs a new compound object and then, recursively, inserts copies into it of the objects found in the original.

As a concrete example, suppose `A = [[1, 2, 3], [4, 5, 6]]` and `B = copy.copy(A)`. Then if we update `B` as `B[0][0] = 0`, `A[0][0]` also changes to `0`. However, if `B = copy.deepcopy(A)`, then the same update will leave `A` unchanged.

The difference between shallow and deep copying is only relevant for compound objects, that is objects that contain other objects, such as lists or class instances. If `A = [1, 2, 3]` and `B = copy.copy(A)`, then if we update `B` as `B[0] = 0`, `A` is unchanged.

Diving under the hood, deep copy is more challenging to implement since recursive objects (compound objects that, directly or indirectly, contain a reference to themselves) result in infinite recursion in a naive implementation of deep copy. Furthermore, since a deep copy copies everything, it may copy too much, e.g., book-keeping data structures that should be shared even between copies. The `copy.deepcopy()` function avoids these problems by caching objects already copied, and letting user-defined classes override the copying operation. In particular, a class can override one or both of `__copy__()` and `__deepcopy__()`.

Copying is defensive—it may be avoided if it's known that the client will not mutate the object. Similarly, if the object is itself immutable, e.g., a tuple, then there's no need to copy it.

Object-Oriented Design

One thing expert designers know not to do is solve every problem from first principles.

— “*Design Patterns: Elements of Reusable Object-Oriented Software*,”
E. GAMMA, R. HELM, R. E. JOHNSON, AND J. M. VLISSIDES, 1994

A class is an encapsulation of data and methods that operate on that data. Classes match the way we think about computation. They provide encapsulation, which reduces the conceptual burden of writing code, and enable code reuse, through the use of inheritance and polymorphism. However, naive use of object-oriented constructs can result in code that is hard to maintain.

A design pattern is a general repeatable solution to a commonly occurring problem. It is not a complete design that can be coded up directly—rather, it is a description of how to solve a problem that arises in many different situations. In the context of object-oriented programming, design patterns address both reuse and maintainability. In essence, design patterns make some parts of a system vary independently from the other parts.

Adnan’s Design Pattern course material, available freely online, contains lecture notes, homeworks, and labs that may serve as a good resource on the material in this chapter.

19.1 CREATIONAL PATTERNS

Explain what each of these creational patterns is: builder, static factory, factory method, and abstract factory.

Solution: The idea behind the builder pattern is to build a complex object in phases. It avoids mutability and inconsistent state by using an mutable inner class that has a build method that returns the desired object. Its key benefits are that it breaks down the construction process, and can give names to steps. Compared to a constructor, it deals far better with optional parameters and when the parameter list is very long.

A static factory is a function for construction of objects. Its key benefits are as follow: the function’s name can make what it’s doing much clearer compared to a call to a constructor. The function is not obliged to create a new object—in particular, it can return a flyweight. It can also return a subtype that’s more optimized, e.g., it can choose to construct an object that uses an integer in place of a Boolean array if the array size is not more than the integer word size.

A factory method defines interface for creating an object, but lets subclasses decide which class to instantiate. The classic example is a maze game with two modes—one with regular rooms, and one with magic rooms.

```
from abc import ABC, abstractmethod
```

```

class Room(ABC):

    @abstractmethod
    def connect(self, room2):
        pass

class MazeGame(ABC):

    @abstractmethod
    def make_room(self):
        print("abstract make_room")
        pass

    def addRoom(self, room):
        print("adding room")

    def __init__(self):
        room1 = self.make_room()
        room2 = self.make_room()
        room1.connect(room2)
        self.addRoom(room1)
        self.addRoom(room2)

```

This snippet implements the regular rooms. This snippet implements the magic rooms.

```

class MagicMazeGame(MazeGame):

    def make_room(self):
        return MagicRoom()

class MagicRoom(Room):

    def connect(self, room2):
        print("Connecting magic room")

```

Here's how you use the factory to create regular and magic games.

```

ordinary_maze_game = ordinary_maze_game.OrdinaryMazeGame()
magic_maze_game = magic_maze_game.MagicMazeGame()

```

A drawback of the factory method pattern is that it makes subclassing challenging.

An abstract factory provides an interface for creating families of related objects without specifying their concrete classes. For example, a class `DocumentCreator` could provide interfaces to create a number of products, such as `createLetter()` and `createResume()`. Concrete implementations of this class could choose to implement these products in different ways, e.g., with modern or classic fonts, right-flush or right-ragged layout, etc. Client code gets a `DocumentCreator` object and calls its factory methods. Use of this pattern makes it possible to interchange concrete implementations without changing the code that uses them, even at runtime. The price for this flexibility is more planning and upfront coding, as well as and code that may be harder to understand, because of the added indirections.

Common Tools

UNIX is a general-purpose, multi-user, interactive operating system for the Digital Equipment Corporation PDP-11/40 and 11/45 computers. It offers a number of features seldom found even in larger operating systems, including: (1) a hierarchical file system incorporating demountable volumes; (2) compatible file, device, and inter-process I/O; (3) the ability to initiate asynchronous processes; (4) system command language selectable on a per-user basis; and (5) over 100 subsystems including a dozen languages.

— “The UNIX TimeSharing System,”
D. RITCHIE AND K. THOMPSON, 1974

The problems in this chapter are concerned with tools: version control systems, scripting languages, build systems, databases, and the networking stack. Such problems are not commonly asked—expect them only if you are interviewing for a specialized role, or if you claim specialized knowledge, e.g., network security or databases. We emphasize these are vast subjects, e.g., networking is taught as a sequence of courses in a university curriculum. Our goal here is to give you a flavor of what you might encounter. Adnan’s Advanced Programming Tools course material, available freely online, contains lecture notes, homeworks, and labs that may serve as a good resource on the material in this chapter.

Version control

Version control systems are a cornerstone of software development—every developer should understand how to use them in an optimal way.

20.1 MERGING IN A VERSION CONTROL SYSTEM

What is merging in a version control system? Specifically, describe the limitations of line-based merging and ways in which they can be overcome.

Solution:

Modern version control systems allow developers to work concurrently on a personal copy of the entire codebase. This allows software developers to work independently. The price for this merging: periodically, the personal copies need to be integrated to create a new shared version. There is the possibility that parallel changes are conflicting, and these must be resolved during the merge.

By far, the most common approach to merging is text-based. Text-based merge tools view software as text. Line-based merging is a kind of text-based merging, specifically one in which each line is treated as an indivisible unit. The merging is “three-way”: the lowest common ancestor in the revision history is used in conjunction with the two versions. With line-based merging of text files, common text lines can be detected in parallel modifications, as well as text lines that have been

inserted, deleted, modified, or moved. Figure 20.1 on the next page shows an example of such a line-based merge.

One issue with line-based merging is that it cannot handle two parallel modifications to the same line: the two modifications cannot be combined, only one of the two modifications must be selected. A bigger issue is that a line-based merge may succeed, but the resulting program is broken because of syntactic or semantic conflicts. In the scenario in Figure 20.1 on the facing page the changes made by the two developers are made to independent lines, so the line-based merge shows no conflicts. However, the program will not compile because a function is called incorrectly.

In spite of this, line-based merging is widely used because of its efficiency, scalability, and accuracy. A three-way, line-based merge tool in practice will merge 90% of the changed files without any issues. The challenge is to automate the remaining 10% of the situations that cannot be merged automatically.

Text-based merging fails because it does not consider any syntactic or semantic information.

Syntactic merging takes the syntax of the programming language into account. Text-based merge often yields unimportant conflicts such as a code comment that has been changed by different developers or conflicts caused by code reformatting. A syntactic merge can ignore all these: it displays a merge conflict when the merged result is not syntactically correct.

We illustrate syntactic merging with a small example:

```
if (n%2 == 0) then m = n/2;
```

Two developers change this code in a different ways, but with the same overall effect. The first updates it to

```
if (n%2 == 0) then m = n/2 else m = (n-1)/2;
```

and the second developer's update is

```
m = n/2;
```

Both changes to the same thing. However, a textual-merge will likely result in

```
m := n div 2 else m := (n-1)/2
```

which is syntactically incorrect. Syntactic merge identifies the conflict; it is the integrator's responsibility to manually resolve it, e.g., by removing the else portion.

Syntactic merge tools are unable to detect some frequently occurring conflicts. For example, in the merge of Versions 1a and 1b in Figure 20.1 on the next page will not compile, since the call to `sum(10)` has the wrong signature. Syntactic merge will not detect this conflict since the program is still syntactically correct. The conflict is a semantic conflict, specifically, a static semantic conflict, since it is detectable at compile-time. (The compile will return something like "function argument mismatch error".)

Technically, syntactic merge algorithms operate on the parse trees of the programs to be merged. More powerful static semantic merge algorithms have been proposed that operate on a graph representation of the programs, wherein definitions and usage are linked, making it easier to identify mismatched usage.

Static semantic merging also has shortcomings. For example, suppose `Point` is a class representing 2D-points using Cartesian coordinates, and that this class has a distance function that returns $\sqrt{x^2 + y^2}$. Suppose Alice checks out the project, and subclasses `Point` to create a class that supports polar coordinates, and uses `Point`'s distance function to return the radius. Concurrently, Bob checks out the project and changes `Point`'s distance function to return $|x| + |y|$. Static semantic

merging reports no conflicts: the merged program compiles without any trouble. However the behavior is not what was expected.

Both syntactic merging and semantic merging greatly increase runtimes, and are very limiting since they are tightly coupled to specific programming languages. In practice, line-based merging is used in conjunction with a small set of unit tests (a “smoke suite”) invoked with a pre-commit hook script. Compilation finds syntax and static semantics errors, and the tests (hopefully) identify deeper semantic issues.

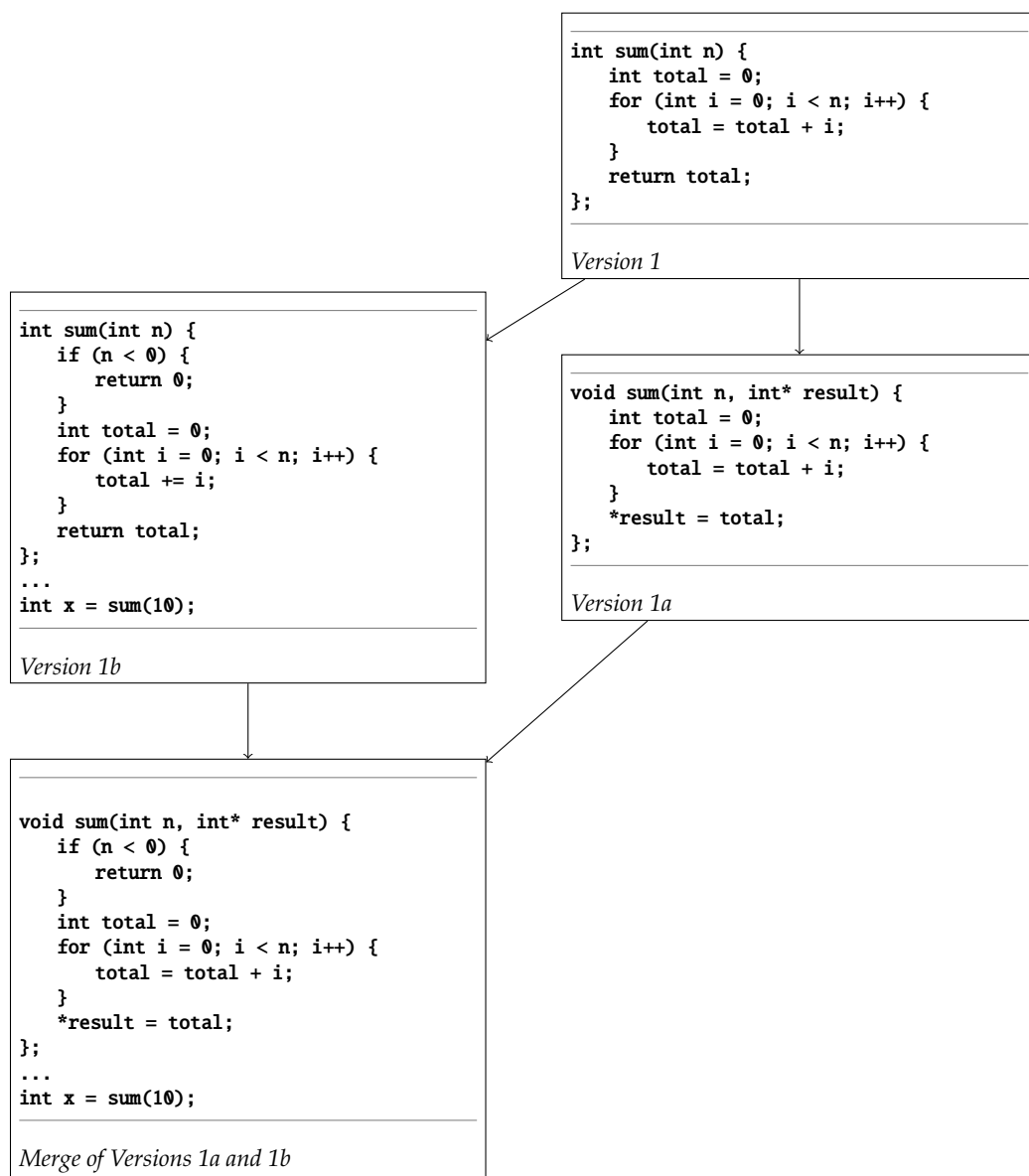


Figure 20.1: An example of a 3-way line based merge.

Scripting languages

Scripting languages, such as AWK, Perl, and Python were originally developed as tools for quick hacks, rapid prototyping, and gluing together other programs. They have evolved into mainstream

programming languages. Some of their characteristics are as follows.

- Text strings are the basic (sometimes only) data type.
- Associative arrays are a basic aggregate type.
- Regexp are usually built in.
- Programs are interpreted rather than compiled.
- Declarations are often not required.

They are easy to get started with, and can often do a great deal with very little code.

Build systems

A program typically cannot be executed immediately after change: intermediate steps are required to build the program and test it before deploying it. Other components can also be affected by changes, such as documentation generated from program text. Build systems automate these steps.

Database

Most software systems today interact with databases, and it's important for developers to have basic knowledge of databases.

Networking

Most software systems today interact with the network, and it's important for developers even higher up the stack to have basic knowledge of networking. Here are networking questions you may encounter in an interview setting.

Part III

The Honors Class

Honors Class

The supply of problems in mathematics is inexhaustible, and as soon as one problem is solved numerous others come forth in its place.

—“Mathematical Problems,”
D. HILBERT, 1900

This chapter contains problems that are more difficult to solve than the ones presented earlier. Many of them are commonly asked at interviews, albeit with the expectation that the candidate will not deliver the best solution.

There are several reasons why we included these problems:

- Although mastering these problems is not essential to your success, if you do solve them, you will have put yourself into a very strong position, similar to training for a race with ankle weights.
- If you are asked one of these questions or a variant thereof, and you successfully derive the best solution, you will have strongly impressed your interviewer. The implications of this go beyond being made an offer—your salary, position, and status will all go up.
- Some of the problems are appropriate for candidates interviewing for specialized positions, e.g., optimizing large scale distributed systems, machine learning for computational finance, etc. They are also commonly asked of candidates with advanced degrees.
- Finally, if you love a good challenge, these problems will provide it!

You should be happy if your interviewer asks you a hard question—it implies high expectations, and is an opportunity to shine, compared, for example, to being asked to write a program that tests if a string is palindromic.

INTRACTABILITY

Coding problems asked in interviews almost always have one (or more) solutions that are fast, e.g., $O(n)$ time complexity. Sometimes, you may be asked a problem that is computationally intractable—there may not exist an efficient algorithm for it. (Such problems are common in the real world.)

There are a number of approaches to solving intractable problems:

- brute-force solutions, including dynamic programming, which have exponential time complexity, may be acceptable, if the instances encountered are small, or if the specific parameter that the complexity is exponential in is small;
- search algorithms, such as backtracking, branch-and-bound, and hill-climbing, which prune much of the complexity of a brute-force search;

- approximation algorithms which return a solution that is provably close to optimum;
- heuristics based on insight, common case analysis, and careful tuning that may solve the problem reasonably well;
- parallel algorithms, wherein a large number of computers can work on subparts simultaneously.

21.1 COMPUTE THE GREATEST COMMON DIVISOR

The greatest common divisor (GCD) of nonnegative integers x and y is the largest integer d such that d divides x evenly, and d divides y evenly, i.e., $x \bmod d = 0$ and $y \bmod d = 0$. (When both x and y are 0, take their GCD to be 0.)

Design an efficient algorithm for computing the GCD of two nonnegative integers without using multiplication, division or the modulus operators.

Hint: Use case analysis: both even; both odd; one even and one odd.

Solution: The straightforward algorithm is based on recursion. If $x = y$, $\text{GCD}(x, y) = x$; otherwise, assume without loss of generality, that $x > y$. Then $\text{GCD}(x, y)$ is the $\text{GCD}(x - y, y)$.

The recursive algorithm based on the above does not use multiplication, division or modulus, but for some inputs it is very slow. As an example, if the input is $x = 2^n$, $y = 2$, the algorithm makes 2^{n-1} recursive calls. The time complexity can be improved by observing that the repeated subtraction amounts to division, i.e., when $x > y$, $\text{GCD}(x, y) = \text{GCD}(y, x \bmod y)$, but this approach uses integer division which was explicitly disallowed in the problem statement.

Here is a fast solution, which is also based on recursion, but does not use general multiplication or division. Instead it uses case-analysis, specifically, the cases of one, two, or none of the numbers being even.

An example is illustrative. Suppose we were to compute the GCD of 24 and 300. Instead of repeatedly subtracting 24 from 300, we can observe that since both are even, the result is $2 \times \text{GCD}(12, 150)$. Dividing by 2 is a right shift by 1, so we do not need a general division operation. Since 12 and 150 are both even, $\text{GCD}(12, 150) = 2 \times \text{GCD}(6, 75)$. Since 75 is odd, the GCD of 6 and 75 is the same as the GCD of 3 and 75, since 2 cannot divide 75. The GCD of 3 and 75 is the GCD of 3 and $75 - 3 = 72$. Repeatedly applying the same logic, $\text{GCD}(3, 72) = \text{GCD}(3, 36) = \text{GCD}(3, 18) = \text{GCD}(3, 9) = \text{GCD}(3, 6) = \text{GCD}(3, 3) = 3$. This implies $\text{GCD}(24, 300) = 2 \times 2 \times 3 = 12$.

More generally, the base case is when the two arguments are equal, in which case the GCD is that value, e.g., $\text{GCD}(12, 12) = 12$, or one of the arguments is zero, in which case the other is the GCD, e.g., $\text{GCD}(0, 6) = 6$.

Otherwise, we check if none, one, or both numbers are even. If both are even, we compute the GCD of the halves of the original numbers, and return that result times 2; if exactly one is even, we half it, and return the GCD of the resulting pair; if both are odd, we subtract the smaller from the larger and return the GCD of the resulting pair. Multiplication by 2 is trivially implemented with a single left shift. Division by 2 is done with a single right shift.

```
def gcd(x, y):
    if x > y:
        return gcd(y, x)
    elif x == 0:
```

```

    return y
elif not x & 1 and not y & 1: # x and y are even.
    return gcd(x >> 1, y >> 1) << 1
elif not x & 1 and y & 1: # x is even, y is odd.
    return gcd(x >> 1, y)
elif x & 1 and not y & 1: # x is odd, y is even.
    return gcd(x, y >> 1)
return gcd(x, y - x) # Both x and y are odd.

```

Note that the last step leads to a recursive call with one even and one odd number. Consequently, in every two calls, we reduce the combined bit length of the two numbers by at least one, meaning that the time complexity is proportional to the sum of the number of bits in x and y , i.e., $O(\log x + \log y)$.

21.2 COMPUTE THE MAXIMUM PRODUCT OF ALL ENTRIES BUT ONE

Suppose you are given an array A of integers, and are asked to find the largest product that can be made by multiplying all but one of the entries in A . (You cannot use an entry more than once.) For example, if $A = \langle 3, 2, 5, 4 \rangle$, the result is $3 \times 5 \times 4 = 60$, if $A = \langle 3, 2, -1, 4 \rangle$, the result is $3 \times 2 \times 4 = 24$, and if $A = \langle 3, 2, -1, 4, -1, 6 \rangle$, the result is $3 \times -1 \times 4 \times -1 \times 6 = 72$.

One approach is to form the product P of all the elements, and then find the maximum of $P/A[i]$ over all i . This takes $n - 1$ multiplications (to form P) and n divisions (to compute each $P/A[i]$). Suppose because of finite precision considerations we cannot use a division-based approach; we can only use multiplications. The brute-force solution entails computing all n products of $n - 1$ elements; each such product takes $n - 2$ multiplications, i.e., $O(n^2)$ time complexity.

Given an array A of length n whose entries are integers, compute the largest product that can be made using $n - 1$ entries in A . You cannot use an entry more than once. Array entries may be positive, negative, or 0. Your algorithm cannot use the division operator, explicitly or implicitly.

Hint: Consider the products of the first $i - 1$ and the last $n - i$ elements. Alternatively, count the number of negative entries and zero entries.

Solution: The brute-force approach to compute $P/A[i]$ is to multiplying the entries appearing before i with those that appear after i . This leads to $n(n - 2)$ multiplications, since for each term $P/A[i]$ we need $n - 2$ multiplications.

Note that there is substantial overlap in computation when computing $P/A[i]$ and $P/A[i + 1]$. In particular, the product of the entries appearing before $i + 1$ is $A[i]$ times the product of the entries appearing before i . We can compute all products of entries before i with $n - 1$ multiplications, and all products of entries after i with $n - 1$ multiplications, and store these values in an array of prefix products, and an array of suffix products. The desired result then is the maximum prefix-suffix product. A slightly more space efficient approach is to build the suffix product array and then iterate forward through the array, using a single variable to compute the prefix products. This prefix product is multiplied with the corresponding suffix product and the maximum prefix-suffix product is updated if required.

```

def find_biggest_n_minus_one_product(A):
    # Builds suffix products.
    suffix_products = list(

```

```

    reversed(list(itertools.accumulate(reversed(A), operator.mul))))

# Finds the biggest product of (n - 1) numbers.
prefix_product, max_product = 1, float('-inf')
for i in range(len(A)):
    suffix_product = suffix_products[i + 1] if i + 1 < len(A) else 1
    max_product = max(max_product, prefix_product * suffix_product)
    prefix_product *= A[i]
return max_product

```

The time complexity is $O(n)$; the space complexity is $O(n)$, since the solution uses a single array of length n .

We now solve this problem in $O(n)$ time and $O(1)$ additional storage. The insight comes from the fact that if there are no negative entries, the maximum product comes from using all but the smallest element. (Note that this result is correct if the number of 0 entries is zero, one, or more.)

If the number of negative entries is odd, regardless of how many 0 entries and positive entries, the maximum product uses all entries except for the negative entry with the smallest absolute value, i.e., the least negative entry.

Going further, if the number of negative entries is even, the maximum product again uses all but the smallest nonnegative element, assuming the number of nonnegative entries is greater than zero. (This is correct, even in the presence of 0 entries.)

If the number of negative entries is even, and there are no nonnegative entries, the result must be negative. Since we want the largest product, we leave out the entry whose magnitude is largest, i.e., the greatest negative entry.

This analysis yields a two-stage algorithm. First, determine the applicable scenario, e.g., are there an even number of negative entries? Consequently, perform the actual multiplication to get the result.

```

def find_biggest_n_minus_one_product(A):
    number_of_negatives = 0
    least_nonnegative_idx = least_negative_idx = greatest_negative_idx = None

    # Identify the least negative, greatest negative, and least nonnegative
    # entries.
    for i, a in enumerate(A):
        if a < 0:
            number_of_negatives += 1
            if least_negative_idx is None or A[least_negative_idx] < a:
                least_negative_idx = i
            if greatest_negative_idx is None or a < A[greatest_negative_idx]:
                greatest_negative_idx = i
        else: # a >= 0.
            if least_nonnegative_idx is None or a < A[least_nonnegative_idx]:
                least_nonnegative_idx = i

    idx_to_skip = (least_negative_idx
                   if number_of_negatives % 2 else least_nonnegative_idx if
                   least_nonnegative_idx is not None else greatest_negative_idx)
    return functools.reduce(lambda product, a: product * a,
                           # Use a generator rather than list comprehension to
                           # avoid extra space.

```

```
(a for i, a in enumerate(A) if i != idx_to_skip), 1)
```

The algorithm performs a traversal of the array, with a constant amount of computation per entry, a nested conditional, followed by another traversal of the array, with a constant amount of computation per entry. Hence, the time complexity is $O(n) + O(1) + O(n) = O(n)$. The additional space complexity is $O(1)$, corresponding to the local variables.

Variante: Let A be as above. Compute an array B where $B[i]$ is the product of all elements in A except $A[i]$. You cannot use division. Your time complexity should be $O(n)$, and you can only use $O(1)$ additional space.

Variante: Let A be as above. Compute the maximum over the product of all triples of distinct elements of A .

Part IV

Notation and Index

Notation

To speak about notation as the only way that you can guarantee structure of course is already very suspect.

— E. S. PARKER

We use the following convention for symbols, unless the surrounding text specifies otherwise:

- A k -dimensional array
- L linked list or doubly linked list
- S set
- T tree
- G graph
- V set of vertices of a graph
- E set of edges of a graph

Symbolism	Meaning
$(d_{k-1} \dots d_0)_r$	radix- r representation of a number, e.g., $(1011)_2$
$\log_b x$	logarithm of x to the base b ; if b is not specified, $b = 2$
$ S $	cardinality of set S
$S \setminus T$	set difference, i.e., $S \cap T'$, sometimes written as $S - T$
$ x $	absolute value of x
$\lfloor x \rfloor$	greatest integer less than or equal to x
$\lceil x \rceil$	smallest integer greater than or equal to x
$\langle a_0, a_1, \dots, a_{n-1} \rangle$	sequence of n elements
$\sum_{R(k)} f(k)$	sum of all $f(k)$ such that relation $R(k)$ is true
$\min_{R(k)} f(k)$	minimum of all $f(k)$ such that relation $R(k)$ is true
$\max_{R(k)} f(k)$	maximum of all $f(k)$ such that relation $R(k)$ is true
$\sum_{k=a}^b f(k)$	shorthand for $\sum_{a \leq k \leq b} f(k)$
$\{a \mid R(a)\}$	set of all a such that the relation $R(a) = \text{true}$
$[l, r]$	closed interval: $\{x \mid l \leq x \leq r\}$
$[l, r)$	left-closed, right-open interval: $\{x \mid l \leq x < r\}$
$\{a, b, \dots\}$	well-defined collection of elements, i.e., a set
A_i or $A[i]$	the i th element of one-dimensional array A
$A[i, j]$	subarray of one-dimensional array A consisting of elements at indices i to j inclusive
$A[i][j]$	the element in i th row and j th column of 2D array A

$A[i_1, i_2][j_1, j_2]$	2D subarray of 2D array A consisting of elements from i_1 th to i_2 th rows and from j_1 th to j_2 th column, inclusive
$\binom{n}{k}$	binomial coefficient: number of ways of choosing k elements from a set of n items
$n!$	n -factorial, the product of the integers from 1 to n , inclusive
$\mathcal{O}(f(n))$	big-oh complexity of $f(n)$, asymptotic upper bound
$x \bmod y$	mod function
$x \oplus y$	bitwise-XOR function
$x \approx y$	x is approximately equal to y
null	pointer value reserved for indicating that the pointer does not refer to a valid address
$\emptyset$	empty set
∞	infinity: Informally, a number larger than any number.
$x \ll y$	much less than
$x \gg y$	much greater than
$\Rightarrow$	logical implication

Index of Terms

- 2D array, [65](#), [67](#), [76](#), [100](#), [101](#)
- 2D subarray, [101](#)
- $O(1)$ space, [12](#), [24](#), [29](#), [48](#), [65](#), [71](#), [97](#), [98](#)
- adjacency list, [74](#), [74](#)
- adjacency matrix, [74](#), [74](#)
- amortized, [21](#)
- amortized analysis, [42](#)
- approximation algorithm, [95](#)
- array, [4](#), [7](#), [7](#), [9](#), [10](#), [17](#), [21](#), [38–40](#), [42](#), [47](#), [48](#), [50](#), [51](#), [54](#), [65](#), [71](#), [77](#), [83](#), [96–98](#)
 - bit, *see* bit array
- BFS, [76](#), [76](#)
- binary search, [37–39](#), [48](#), [50](#), [70](#)
- binary search tree, *see* BST
- binary tree, [26–28](#), [28](#), [29–33](#), [54](#), [56](#), [57](#), [75](#)
 - complete, [29](#), [33](#)
 - full, [29](#)
 - height of, [29–32](#)
 - left-skewed, [29](#)
 - perfect, [29](#)
 - right-skewed, [29](#)
 - skewed, [29](#)
- binomial coefficient, [101](#)
- bit array, [62](#)
- breadth-first search, *see* BFS
- BST, [42](#), [43](#), [48](#), [54](#), [54](#), [56–58](#), [66](#)
 - height of, [54](#)
 - red-black tree, [54](#)
- caching, [83](#)
- case analysis, [95](#)
- central processing unit, *see* CPU
- chessboard, [60](#)
 - mutilated, [59](#)
- child, [29](#), [75](#)
- closed interval, [100](#)
- coloring, [77](#)
- complete binary tree, [29](#), [29](#), [30](#), [33](#)
 - height of, [29](#)
- connected component, [74](#), [74](#)
- connected directed graph, [74](#)
- connected undirected graph, [74](#)
- connected vertices, [74](#), [74](#)
- constraint, [57](#)
- counting sort, [48](#)
- CPU, [83](#)
- DAG, [73](#), [73](#)
- database, [83](#)
- deadlock, [78](#)
- decomposition, [82](#)
- deletion
 - from doubly linked lists, [25](#)
 - from hash tables, [42](#)
 - from heap, [33](#)
 - from max-heaps, [33](#)
- depth
 - of a node in a binary search tree, [57](#), [58](#)
 - of a node in a binary tree, [29](#), [29](#)
- depth-first search, *see* DFS
- deque, [25](#)
- dequeue, [25](#), [27](#)
- DFS, [76](#), [76](#)
- directed acyclic graph, *see* DAG, [74](#)
- directed graph, [73](#), *see also* directed acyclic graph, *see also* graph, [73](#), [74](#)
 - connected directed graph, [74](#)
 - weakly connected graph, [74](#)
- discovery time, [76](#)
- distributed memory, [78](#)
- distribution
 - of the numbers, [83](#)
- divide-and-conquer, [60](#), [63](#), [64](#)
- divisor
 - greatest common divisor, [95](#)
- double-ended queue, *see* deque
- doubly linked list, [17](#), [17](#), [25](#), [100](#)
 - deletion from, [25](#)
- DP, [63](#), [63](#), [65](#), [66](#)
- dynamic programming, *see* DP
- edge, [73](#), [74](#), [76](#)
- edge set, [75](#)
- elimination, [38](#)
- enqueue, [25](#), [27](#), [58](#)
- extract-max, [33](#)

- extract-min, 36
- Fibonacci number, 63
- finishing time, 76
- first-in, first-out, 21, *see also* queue, 25
- free tree, 75, 75
- full binary tree, 29, 29
- function
 - recursive, 59
- garbage collection, 78
- GCD, 95, 95
- graph, 73, *see also* directed graph, 73, 74, 75, *see also* tree
- graph modeling, 73
- graphical user interfaces, *see* GUI
- greatest common divisor, *see* GCD
- greedy, 69
- GUI, 78
- hash code, 42, 42, 43
- hash function, 42, 43
- hash table, 13, 19, 31, 42, 42, 43, 44, 66, 80
 - deletion from, 42
 - lookup of, 42
- head
 - of a deque, 25
 - of a linked list, 17, 19
- heap, 33, 48
 - deletion from, 33
 - max-heap, 33, 48
 - min-heap, 33, 48
- heapsort, 48
- height
 - of a binary search tree, 54
 - of a binary tree, 29, 29, 30–32
 - of a BST, 54
 - of a complete binary tree, 29
 - of a event rectangle, 51
 - of a perfect binary tree, 29
 - of a stack, 32
- I/O, 35
- in-place sort, 48
- invariant, 70, 70
- inverted index, 50
- last-in, first-out, 21, *see also* stack
- leaf, 29
- left child, 28, 29, 31, 57, 75
- left subtree, 28–30, 32, 54, 56, 57
- left-closed, right-open interval, 100
- level
 - of a tree, 29
- linked list, 17, 17, 100
- list, *see also* singly linked list, 17, 19, 21, 25, 42, 48
- livelock, 79
- load
 - of a hash table, 42
- lock
 - deadlock, 78
 - livelock, 79
- matching
 - of strings, 13
- matrix, 66, 74
 - adjacency, 74
 - multiplication of, 78
- matrix multiplication, 78
- max-heap, 33, 48
 - deletion from, 33
- merge sort, 36, 48
- min-heap, 33, 35, 36, 48, 80, 83
- multicore, 78
- mutilated chessboard, 59
- network, 78
 - network bandwidth, 83
- network bandwidth, 83
- node, 26–32, 43, 54, 56–58, 75
- ordered tree, 75, 75
- OS, 82
- overflow
 - integer, 38
- overlapping intervals, 52
- parallel algorithm, 95
- parallelism, 78, 79, 83
- parent-child relationship, 29, 75
- path, 73
- perfect binary tree, 29, 29, 30
 - height of, 29
- permutation
 - random, 12
- power set, 60, 60
- prime, 54
- queue, 21, 25, 25, 26, 33, 57
- quicksort, 9, 48, 65, 66
- race, 78
- radix sort, 48
- RAM, 83
- random access memory, *see* RAM
- random permutation, 12
- randomization, 42
- reachable, 73, 76
- recursion, 59, 60, 95
- recursive function, 59
- red-black tree, 54
- reduction, 15
- rehashing, 42
- right child, 28, 29, 31, 57, 75
- right subtree, 28–30, 32, 54, 56, 57
- rolling hash, 43
- root, 28, 29, 31, 32, 56–58, 75
- rooted tree, 75, 75

- shared memory, 78, 78
- Short Message Service, *see* SMS
- signature, 84
- singly linked list, 17, 17, 19
- sinks, 73
- SMS, 80
- sorting, 9, 38, 48, 48, 52, 83
 - counting sort, 48
 - heapsort, 48
 - in-place, 48
 - in-place sort, 48
 - merge sort, 36, 48
 - quicksort, 9, 48, 65, 66
 - radix sort, 48
 - stable, 48
 - stable sort, 48
- sources, 73
- spanning tree, 75, *see also* minimum spanning tree
- stable sort, 48
- stack, 21, 21, 22, 24
 - height of, 32
- starvation, 78
- string, 13, 13, 14, 15, 43, 44
- string matching, 13
- strongly connected directed graph, 74
- subarray, 9, 12, 40, 64–66
- subtree, 29, 32, 57, 58
- tail
 - of a deque, 25
 - of a linked list, 17, 19
- topological ordering, 73
- tree, 75, 75
 - binary, *see* binary tree
 - free, 75
 - ordered, 75
 - red-black, 54
 - rooted, 75
- triomino, 59, 60
- UI, 78
- undirected graph, 73, 74, 75
- vertex, 73, 73, 74–76, 100
 - connected, 74
- weakly connected graph, 74

Acknowledgments

Several of our friends, colleagues, and readers gave feedback. We would like to thank Taras Bobrovysky, Senthil Chellappan, Yi-Ting Chen, Monica Farkash, Dongbo Hu, Jing-Tang Keith Jang, Matthieu Jeanson, Gerson Kurz, Danyu Liu, Hari Mony, Shaun Phillips, Gayatri Ramachandran, Ulises Reyes, Kumud Sanwal, Tom Shiple, Ian Varley, Shaohua Wan, Don Wong, and Xiang Wu for their input.

I, Adnan Aziz, thank my teachers, friends, and students from IIT Kanpur, UC Berkeley, and UT Austin for having nurtured my passion for programming. I especially thank my friends Vineet Gupta, Tom Shiple, and Vigyan Singhal, and my teachers Robert Solovay, Robert Brayton, Richard Karp, Raimund Seidel, and Somenath Biswas, for all that they taught me. My coauthor, Tsung-Hsien Lee, brought a passion that was infectious and inspirational. My coauthor, Amit Prakash, has been a wonderful collaborator for many years—this book is a testament to his intellect, creativity, and enthusiasm. I look forward to a lifelong collaboration with both of them.

I, Tsung-Hsien Lee, would like to thank my coauthors, Adnan Aziz and Amit Prakash, who give me this once-in-a-life-time opportunity. I also thank my teachers Wen-Lian Hsu, Ren-Song Tsay, Biing-Feng Wang, and Ting-Chi Wang for having initiated and nurtured my passion for computer science in general, and algorithms in particular. I would like to thank my friends Cheng-Yi He, Da-Cheng Juan, Chien-Hsin Lin, and Chih-Chiang Yu, who accompanied me on the road of savoring the joy of programming contests; and Kuan-Chieh Chen, Chun-Cheng Chou, Ray Chuang, Wilson Hong, Wei-Lun Hung, Nigel Liang, and Huan-Kai Peng, who give me valuable feedback on this book. Last, I would like to thank all my friends and colleagues at Google, Facebook, National Tsing Hua University, and UT Austin for the brain-storming on puzzles; it is indeed my greatest honor to have known all of you.

I, Amit Prakash, have my coauthor and mentor, Adnan Aziz, to thank the most for this book. To a great extent, my problem solving skills have been shaped by Adnan. There have been occasions in life when I would not have made it through without his help. He is also the best possible collaborator I can think of for any intellectual endeavor. I have come to know Tsung-Hsien through working on this book. He has been a great coauthor. His passion and commitment to excellence can be seen everywhere in this book. Over the years, I have been fortunate to have had great teachers at IIT Kanpur and UT Austin. I would especially like to thank my teachers Scott Nettles, Vijaya Ramachandran, and Gustavo de Veciana. I would also like to thank my friends and colleagues at Google, Microsoft, IIT Kanpur, and UT Austin for many stimulating conversations and problem solving sessions. Finally, and most importantly, I want to thank my family who have been a constant source of support, excitement, and joy all my life and especially during the process of writing this book.

ADNAN AZIZ
TSUNG-HSIEN LEE
AMIT PRAKASH
April 10, 2017

*Palo Alto, California
Mountain View, California
Saratoga, California*